

Stage 05 — Feature Engineering y Dataset Predictivo

S05_feature_engineering_predictive_dataset.ipynb

Stage 05 no busca encontrar “el mejor indicador”, sino construir un dataset predictivo causal: $X_t + y_t$.

Esto está alineado con el workflow ML4T: primero se preparan datos, luego se construyen factores/features predictivos y recién después se diseñan/evalúan modelos. El libro marca esa secuencia explícitamente: preparar datos, hacer feature engineering, diseñar modelos, generar señales y luego evaluar/backtestear.

Objetivo

Construir un dataset donde cada fila represente una decisión posible en el tiempo t :

X_t = información disponible hasta t

y_t = target futuro asociado a t

Es decir:

features históricas + targets DIR/BAR/OPC preseleccionados

Pero con esta condición estricta:

Las features solo pueden mirar hacia atrás.

Los targets solo miran hacia adelante.

Esto es clave porque el libro remarca el riesgo de **look-ahead bias**, es decir, usar información histórica antes de que realmente estuviera disponible. La solución es validar los timestamps y asegurar que todo sea point-in-time.

Configuración del Entorno

Importación de librerías

```
In [2]: import sys
import re
#Instalación de librería pandas_market_calendars
#!{sys.executable} -m pip install -q pandas_market_calendars
#print("Librería instalada: pandas_market_calendars")

from functools import reduce
# Utilidades generales
from datetime import datetime, timedelta
import os
import glob
import requests
import warnings
warnings.filterwarnings('ignore')
```

```

# Manejo y procesamiento de datos
#import ta
import pandas as pd
import numpy as np
from tabulate import tabulate
import matplotlib.pyplot as plt
# Calendario de mercados
import pandas_market_calendars as mcal

#from ta.momentum import StochasticOscillator, ROCIndicator
#from ta.volatility import BollingerBands, AverageTrueRange

#from scipy.stats import spearmanr
from __future__ import annotations # permite type hints modernos (Python < 3.11
import json # para guardar el summary como JSON
import os # para leer variables de entorno
from pathlib import Path # manejo robusto de rutas
import pandas as pd

```

Rutas de archivos

```

In [3]: from pathlib import Path

# Buscar la raíz del proyecto
PROJECT_ROOT = Path.cwd()

while PROJECT_ROOT.name != "neural_profit_local":
    PROJECT_ROOT = PROJECT_ROOT.parent

print("Project root:", PROJECT_ROOT)

```

Project root: c:\Users\heguu\OneDrive\Escritorio\neural_profit_local

Carga de datasets

```

In [4]: MNQ_INTRADAY_PATH = PROJECT_ROOT / "data" / "02_mnq_intraday"
MNQ_INTRADAY_PATH.mkdir(parents=True, exist_ok=True)
MNQ_INTRADAY_PARQUET = MNQ_INTRADAY_PATH / "mnq_intraday.parquet"

MNQ_TARGETS_PATH = PROJECT_ROOT / "data" / "04_mnq_targets"
MNQ_TARGETS_PATH.mkdir(parents=True, exist_ok=True)

MNQ_TARGETS_DIR_PARQUET = MNQ_TARGETS_PATH / "mnq_targets_dir.parquet"
MNQ_TARGETS_BAR_PARQUET = MNQ_TARGETS_PATH / "mnq_targets_bar.parquet"
MNQ_TARGETS_OPC_PARQUET = MNQ_TARGETS_PATH / "mnq_targets_opc.parquet"

MNQ_TARGETS_ALL_PARQUET = MNQ_TARGETS_PATH / "mnq_targets_selected_final.parquet"

```

Archivos guardados: Targets seleccionados parquet:

c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\04_mnq_targets\mnq_targets_se

Targets seleccionados csv:

c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\04_mnq_targets\mnq_targets_se

Metadata seleccionados parquet:

c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\04_mnq_targets\mnq_targets_se

Metadata seleccionados csv:

c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\04_mnq_targets\mnq_targets_se

Decisión del stage json:

c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\04_mnq_targets\mnq_targets_st

```
In [5]: def load_mnq_parquet(dataset_path):
        os.path.exists(dataset_path)
        print("Archivo encontrado en disco. Cargando dataset local...")
        mnq_parquet = pd.read_parquet(dataset_path)
        return mnq_parquet
```

```
In [6]: mnq_intraday = load_mnq_parquet(MNQ_INTRADAY_PARQUET)
        mnq_targets_dir = load_mnq_parquet(MNQ_TARGETS_DIR_PARQUET)
        mnq_targets_bar = load_mnq_parquet(MNQ_TARGETS_BAR_PARQUET)
        mnq_targets_opc = load_mnq_parquet(MNQ_TARGETS_OPC_PARQUET)
```

Archivo encontrado en disco. Cargando dataset local...

Archivo encontrado en disco. Cargando dataset local...

Archivo encontrado en disco. Cargando dataset local...

Archivo encontrado en disco. Cargando dataset local...

```
In [7]: import pandas as pd

        datasets = {
            "mnq_intraday": mnq_intraday,
            "mnq_targets_dir": mnq_targets_dir,
            "mnq_targets_bar": mnq_targets_bar,
            "mnq_targets_opc": mnq_targets_opc,
        }

        print("VERIFICACIÓN DE ÍNDICES\n")

        for name, df in datasets.items():
            idx = df.index

            print(f"{name}")
            print(f"  Shape: {df.shape}")
            print(f"  Tipo de índice: {type(idx)}")
            print(f"  Es DatetimeIndex: {isinstance(idx, pd.DatetimeIndex)}")
            print(f"  Nombre del índice: {idx.name}")
            print(f"  Timezone: {idx.tz if isinstance(idx, pd.DatetimeIndex) else None}")
            print(f"  Ordenado creciente: {idx.is_monotonic_increasing}")
            print(f"  Duplicados en índice: {idx.duplicated().sum():,}")

            if isinstance(idx, pd.DatetimeIndex):
                print(f"  Fecha mínima: {idx.min()}")
                print(f"  Fecha máxima: {idx.max()}")
                print(f"  Frecuencia inferida: {pd.infer_freq(idx)}")

            print("-" * 70)
```

VERIFICACIÓN DE ÍNDICES

```
mnq_intraday
Shape: (1024062, 9)
Tipo de índice: <class 'pandas.DatetimeIndex'>
Es DatetimeIndex: True
Nombre del índice: datetime
Timezone: America/New_York
Ordenado creciente: True
Duplicados en índice: 0
Fecha mínima: 2020-01-02 04:30:00-05:00
Fecha máxima: 2026-04-17 16:00:00-04:00
Frecuencia inferida: None
```

```
-----
mnq_targets_dir
Shape: (1024062, 31)
Tipo de índice: <class 'pandas.DatetimeIndex'>
Es DatetimeIndex: True
Nombre del índice: datetime
Timezone: America/New_York
Ordenado creciente: True
Duplicados en índice: 0
Fecha mínima: 2020-01-02 04:30:00-05:00
Fecha máxima: 2026-04-17 16:00:00-04:00
Frecuencia inferida: None
```

```
-----
mnq_targets_bar
Shape: (1024062, 169)
Tipo de índice: <class 'pandas.DatetimeIndex'>
Es DatetimeIndex: True
Nombre del índice: datetime
Timezone: America/New_York
Ordenado creciente: True
Duplicados en índice: 0
Fecha mínima: 2020-01-02 04:30:00-05:00
Fecha máxima: 2026-04-17 16:00:00-04:00
Frecuencia inferida: None
```

```
-----
mnq_targets_opc
Shape: (1024062, 30)
Tipo de índice: <class 'pandas.DatetimeIndex'>
Es DatetimeIndex: True
Nombre del índice: datetime
Timezone: America/New_York
Ordenado creciente: True
Duplicados en índice: 0
Fecha mínima: 2020-01-02 04:30:00-05:00
Fecha máxima: 2026-04-17 16:00:00-04:00
Frecuencia inferida: None
```

```
In [8]: print("COMPARACIÓN EXACTA DE ÍNDICES\n")

base_name = "mnq_intraday"
base_index = datasets[base_name].index

for name, df in datasets.items():
    same_index = base_index.equals(df.index)
    print(f"{base_name} vs {name}: mismo índice exacto = {same_index}")
```

COMPARACIÓN EXACTA DE ÍNDICES

mnq_intraday vs mnq_intraday: mismo índice exacto = True
 mnq_intraday vs mnq_targets_dir: mismo índice exacto = True
 mnq_intraday vs mnq_targets_bar: mismo índice exacto = True
 mnq_intraday vs mnq_targets_opc: mismo índice exacto = True

```
In [9]: print("ALINEACIÓN POR INTERSECCIÓN DE ÍNDICES\n")

common_index = None

for name, df in datasets.items():
    if common_index is None:
        common_index = df.index
    else:
        common_index = common_index.intersection(df.index)

print(f"Cantidad de timestamps comunes: {len(common_index):,}")

for name, df in datasets.items():
    total_rows = len(df)
    common_rows = df.index.isin(common_index).sum()
    missing_rows = total_rows - common_rows

    print(f"{name}")
    print(f"  Filas totales: {total_rows:,}")
    print(f"  Filas alineables: {common_rows:,}")
    print(f"  Filas fuera de intersección: {missing_rows:,}")
    print(f"  % alineable: {common_rows / total_rows * 100:.2f}%")
    print(f"  -" * 70)
```

ALINEACIÓN POR INTERSECCIÓN DE ÍNDICES

Cantidad de timestamps comunes: 1,024,062

mnq_intraday

Filas totales: 1,024,062
 Filas alineables: 1,024,062
 Filas fuera de intersección: 0
 % alineable: 100.00%

 mnq_targets_dir

Filas totales: 1,024,062
 Filas alineables: 1,024,062
 Filas fuera de intersección: 0
 % alineable: 100.00%

 mnq_targets_bar

Filas totales: 1,024,062
 Filas alineables: 1,024,062
 Filas fuera de intersección: 0
 % alineable: 100.00%

 mnq_targets_opc

Filas totales: 1,024,062
 Filas alineables: 1,024,062
 Filas fuera de intersección: 0
 % alineable: 100.00%

```
In [10]: targets_preselected = ['dir_p50_h30', 'dir_p50_h60', 'dir_p50_h90', 'dir_p40_h60']
```

Función para ver información de dataset

```
In [11]: from __future__ import annotations

from dataclasses import dataclass
from typing import Any, Dict, Optional, Tuple

import pandas as pd

def mnq_dataset_info(
    df: pd.DataFrame,
    *,
    name: str = "mnq_raw",
    tz_assume_if_naive: Optional[str] = None, # ej: "UTC" o "America/New_York"
    day_def: str = "calendar", # "calendar" (fecha calendario) o "trading" (día)
) -> Dict[str, Any]:
    """
    Resume un dataset OHLCV con DatetimeIndex (ideal para mnq_raw).

    - Si el índice es tz-naive:
        - Si tz_assume_if_naive != None, lo localiza a esa tz.
        - Si no, reporta "tz-naive" (no se puede afirmar horario UTC).
    - Devuelve dict con métricas principales (y lo imprime bonito si se desea).
    """
    if not isinstance(df.index, pd.DatetimeIndex):
        raise TypeError(f"{name}: se requiere DatetimeIndex, recibido: {type(df.index)}")

    idx = df.index

    # --- timezone / UTC info ---
    tzinfo = idx.tz
    if tzinfo is None:
        tz_status = "tz-naive (sin zona horaria)"
        if tz_assume_if_naive:
            idx = idx.tz_localize(tz_assume_if_naive)
            tzinfo = idx.tz
            tz_status = f"localizado como {tzinfo}"
    else:
        tz_status = f"{tzinfo}"

    # --- rango temporal ---
    ts_min = idx.min()
    ts_max = idx.max()

    first_day = ts_min.date()
    last_day = ts_max.date()

    # --- días ---
    if day_def == "calendar":
        total_days = (pd.Timestamp(last_day) - pd.Timestamp(first_day)).days + 1
    elif day_def == "trading":
        total_days = idx.normalize().nunique()
    else:
        raise ValueError("day_def debe ser 'calendar' o 'trading'")

    # --- columnas ---
    columns = list(df.columns)
```

```

# --- checks útiles ---
n_rows = len(df)
n_cols = df.shape[1]
n_missing = int(df.isna().sum().sum())
missing_by_col = df.isna().sum().to_dict()
dup_index = int(idx.duplicated().sum())
is_monotonic = bool(idx.is_monotonic_increasing)

# Frecuencia estimada (puede fallar si hay huecos grandes)
freq = pd.infer_freq(idx[: min(50000, len(idx))]) # muestra grande pero aco

# Cobertura por día (min/max de hora del día, en tz del índice)
# (Útil para ver si es 24/7 o horario de sesión)
tod = pd.Series(idx.time)
# Convertimos time a minutos del día para resumen robusto
tod_minutes = pd.Series([t.hour * 60 + t.minute for t in tod])
typical_minute_min = int(tod_minutes.min())
typical_minute_max = int(tod_minutes.max())

# Rango promedio de filas por día (sólo días con datos)
rows_per_day = df.groupby(idx.normalize()).size()
rows_per_day_stats = {
    "days_with_data": int(rows_per_day.shape[0]),
    "rows_per_day_min": int(rows_per_day.min()),
    "rows_per_day_p50": float(rows_per_day.median()),
    "rows_per_day_max": int(rows_per_day.max()),
}

# Si hay tz, también mostramos rango en UTC
if tzinfo is not None:
    ts_min_utc = ts_min.tz_convert("UTC")
    ts_max_utc = ts_max.tz_convert("UTC")
    utc_range = (str(ts_min_utc), str(ts_max_utc))
    utc_note = "El índice está tz-aware; el horario UTC es inequívoco."
else:
    utc_range = None
    utc_note = "El índice es tz-naive; no se puede asegurar si está en UTC s

info: Dict[str, Any] = {
    "name": name,
    "shape": (n_rows, n_cols),
    "columns": columns,
    "index_type": type(df.index).__name__,
    "index_tz": tz_status,
    "utc_note": utc_note,
    "datetime_min": str(ts_min),
    "datetime_max": str(ts_max),
    "first_day": str(first_day),
    "last_day": str(last_day),
    "total_days": int(total_days),
    "day_definition": day_def,
    "utc_range_if_applicable": utc_range,
    "is_index_monotonic_increasing": is_monotonic,
    "duplicated_timestamps_in_index": dup_index,
    "inferred_freq_sample": freq,
    "missing_total_cells": n_missing,
    "missing_by_col": missing_by_col,
    "rows_per_day_stats": rows_per_day_stats,
    "time_of_day_minutes_range": {

```

```

        "min_minute_of_day": typical_minute_min,
        "max_minute_of_day": typical_minute_max,
    },
}
return info

def print_mnq_dataset_info(info: Dict[str, Any]) -> None:
    """Imprime el dict de mnq_dataset_info de forma ordenada."""
    print(f"Dataset: {info['name']}")
    print(f"Shape: {info['shape']}")
    print(f"Columns: {info['columns']}")
    print(f"Index: {info['index_type']} | TZ: {info['index_tz']}")
    print(f"Datetime min/max: {info['datetime_min']} -> {info['datetime_max']}")
    print(f"First/Last day: {info['first_day']} -> {info['last_day']}")
    print(f"Total days ({info['day_definition']}): {info['total_days']}")
    #print(f"Inferred freq (sample): {info['inferred_freq_sample']}")
    #print(f"Index monotonic increasing: {info['is_index_monotonic_increasing']}")
    #print(f"Duplicated timestamps in index: {info['duplicated_timestamps_in_index']}")
    #print(f"Missing total cells: {info['missing_total_cells']}")
    #print(f"Missing by col: {info['missing_by_col']}")
    #print(f"Rows/day stats: {info['rows_per_day_stats']}")
    print(f"Time-of-day range (minutes): {info['time_of_day_minutes_range']}")
    print(f"UTC note: {info['utc_note']}")
    if info["utc_range_if_applicable"] is not None:
        print(f"UTC range: {info['utc_range_if_applicable'][0]} -> {info['utc_

```

```
In [12]: info = mnq_dataset_info(mnq_intraday, name="mnq_intraday", tz_assume_if_naive=False)
print_mnq_dataset_info(info)
```

```

Dataset: mnq_intraday
Shape: (1024062, 9)
Columns: ['date', 'minute_of_day', 'regime_id', 'open', 'high', 'low', 'close',
'volume', 'contract']
Index: DatetimeIndex | TZ: America/New_York
Datetime min/max: 2020-01-02 04:30:00-05:00 -> 2026-04-17 16:00:00-04:00
First/Last day: 2020-01-02 -> 2026-04-17
Total days (trading): 1482
Time-of-day range (minutes): {'min_minute_of_day': 270, 'max_minute_of_day': 960}
UTC note: El índice está tz-aware; el horario UTC es inequívoco.
UTC range: 2020-01-02 09:30:00+00:00 -> 2026-04-17 20:00:00+00:00

```

1. Introducción y objetivo del Stage 05

En los stages anteriores se analizaron y preseleccionaron distintos targets operativos para el MNQ intradiario: DIR, BAR y OPC. El objetivo del Stage 05 es construir un dataset predictivo causal que permita evaluar, en el Stage 06, si existe señal predictiva real y estable sobre dichos targets.

Este stage no busca entrenar modelos ni evaluar rentabilidad. Tampoco busca seleccionar features en función de su performance futura. Su objetivo es construir correctamente X_t , asegurando que cada feature utilice únicamente información disponible hasta el instante t .

La unidad de observación será cada timestamp operativo elegible. Para cada timestamp se calcularán features históricas basadas en precio, volumen, volatilidad, momentum,

rango, comportamiento intradiario, régimen y contrato. Luego se asociarán los targets futuros previamente construidos.

Regla principal:

- Ninguna feature puede usar información posterior a t.
- Ningún target puede ser usado como feature.
- Ninguna transformación estadística puede ajustarse usando información futura.

2. Carga de outputs del Stage 04

En esta sección se cargan los archivos generados al cierre del Stage 04, donde se consolidaron los targets preseleccionados para las familias DIR, BAR y OPC.

Estos archivos no se utilizan para construir features, sino para definir las variables objetivo que serán asociadas posteriormente al dataset predictivo del Stage 05.

Se cargan tres elementos principales:

1. Dataset final de targets seleccionados.
2. Metadata descriptiva de los targets seleccionados.
3. Archivo JSON con la decisión final del Stage 04.

La regla principal es que los targets solo se usarán como variables objetivo `y_t`, nunca como variables predictoras `X_t`.

```
In [13]: # 2. Carga de outputs del Stage 04
# =====

from pathlib import Path
import json
import pandas as pd

# -----
# Rutas base del proyecto
# -----

PROJECT_ROOT = Path(r"c:\Users\heguu\OneDrive\Escritorio\neural_profit_local")

STAGE_04_PATH = PROJECT_ROOT / "data" / "04_mnq_targets"

# -----
# Archivos generados en Stage 04
# -----

TARGETS_SELECTED_PARQUET = STAGE_04_PATH / "mnq_targets_selected_final.parquet"
TARGETS_SELECTED_CSV = STAGE_04_PATH / "mnq_targets_selected_final.csv"

TARGETS_METADATA_PARQUET = STAGE_04_PATH / "mnq_targets_selected_metadata.parquet"
TARGETS_METADATA_CSV = STAGE_04_PATH / "mnq_targets_selected_metadata.csv"

TARGETS_STAGE_DECISION_JSON = STAGE_04_PATH / "mnq_targets_stage_decision.json"

# -----
```

```
# Verificación de existencia de archivos
# -----

stage_04_files = {
    "targets_selected_parquet": TARGETS_SELECTED_PARQUET,
    "targets_selected_csv": TARGETS_SELECTED_CSV,
    "targets_metadata_parquet": TARGETS_METADATA_PARQUET,
    "targets_metadata_csv": TARGETS_METADATA_CSV,
    "stage_decision_json": TARGETS_STAGE_DECISION_JSON,
}

print("Verificación de archivos del Stage 04:")
print("-" * 80)

for name, path in stage_04_files.items():
    print(f"{name:<30} | Existe: {path.exists()} | {path}")
```

Verificación de archivos del Stage 04:

```
-----
targets_selected_parquet      | Existe: True | c:\Users\heguu\OneDrive\Escritori
o\neural_profit_local\data\04_mnq_targets\mnq_targets_selected_final.parquet
targets_selected_csv          | Existe: True | c:\Users\heguu\OneDrive\Escritori
o\neural_profit_local\data\04_mnq_targets\mnq_targets_selected_final.csv
targets_metadata_parquet      | Existe: True | c:\Users\heguu\OneDrive\Escritori
o\neural_profit_local\data\04_mnq_targets\mnq_targets_selected_metadata.parquet
targets_metadata_csv          | Existe: True | c:\Users\heguu\OneDrive\Escritori
o\neural_profit_local\data\04_mnq_targets\mnq_targets_selected_metadata.csv
stage_decision_json           | Existe: True | c:\Users\heguu\OneDrive\Escritori
o\neural_profit_local\data\04_mnq_targets\mnq_targets_stage_decision.json
```

```
In [14]: # 2.1 Carga de targets seleccionados
# =====

df_targets_selected = pd.read_parquet(TARGETS_SELECTED_PARQUET)

print("Targets seleccionados cargados correctamente.")
print(f"Shape: {df_targets_selected.shape}")
print("\nColumnas:")
print(df_targets_selected.columns.tolist())

df_targets_selected.head()
```

Targets seleccionados cargados correctamente.

Shape: (1024062, 26)

Columnas:

```
['date', 'year', 'dataset_split', 'minute_of_day', 'regime_id', 'contract', 'dir_
p50_h30', 'dir_p50_h60', 'dir_p50_h90', 'dir_p40_h60', 'dir_p60_h60', 'bar_p50_h3
0_tp15_sl10', 'bar_p50_h60_tp15_sl10', 'bar_p50_h90_tp15_sl10', 'bar_p40_h60_tp15
_sl10', 'bar_p60_h60_tp15_sl10', 'opc_p50_h30_tp15_sl10_label', 'opc_p50_h60_tp15
_sl10_label', 'opc_p50_h90_tp15_sl10_label', 'opc_p40_h60_tp15_sl10_label', 'opc_
p60_h60_tp15_sl10_label', 'opc_p50_h30_tp15_sl10', 'opc_p50_h60_tp15_sl10', 'opc_
p50_h90_tp15_sl10', 'opc_p40_h60_tp15_sl10', 'opc_p60_h60_tp15_sl10']
```

Out[14]:

	date	year	dataset_split	minute_of_day	regime_id	contract	dir.
datetime							
2020-01-02 04:30:00-05:00	2020-01-02	2020	development_2020_2024	270	0	H20	
2020-01-02 04:31:00-05:00	2020-01-02	2020	development_2020_2024	271	0	H20	
2020-01-02 04:32:00-05:00	2020-01-02	2020	development_2020_2024	272	0	H20	
2020-01-02 04:33:00-05:00	2020-01-02	2020	development_2020_2024	273	0	H20	
2020-01-02 04:34:00-05:00	2020-01-02	2020	development_2020_2024	274	0	H20	

5 rows × 26 columns



In [15]:

```
# 2.3 Carga de decisión final del Stage 04
# =====

with TARGETS_STAGE_DECISION_JSON.open("r", encoding="utf-8") as f:
    stage_04_decision = json.load(f)

print("Decisión final del Stage 04 cargada correctamente.")
print("-" * 80)

for key, value in stage_04_decision.items():
    print(f"{key}: {value}")
```

Decisión final del Stage 04 cargada correctamente.

```
-----
stage: 04_targets
description: Targets preseleccionados para evaluación predictiva posterior.
main_operational_candidate: opc_p50_h30_tp15_sl10
main_dir_candidate: dir_p50_h30
main_bar_candidate: bar_p50_h30_tp15_sl10
selected_dir_targets: ['dir_p50_h30', 'dir_p50_h60', 'dir_p50_h90', 'dir_p40_h60', 'dir_p60_h60']
selected_bar_targets: ['bar_p50_h30_tp15_sl10', 'bar_p50_h60_tp15_sl10', 'bar_p50_h90_tp15_sl10', 'bar_p40_h60_tp15_sl10', 'bar_p60_h60_tp15_sl10']
selected_opc_targets: ['opc_p50_h30_tp15_sl10', 'opc_p50_h60_tp15_sl10', 'opc_p50_h90_tp15_sl10', 'opc_p40_h60_tp15_sl10', 'opc_p60_h60_tp15_sl10']
selected_opc_label_targets: ['opc_p50_h30_tp15_sl10_label', 'opc_p50_h60_tp15_sl10_label', 'opc_p50_h90_tp15_sl10_label', 'opc_p40_h60_tp15_sl10_label', 'opc_p60_h60_tp15_sl10_label']
families: [{'family_name': 'p50_h30', 'target_role': 'principal', 'dir_col': 'dir_p50_h30', 'bar_col': 'bar_p50_h30_tp15_sl10', 'opc_col': 'opc_p50_h30_tp15_sl10', 'opc_label_col': 'opc_p50_h30_tp15_sl10_label', 'percentile': 50, 'horizon': 30, 'tp_mult': 1.5, 'sl_mult': 1.0}, {'family_name': 'p50_h60', 'target_role': 'principal', 'dir_col': 'dir_p50_h60', 'bar_col': 'bar_p50_h60_tp15_sl10', 'opc_col': 'opc_p50_h60_tp15_sl10', 'opc_label_col': 'opc_p50_h60_tp15_sl10_label', 'percentile': 50, 'horizon': 60, 'tp_mult': 1.5, 'sl_mult': 1.0}, {'family_name': 'p50_h90', 'target_role': 'principal', 'dir_col': 'dir_p50_h90', 'bar_col': 'bar_p50_h90_tp15_sl10', 'opc_col': 'opc_p50_h90_tp15_sl10', 'opc_label_col': 'opc_p50_h90_tp15_sl10_label', 'percentile': 50, 'horizon': 90, 'tp_mult': 1.5, 'sl_mult': 1.0}, {'family_name': 'p40_h60', 'target_role': 'comparativo_activo', 'dir_col': 'dir_p40_h60', 'bar_col': 'bar_p40_h60_tp15_sl10', 'opc_col': 'opc_p40_h60_tp15_sl10', 'opc_label_col': 'opc_p40_h60_tp15_sl10_label', 'percentile': 40, 'horizon': 60, 'tp_mult': 1.5, 'sl_mult': 1.0}, {'family_name': 'p60_h60', 'target_role': 'comparativo_selectivo', 'dir_col': 'dir_p60_h60', 'bar_col': 'bar_p60_h60_tp15_sl10', 'opc_col': 'opc_p60_h60_tp15_sl10', 'opc_label_col': 'opc_p60_h60_tp15_sl10_label', 'percentile': 60, 'horizon': 60, 'tp_mult': 1.5, 'sl_mult': 1.0}]
methodological_rules: ['Targets contruidos sin cruzar días.', 'Thresholds calibrados con 2020-2024.', '2025-2026 reservado como final_test.', 'No usar targets DIR, BAR u OPC como features.', 'No usar predicciones de otros modelos como features en esta etapa.', 'Cada target debe evaluarse como problema predictivo independiente.']
```

```
In [16]: # 2.4 Identificación de columnas target
# =====

target_cols = [
    col for col in df_targets_selected.columns
    if (
        col.startswith("target_dir")
        or col.startswith("target_bar")
        or col.startswith("target_opc")
        or col.startswith("dir_")
        or col.startswith("bar_")
        or col.startswith("opc_")
    )
]

print("Columnas identificadas como targets:")
print("-" * 80)
print(f"Cantidad de targets encontrados: {len(target_cols)}")

for col in target_cols:
    print(col)
```

Columnas identificadas como targets:

Cantidad de targets encontrados: 20

dir_p50_h30
dir_p50_h60
dir_p50_h90
dir_p40_h60
dir_p60_h60
bar_p50_h30_tp15_sl10
bar_p50_h60_tp15_sl10
bar_p50_h90_tp15_sl10
bar_p40_h60_tp15_sl10
bar_p60_h60_tp15_sl10
opc_p50_h30_tp15_sl10_label
opc_p50_h60_tp15_sl10_label
opc_p50_h90_tp15_sl10_label
opc_p40_h60_tp15_sl10_label
opc_p60_h60_tp15_sl10_label
opc_p50_h30_tp15_sl10
opc_p50_h60_tp15_sl10
opc_p50_h90_tp15_sl10
opc_p40_h60_tp15_sl10
opc_p60_h60_tp15_sl10

```
In [17]: # 2.5 Validación básica del dataset de targets
# =====

print("Resumen básico de targets seleccionados")
print("=" * 80)

print(f"Filas:      {df_targets_selected.shape[0]:,}")
print(f"Columnas:   {df_targets_selected.shape[1]:,}")
print(f"Targets:     {len(target_cols):,}")

print("\nValores nulos por target:")
print(df_targets_selected[target_cols].isna().sum())

print("\nDistribución de clases por target:")
print("=" * 80)

for col in target_cols:
    print(f"\n{col}")
    print(df_targets_selected[col].value_counts(dropna=False).sort_index())
```

Resumen básico de targets seleccionados

=====

Filas: 1,024,062

Columnas: 26

Targets: 20

Valores nulos por target:

dir_p50_h30	44460
dir_p50_h60	88920
dir_p50_h90	133380
dir_p40_h60	88920
dir_p60_h60	88920
bar_p50_h30_tp15_sl10	494080
bar_p50_h60_tp15_sl10	519042
bar_p50_h90_tp15_sl10	541459
bar_p40_h60_tp15_sl10	427038
bar_p60_h60_tp15_sl10	610264
opc_p50_h30_tp15_sl10_label	44460
opc_p50_h60_tp15_sl10_label	88920
opc_p50_h90_tp15_sl10_label	133380
opc_p40_h60_tp15_sl10_label	88920
opc_p60_h60_tp15_sl10_label	88920
opc_p50_h30_tp15_sl10	44460
opc_p50_h60_tp15_sl10	88920
opc_p50_h90_tp15_sl10	133380
opc_p40_h60_tp15_sl10	88920
opc_p60_h60_tp15_sl10	88920

dtype: int64

Distribución de clases por target:

=====

dir_p50_h30
dir_p50_h30
-1 270082
0 449620
1 259900
<NA> 44460
Name: count, dtype: Int64

dir_p50_h60
dir_p50_h60
-1 258746
0 430122
1 246274
<NA> 88920
Name: count, dtype: Int64

dir_p50_h90
dir_p50_h90
-1 248018
0 408079
1 234585
<NA> 133380
Name: count, dtype: Int64

dir_p40_h60
dir_p40_h60
-1 300864
0 338118

```

1      296160
<NA>   88920
Name: count, dtype: Int64

```

```

dir_p60_h60
dir_p60_h60
-1      215897
0       521344
1      197901
<NA>    88920
Name: count, dtype: Int64

```

```

bar_p50_h30_tp15_sl10
bar_p50_h30_tp15_sl10
-1      30244
0       215684
1      284054
<NA>   494080
Name: count, dtype: Int64

```

```

bar_p50_h60_tp15_sl10
bar_p50_h60_tp15_sl10
-1      27972
0       213252
1      263796
<NA>   519042
Name: count, dtype: Int64

```

```

bar_p50_h90_tp15_sl10
bar_p50_h90_tp15_sl10
-1      28618
0       201183
1      252802
<NA>   541459
Name: count, dtype: Int64

```

```

bar_p40_h60_tp15_sl10
bar_p40_h60_tp15_sl10
-1      43645
0       217259
1      336120
<NA>   427038
Name: count, dtype: Int64

```

```

bar_p60_h60_tp15_sl10
bar_p60_h60_tp15_sl10
-1      17137
0       196901
1      199760
<NA>   610264
Name: count, dtype: Int64

```

```

opc_p50_h30_tp15_sl10_label
opc_p50_h30_tp15_sl10_label
LONG_SL      15085
LONG_TP      132636
NO_TRADE     665304
SHORT_SL     15159
SHORT_TP     151418
NaN          44460

```

Name: count, dtype: int64

```
opc_p50_h60_tp15_sl10_label
opc_p50_h60_tp15_sl10_label
LONG_SL      14172
LONG_TP      120384
NO_TRADE     643374
SHORT_SL     13800
SHORT_TP     143412
NaN          88920
```

Name: count, dtype: int64

```
opc_p50_h90_tp15_sl10_label
opc_p50_h90_tp15_sl10_label
LONG_SL      14518
LONG_TP      114341
NO_TRADE     609262
SHORT_SL     14100
SHORT_TP     138461
NaN          133380
```

Name: count, dtype: int64

```
opc_p40_h60_tp15_sl10_label
opc_p40_h60_tp15_sl10_label
LONG_SL      21891
LONG_TP      157597
NO_TRADE     555377
SHORT_SL     21754
SHORT_TP     178523
NaN          88920
```

Name: count, dtype: int64

```
opc_p60_h60_tp15_sl10_label
opc_p60_h60_tp15_sl10_label
LONG_SL      8768
LONG_TP      88560
NO_TRADE     718245
SHORT_SL     8369
SHORT_TP     111200
NaN          88920
```

Name: count, dtype: int64

```
opc_p50_h30_tp15_sl10
opc_p50_h30_tp15_sl10
0.0      665304
1.0      132636
2.0       15085
3.0      151418
4.0       15159
NaN       44460
```

Name: count, dtype: int64

```
opc_p50_h60_tp15_sl10
opc_p50_h60_tp15_sl10
0.0      643374
1.0      120384
2.0       14172
3.0      143412
4.0       13800
NaN       88920
```


Name: count, dtype: int64

opc_p50_h90_tp15_sl10

opc_p50_h90_tp15_sl10

0.0 609262

1.0 114341

2.0 14518

3.0 138461

4.0 14100

NaN 133380

Name: count, dtype: int64

opc_p40_h60_tp15_sl10

opc_p40_h60_tp15_sl10

0.0 555377

1.0 157597

2.0 21891

3.0 178523

4.0 21754

NaN 88920

Name: count, dtype: int64

opc_p60_h60_tp15_sl10

opc_p60_h60_tp15_sl10

0.0 718245

1.0 88560

2.0 8768

3.0 111200

4.0 8369

NaN 88920

Name: count, dtype: int64

3. Verificación con dataset MNQ_INTRADAY

```
In [18]: # 3. Verificación de alineación temporal entre MNQ y targets
# =====

import pandas as pd

# -----
# Copias de trabajo
# -----

df_ohlcv = mnq_intraday.copy()
df_targets = df_targets_selected.copy()

# -----
# Asegurar que ambos datasets tengan DatetimeIndex
# -----

def ensure_datetime_index(df, df_name):
    """
    Verifica que el DataFrame tenga índice datetime.
    Si existe una columna 'datetime', la usa como índice.
    """
    df = df.copy()
```

```

if isinstance(df.index, pd.DatetimeIndex):
    print(f"{df_name}: ya tiene DatetimeIndex.")
    return df

if "datetime" in df.columns:
    df["datetime"] = pd.to_datetime(df["datetime"])
    df = df.set_index("datetime")
    print(f"{df_name}: se usó la columna 'datetime' como índice.")
    return df

raise ValueError(f"{df_name}: no tiene DatetimeIndex ni columna 'datetime'.")

df_ohlcv = ensure_datetime_index(df_ohlcv, "mnq_intraday")
df_targets = ensure_datetime_index(df_targets, "df_targets_selected")

# -----
# Ordenar índices
# -----

df_ohlcv = df_ohlcv.sort_index()
df_targets = df_targets.sort_index()

# -----
# Información general de índices
# -----

print("\nResumen de índices")
print("=" * 80)

print(f"mnq_intraday index name:      {df_ohlcv.index.name}")
print(f"df_targets_selected index name: {df_targets.index.name}")

print(f"\nmnq_intraday index type:      {type(df_ohlcv.index)}")
print(f"df_targets_selected index type: {type(df_targets.index)}")

print(f"\nmnq_intraday timezone:        {df_ohlcv.index.tz}")
print(f"df_targets_selected timezone:   {df_targets.index.tz}")

print(f"\nmnq_intraday rango:            {df_ohlcv.index.min()} -> {df_ohlcv.index.max()}")
print(f"df_targets_selected rango:      {df_targets.index.min()} -> {df_targets.index.max()}")

print(f"\nmnq_intraday filas:            {len(df_ohlcv):,}")
print(f"df_targets_selected filas:      {len(df_targets):,}")

```

mnq_intraday: ya tiene DatetimeIndex.
df_targets_selected: ya tiene DatetimeIndex.

Resumen de índices

```
=====
mnq_intraday index name:      datetime
df_targets_selected index name: datetime

mnq_intraday index type:      <class 'pandas.DatetimeIndex'>
df_targets_selected index type: <class 'pandas.DatetimeIndex'>

mnq_intraday timezone:       America/New_York
df_targets_selected timezone: America/New_York

mnq_intraday rango:          2020-01-02 04:30:00-05:00 -> 2026-04-17 16:00:00-04:00
df_targets_selected rango:    2020-01-02 04:30:00-05:00 -> 2026-04-17 16:00:00-04:00

mnq_intraday filas:          1,024,062
df_targets_selected filas:    1,024,062
```

```
In [19]: # 3.1 Validación de unicidad, duplicados e intersección
# =====

print("Validación de índices")
print("=" * 80)

print(f"mnq_intraday índice único:      {df_ohlcv.index.is_unique}")
print(f"df_targets_selected índice único: {df_targets.index.is_unique}")

print(f"\nDuplicados mnq_intraday:      {df_ohlcv.index.duplicated().sum():,}")
print(f"Duplicados df_targets_selected: {df_targets.index.duplicated().sum():,}")

# Intersección entre ambos índices
common_index = df_ohlcv.index.intersection(df_targets.index)

print("\nIntersección temporal")
print("=" * 80)

print(f"Timestamps en común:      {len(common_index):,}")
print(f"% targets alineables con OHLCV: {len(common_index) / len(df_targets)}")
print(f"% OHLCV con target disponible: {len(common_index) / len(df_ohlcv) * 100}")

# Timestamps de targets que no existen en OHLCV
missing_targets_idx = df_targets.index.difference(df_ohlcv.index)

print("\nTargets sin OHLCV asociado")
print("=" * 80)
print(f"Cantidad: {len(missing_targets_idx):,}")

if len(missing_targets_idx) > 0:
    print("\nPrimeros timestamps faltantes:")
    print(missing_targets_idx[:20])
else:
    print("Todos los targets tienen timestamp presente en mnq_intraday.")
```

Validación de índices

```
=====
mnq_intraday índice único:      True
df_targets_selected índice único: True
```

```
Duplicados mnq_intraday:      0
Duplicados df_targets_selected: 0
```

Intersección temporal

```
=====
Timestamps en común:          1,024,062
% targets alineables con OHLCV: 100.00%
% OHLCV con target disponible: 100.00%
```

Targets sin OHLCV asociado

```
=====
Cantidad: 0
Todos los targets tienen timestamp presente en mnq_intraday.
```

```
In [20]: # 3.2 Prueba de unión por índice datetime
# =====

df_alignment_test = df_ohlc.join(
    df_targets,
    how="inner",
    rsuffix="_target"
)

print("Resultado de prueba de alineación")
print("=" * 80)

print(f"Filas OHLCV originales:      {len(df_ohlc):,}")
print(f"Filas targets originales:      {len(df_targets):,}")
print(f"Filas alineadas por datetime: {len(df_alignment_test):,}")

print("\nRango alineado:")
print(f"{df_alignment_test.index.min()} -> {df_alignment_test.index.max()}")

print("\nColumnas del dataset alineado:")
print(df_alignment_test.columns.tolist())

print("\nVista previa:")
print(df_alignment_test.head())
```

Resultado de prueba de alineación

=====

Filas OHLCV originales: 1,024,062

Filas targets originales: 1,024,062

Filas alineadas por datetime: 1,024,062

Rango alineado:

2020-01-02 04:30:00-05:00 -> 2026-04-17 16:00:00-04:00

Columnas del dataset alineado:

```
[ 'date', 'minute_of_day', 'regime_id', 'open', 'high', 'low', 'close', 'volume',
  'contract', 'date_target', 'year', 'dataset_split', 'minute_of_day_target', 'regime_id_target', 'contract_target', 'dir_p50_h30', 'dir_p50_h60', 'dir_p50_h90', 'dir_p40_h60', 'dir_p60_h60', 'bar_p50_h30_tp15_sl10', 'bar_p50_h60_tp15_sl10', 'bar_p50_h90_tp15_sl10', 'bar_p40_h60_tp15_sl10', 'bar_p60_h60_tp15_sl10', 'opc_p50_h30_tp15_sl10_label', 'opc_p50_h60_tp15_sl10_label', 'opc_p50_h90_tp15_sl10_label', 'opc_p40_h60_tp15_sl10_label', 'opc_p60_h60_tp15_sl10_label', 'opc_p50_h30_tp15_sl10', 'opc_p50_h60_tp15_sl10', 'opc_p50_h90_tp15_sl10', 'opc_p40_h60_tp15_sl10', 'opc_p60_h60_tp15_sl10' ]
```

Vista previa:

	date	minute_of_day	regime_id	open	\
datetime					
2020-01-02 04:30:00-05:00	2020-01-02	270	0	8813.25	
2020-01-02 04:31:00-05:00	2020-01-02	271	0	8812.50	
2020-01-02 04:32:00-05:00	2020-01-02	272	0	8812.50	
2020-01-02 04:33:00-05:00	2020-01-02	273	0	8812.00	
2020-01-02 04:34:00-05:00	2020-01-02	274	0	8810.75	

	high	low	close	volume	contract	\
datetime						
2020-01-02 04:30:00-05:00	8813.25	8812.50	8813.25	13	H20	
2020-01-02 04:31:00-05:00	8812.50	8811.00	8812.50	121	H20	
2020-01-02 04:32:00-05:00	8813.25	8811.75	8811.75	53	H20	
2020-01-02 04:33:00-05:00	8812.25	8810.50	8810.50	37	H20	
2020-01-02 04:34:00-05:00	8812.25	8810.50	8812.00	36	H20	

	date_target	...	opc_p50_h30_tp15_sl10_label	\
datetime		...		
2020-01-02 04:30:00-05:00	2020-01-02	...	NO_TRADE	
2020-01-02 04:31:00-05:00	2020-01-02	...	NO_TRADE	
2020-01-02 04:32:00-05:00	2020-01-02	...	NO_TRADE	
2020-01-02 04:33:00-05:00	2020-01-02	...	NO_TRADE	
2020-01-02 04:34:00-05:00	2020-01-02	...	NO_TRADE	

	opc_p50_h60_tp15_sl10_label	\
datetime		
2020-01-02 04:30:00-05:00	NO_TRADE	
2020-01-02 04:31:00-05:00	NO_TRADE	
2020-01-02 04:32:00-05:00	NO_TRADE	
2020-01-02 04:33:00-05:00	NO_TRADE	
2020-01-02 04:34:00-05:00	NO_TRADE	

	opc_p50_h90_tp15_sl10_label	\
datetime		
2020-01-02 04:30:00-05:00	NO_TRADE	
2020-01-02 04:31:00-05:00	NO_TRADE	
2020-01-02 04:32:00-05:00	NO_TRADE	
2020-01-02 04:33:00-05:00	NO_TRADE	
2020-01-02 04:34:00-05:00	NO_TRADE	

```

                                opc_p40_h60_tp15_sl10_label \
datetime
2020-01-02 04:30:00-05:00          NO_TRADE
2020-01-02 04:31:00-05:00          NO_TRADE
2020-01-02 04:32:00-05:00          NO_TRADE
2020-01-02 04:33:00-05:00          NO_TRADE
2020-01-02 04:34:00-05:00          NO_TRADE

                                opc_p60_h60_tp15_sl10_label  opc_p50_h30_tp15_sl10 \
datetime
2020-01-02 04:30:00-05:00          NO_TRADE          0.0
2020-01-02 04:31:00-05:00          NO_TRADE          0.0
2020-01-02 04:32:00-05:00          NO_TRADE          0.0
2020-01-02 04:33:00-05:00          NO_TRADE          0.0
2020-01-02 04:34:00-05:00          NO_TRADE          0.0

                                opc_p50_h60_tp15_sl10  opc_p50_h90_tp15_sl10 \
datetime
2020-01-02 04:30:00-05:00          0.0          0.0
2020-01-02 04:31:00-05:00          0.0          0.0
2020-01-02 04:32:00-05:00          0.0          0.0
2020-01-02 04:33:00-05:00          0.0          0.0
2020-01-02 04:34:00-05:00          0.0          0.0

                                opc_p40_h60_tp15_sl10  opc_p60_h60_tp15_sl10
datetime
2020-01-02 04:30:00-05:00          0.0          0.0
2020-01-02 04:31:00-05:00          0.0          0.0
2020-01-02 04:32:00-05:00          0.0          0.0
2020-01-02 04:33:00-05:00          0.0          0.0
2020-01-02 04:34:00-05:00          0.0          0.0

```

[5 rows x 35 columns]

```

In [21]: # 3.3 Validación de consistencia entre OHLCV y targets
# =====

df_ohlc = mnq_intraday.copy().sort_index()
df_targets = df_targets_selected.copy().sort_index()

checks = {
    "date": ("date", "date"),
    "minute_of_day": ("minute_of_day", "minute_of_day"),
    "regime_id": ("regime_id", "regime_id"),
    "contract": ("contract", "contract"),
}

print("Validación de columnas comunes entre OHLCV y targets")
print("=" * 80)

for name, (col_ohlc, col_target) in checks.items():
    if col_ohlc in df_ohlc.columns and col_target in df_targets.columns:
        same_values = (df_ohlc[col_ohlc].astype(str) == df_targets[col_target].astype(str))
        n_diff = (df_ohlc[col_ohlc].astype(str) != df_targets[col_target].astype(str)).sum()
        print(f"{name:<15} | iguales: {same_values.sum()} | diferencias: {n_diff}")
    else:
        print(f"{name:<15} | columna no encontrada en alguno de los datasets")

```

Validación de columnas comunes entre OHLCV y targets

```
=====
date           | iguales: True | diferencias: 0
minute_of_day  | iguales: True | diferencias: 0
regime_id      | iguales: True | diferencias: 0
contract       | iguales: True | diferencias: 0
```

```
In [22]: # 3.4 Join limpio entre OHLCV y targets seleccionados
# =====

# Columnas de targets
target_cols = [
    col for col in df_targets.columns
    if (
        col.startswith("dir_")
        or col.startswith("bar_")
        or col.startswith("opc_")
    )
]

# Metadata útil proveniente del Stage 04
metadata_cols = [
    col for col in ["year", "dataset_split"]
    if col in df_targets.columns
]

# Dataset de targets limpio
df_targets_clean = df_targets[metadata_cols + target_cols].copy()

# Join horizontal por datetime
df_stage05_base = df_ohlc.join(
    df_targets_clean,
    how="inner"
)

print("Dataset base Stage 05 creado correctamente")
print("=" * 80)
print(f"Filas OHLCV:           {len(df_ohlc):,}")
print(f"Filas targets:         {len(df_targets):,}")
print(f"Filas Stage 05 base:    {len(df_stage05_base):,}")
print(f"Columnas Stage 05:      {df_stage05_base.shape[1]:,}")

print("\nColumnas metadata:")
print(metadata_cols)

print("\nColumnas target:")
print(target_cols)

print("\nVista previa:")
print(df_stage05_base.head())
```

Dataset base Stage 05 creado correctamente

=====

Filas OHLCV: 1,024,062
 Filas targets: 1,024,062
 Filas Stage 05 base: 1,024,062
 Columnas Stage 05: 31

Columnas metadata:

['year', 'dataset_split']

Columnas target:

['dir_p50_h30', 'dir_p50_h60', 'dir_p50_h90', 'dir_p40_h60', 'dir_p60_h60', 'bar_p50_h30_tp15_sl10', 'bar_p50_h60_tp15_sl10', 'bar_p50_h90_tp15_sl10', 'bar_p40_h60_tp15_sl10', 'bar_p60_h60_tp15_sl10', 'opc_p50_h30_tp15_sl10_label', 'opc_p50_h60_tp15_sl10_label', 'opc_p50_h90_tp15_sl10_label', 'opc_p40_h60_tp15_sl10_label', 'opc_p60_h60_tp15_sl10_label', 'opc_p50_h30_tp15_sl10', 'opc_p50_h60_tp15_sl10', 'opc_p50_h90_tp15_sl10', 'opc_p40_h60_tp15_sl10', 'opc_p60_h60_tp15_sl10']

Vista previa:

	date	minute_of_day	regime_id	open	\
datetime					
2020-01-02 04:30:00-05:00	2020-01-02	270	0	8813.25	
2020-01-02 04:31:00-05:00	2020-01-02	271	0	8812.50	
2020-01-02 04:32:00-05:00	2020-01-02	272	0	8812.50	
2020-01-02 04:33:00-05:00	2020-01-02	273	0	8812.00	
2020-01-02 04:34:00-05:00	2020-01-02	274	0	8810.75	

	high	low	close	volume	contract	year	\
datetime							
2020-01-02 04:30:00-05:00	8813.25	8812.50	8813.25	13	H20	2020	
2020-01-02 04:31:00-05:00	8812.50	8811.00	8812.50	121	H20	2020	
2020-01-02 04:32:00-05:00	8813.25	8811.75	8811.75	53	H20	2020	
2020-01-02 04:33:00-05:00	8812.25	8810.50	8810.50	37	H20	2020	
2020-01-02 04:34:00-05:00	8812.25	8810.50	8812.00	36	H20	2020	

	...	opc_p50_h30_tp15_sl10_label	\
datetime	...		
2020-01-02 04:30:00-05:00	...	NO_TRADE	
2020-01-02 04:31:00-05:00	...	NO_TRADE	
2020-01-02 04:32:00-05:00	...	NO_TRADE	
2020-01-02 04:33:00-05:00	...	NO_TRADE	
2020-01-02 04:34:00-05:00	...	NO_TRADE	

	opc_p50_h60_tp15_sl10_label	\
datetime		
2020-01-02 04:30:00-05:00	NO_TRADE	
2020-01-02 04:31:00-05:00	NO_TRADE	
2020-01-02 04:32:00-05:00	NO_TRADE	
2020-01-02 04:33:00-05:00	NO_TRADE	
2020-01-02 04:34:00-05:00	NO_TRADE	

	opc_p50_h90_tp15_sl10_label	\
datetime		
2020-01-02 04:30:00-05:00	NO_TRADE	
2020-01-02 04:31:00-05:00	NO_TRADE	
2020-01-02 04:32:00-05:00	NO_TRADE	
2020-01-02 04:33:00-05:00	NO_TRADE	
2020-01-02 04:34:00-05:00	NO_TRADE	

	opc_p40_h60_tp15_sl10_label	\
datetime		
2020-01-02 04:30:00-05:00	NO_TRADE	
2020-01-02 04:31:00-05:00	NO_TRADE	
2020-01-02 04:32:00-05:00	NO_TRADE	
2020-01-02 04:33:00-05:00	NO_TRADE	
2020-01-02 04:34:00-05:00	NO_TRADE	

datetime		
2020-01-02 04:30:00-05:00	NO_TRADE	
2020-01-02 04:31:00-05:00	NO_TRADE	
2020-01-02 04:32:00-05:00	NO_TRADE	
2020-01-02 04:33:00-05:00	NO_TRADE	
2020-01-02 04:34:00-05:00	NO_TRADE	

	opc_p60_h60_tp15_sl10_label	opc_p50_h30_tp15_sl10	\
datetime			
2020-01-02 04:30:00-05:00	NO_TRADE	0.0	
2020-01-02 04:31:00-05:00	NO_TRADE	0.0	
2020-01-02 04:32:00-05:00	NO_TRADE	0.0	
2020-01-02 04:33:00-05:00	NO_TRADE	0.0	
2020-01-02 04:34:00-05:00	NO_TRADE	0.0	

	opc_p50_h60_tp15_sl10	opc_p50_h90_tp15_sl10	\
datetime			
2020-01-02 04:30:00-05:00	0.0	0.0	
2020-01-02 04:31:00-05:00	0.0	0.0	
2020-01-02 04:32:00-05:00	0.0	0.0	
2020-01-02 04:33:00-05:00	0.0	0.0	
2020-01-02 04:34:00-05:00	0.0	0.0	

	opc_p40_h60_tp15_sl10	opc_p60_h60_tp15_sl10
datetime		
2020-01-02 04:30:00-05:00	0.0	0.0
2020-01-02 04:31:00-05:00	0.0	0.0
2020-01-02 04:32:00-05:00	0.0	0.0
2020-01-02 04:33:00-05:00	0.0	0.0
2020-01-02 04:34:00-05:00	0.0	0.0

[5 rows x 31 columns]

```
In [23]: # 3.5 Validación final del dataset base Stage 05
# =====

print("Validación final df_stage05_base")
print("=" * 80)

print(f"Índice único:      {df_stage05_base.index.is_unique}")
print(f"Duplicados índice: {df_stage05_base.index.duplicated().sum():,}")
print(f"Rango temporal:     {df_stage05_base.index.min()} -> {df_stage05_base.index.max()}")
print(f"Shape:              {df_stage05_base.shape}")

print("\nNulos en columnas OHLCV base:")
ohlc_cols = ["open", "high", "low", "close", "volume"]
print(df_stage05_base[ohlc_cols].isna().sum())

print("\nNulos en targets:")
print(df_stage05_base[target_cols].isna().sum())

print("\nDistribución dataset_split:")
if "dataset_split" in df_stage05_base.columns:
    print(df_stage05_base["dataset_split"].value_counts(dropna=False).sort_index)
else:
    print("No existe columna dataset_split.")
```

Validación final df_stage05_base

```
=====
Índice único:      True
Duplicados índice: 0
Rango temporal:    2020-01-02 04:30:00-05:00 -> 2026-04-17 16:00:00-04:00
Shape:             (1024062, 31)

Nulos en columnas OHLCV base:
open      0
high      0
low       0
close     0
volume    0
dtype: int64

Nulos en targets:
dir_p50_h30      44460
dir_p50_h60      88920
dir_p50_h90     133380
dir_p40_h60      88920
dir_p60_h60      88920
bar_p50_h30_tp15_sl10  494080
bar_p50_h60_tp15_sl10  519042
bar_p50_h90_tp15_sl10  541459
bar_p40_h60_tp15_sl10  427038
bar_p60_h60_tp15_sl10  610264
opc_p50_h30_tp15_sl10_label  44460
opc_p50_h60_tp15_sl10_label  88920
opc_p50_h90_tp15_sl10_label  133380
opc_p40_h60_tp15_sl10_label  88920
opc_p60_h60_tp15_sl10_label  88920
opc_p50_h30_tp15_sl10      44460
opc_p50_h60_tp15_sl10      88920
opc_p50_h90_tp15_sl10     133380
opc_p40_h60_tp15_sl10      88920
opc_p60_h60_tp15_sl10      88920
dtype: int64

Distribución dataset_split:
dataset_split
development_2020_2024      818835
final_test_2025_2026      205227
Name: count, dtype: int64
```

```
In [24]: # 3.6 Definición de grupos de columnas
# =====

base_cols = [
    "date",
    "minute_of_day",
    "regime_id",
    "open",
    "high",
    "low",
    "close",
    "volume",
    "contract",
    "year",
    "dataset_split",
]
```

```
dir_target_cols = [
    col for col in df_stage05_base.columns
    if col.startswith("dir_")
]

bar_target_cols = [
    col for col in df_stage05_base.columns
    if col.startswith("bar_")
]

opc_target_label_cols = [
    col for col in df_stage05_base.columns
    if col.startswith("opc_") and col.endswith("_label")
]

opc_target_numeric_cols = [
    col for col in df_stage05_base.columns
    if col.startswith("opc_") and not col.endswith("_label")
]

target_cols = (
    dir_target_cols
    + bar_target_cols
    + opc_target_label_cols
    + opc_target_numeric_cols
)

print("Columnas base:")
print(base_cols)

print("\nTargets DIR:")
print(dir_target_cols)

print("\nTargets BAR:")
print(bar_target_cols)

print("\nTargets OPC label:")
print(opc_target_label_cols)

print("\nTargets OPC numéricos:")
print(opc_target_numeric_cols)

print("\nResumen:")
print(f"Base cols: {len(base_cols)}")
print(f"DIR targets: {len(dir_target_cols)}")
print(f"BAR targets: {len(bar_target_cols)}")
print(f"OPC label targets: {len(opc_target_label_cols)}")
print(f"OPC numeric targets: {len(opc_target_numeric_cols)}")
print(f"Total targets: {len(target_cols)}")
```

Columnas base:

```
['date', 'minute_of_day', 'regime_id', 'open', 'high', 'low', 'close', 'volume', 'contract', 'year', 'dataset_split']
```

Targets DIR:

```
['dir_p50_h30', 'dir_p50_h60', 'dir_p50_h90', 'dir_p40_h60', 'dir_p60_h60']
```

Targets BAR:

```
['bar_p50_h30_tp15_sl10', 'bar_p50_h60_tp15_sl10', 'bar_p50_h90_tp15_sl10', 'bar_p40_h60_tp15_sl10', 'bar_p60_h60_tp15_sl10']
```

Targets OPC label:

```
['opc_p50_h30_tp15_sl10_label', 'opc_p50_h60_tp15_sl10_label', 'opc_p50_h90_tp15_sl10_label', 'opc_p40_h60_tp15_sl10_label', 'opc_p60_h60_tp15_sl10_label']
```

Targets OPC numéricos:

```
['opc_p50_h30_tp15_sl10', 'opc_p50_h60_tp15_sl10', 'opc_p50_h90_tp15_sl10', 'opc_p40_h60_tp15_sl10', 'opc_p60_h60_tp15_sl10']
```

Resumen:

```
Base cols:          11
DIR targets:        5
BAR targets:        5
OPC label targets:  5
OPC numeric targets:5
Total targets:      20
```

4. Reglas de causalidad y separación entre features y targets

En esta sección se establecen las reglas de causalidad que deben cumplirse durante todo el Stage 05.

El dataset `df_stage05_base` contiene dos tipos de información:

1. Información observable en el instante t : OHLCV, volumen, régimen intradiario, contrato y variables temporales.
2. Targets futuros contruidos en el Stage 04: DIR, BAR y OPC.

Por lo tanto, antes de construir cualquier feature, se separan explícitamente las columnas que pueden utilizarse como entrada del modelo y las columnas que deben quedar prohibidas como features.

La regla principal es:

- Toda feature X_t debe construirse usando únicamente información disponible hasta el timestamp t .
- Ningún target puede usarse directa ni indirectamente como feature.

4.1 Supuesto operativo de tiempo

Se asume que cada fila del dataset representa una barra de 1 minuto ya cerrada.

Por lo tanto, para el timestamp t se consideran disponibles:

```
open_t
high_t
low_t
close_t
volume_t
minute_of_day_t
regime_id_t
contract_t
```

La decisión o predicción se interpreta como una señal generada después del cierre de la barra t , para evaluar un resultado futuro dentro de una ventana H posterior.

Bajo este supuesto, usar `close_t`, `high_t`, `low_t` y `volume_t` como base para features es válido. Lo que no está permitido es usar cualquier información de `t+1` en adelante para construir `X_t`.

4.2 Clasificación de columnas

```
In [ ]: # 4. Reglas de causalidad y separación entre features y targets
# =====

df = df_stage05_base.copy()

print("Dataset base recibido para Stage 05")
print("=" * 80)
print(f"Shape: {df.shape}")
print(f"Rango temporal: {df.index.min()} -> {df.index.max()}")
print(f"Índice único: {df.index.is_unique}")
```

Dataset base recibido para Stage 05

```
=====
Shape: (1024062, 31)
Rango temporal: 2020-01-02 04:30:00-05:00 -> 2026-04-17 16:00:00-04:00
Índice único: True
```

```
In [26]: # 4.1 Definición de columnas base, metadata y targets
# =====

# -----
# Columnas observables en t
# -----

observable_at_t_cols = [
    "open",
    "high",
    "low",
    "close",
    "volume",
    "minute_of_day",
    "regime_id",
    "contract",
]

# -----
# Columnas metadata
# No son targets, pero tampoco necesariamente features directas.
```

```
# Se usan para auditoría, splits, agrupaciones y análisis.
# -----
metadata_cols = [
    "date",
    "year",
    "dataset_split",
]

# -----
# Targets DIR
# -----
dir_target_cols = [
    col for col in df.columns
    if col.startswith("dir_")
]

# -----
# Targets BAR
# -----
bar_target_cols = [
    col for col in df.columns
    if col.startswith("bar_")
]

# -----
# Targets OPC Label
# -----
opc_target_label_cols = [
    col for col in df.columns
    if col.startswith("opc_") and col.endswith("_label")
]

# -----
# Targets OPC numéricos
# -----
opc_target_numeric_cols = [
    col for col in df.columns
    if col.startswith("opc_") and not col.endswith("_label")
]

# -----
# Lista completa de targets
# -----
target_cols = (
    dir_target_cols
    + bar_target_cols
    + opc_target_label_cols
    + opc_target_numeric_cols
)

print("Clasificación inicial de columnas")
print("=" * 80)

print(f"Columnas observables en t: {len(observable_at_t_cols)}")
print(observable_at_t_cols)

print(f"\nColumnas metadata: {len(metadata_cols)}")
print(metadata_cols)

print(f"\nTargets DIR: {len(dir_target_cols)}")
```

```

print(dir_target_cols)

print(f"\nTargets BAR: {len(bar_target_cols)}")
print(bar_target_cols)

print(f"\nTargets OPC label: {len(opc_target_label_cols)}")
print(opc_target_label_cols)

print(f"\nTargets OPC numéricos: {len(opc_target_numeric_cols)}")
print(opc_target_numeric_cols)

print(f"\nTotal targets: {len(target_cols)}")

```

Clasificación inicial de columnas

```

=====
Columnas observables en t: 8
['open', 'high', 'low', 'close', 'volume', 'minute_of_day', 'regime_id', 'contrac
t']

```

```

Columnas metadata: 3
['date', 'year', 'dataset_split']

```

```

Targets DIR: 5
['dir_p50_h30', 'dir_p50_h60', 'dir_p50_h90', 'dir_p40_h60', 'dir_p60_h60']

```

```

Targets BAR: 5
['bar_p50_h30_tp15_sl10', 'bar_p50_h60_tp15_sl10', 'bar_p50_h90_tp15_sl10', 'bar_
p40_h60_tp15_sl10', 'bar_p60_h60_tp15_sl10']

```

```

Targets OPC label: 5
['opc_p50_h30_tp15_sl10_label', 'opc_p50_h60_tp15_sl10_label', 'opc_p50_h90_tp15_
sl10_label', 'opc_p40_h60_tp15_sl10_label', 'opc_p60_h60_tp15_sl10_label']

```

```

Targets OPC numéricos: 5
['opc_p50_h30_tp15_sl10', 'opc_p50_h60_tp15_sl10', 'opc_p50_h90_tp15_sl10', 'opc_
p40_h60_tp15_sl10', 'opc_p60_h60_tp15_sl10']

```

```

Total targets: 20

```

4.3 Columnas prohibidas como features

Las columnas target quedan prohibidas como features.

También se excluyen columnas de control que no deberían alimentar directamente al modelo inicial, como `dataset_split` o `date`. Estas columnas se conservarán para auditoría y separación temporal, pero no como variables predictoras.

La columna `year` puede ser útil para análisis por año, pero no conviene usarla como feature inicial porque puede inducir al modelo a memorizar períodos históricos en lugar de aprender patrones intradiarios generalizables.

```

In [27]: # 4.2 Definición de columnas prohibidas como features
# =====

forbidden_as_features_cols = target_cols + [
    "date",
    "year",

```

```

    "dataset_split",
]

# Eliminar duplicados preservando orden
forbidden_as_features_cols = list(dict.fromkeys(forbidden_as_features_cols))

print("Columnas prohibidas como features")
print("=" * 80)
print(f"Cantidad: {len(forbidden_as_features_cols)}")

for col in forbidden_as_features_cols:
    print(col)

```

Columnas prohibidas como features

```

=====
Cantidad: 23
dir_p50_h30
dir_p50_h60
dir_p50_h90
dir_p40_h60
dir_p60_h60
bar_p50_h30_tp15_sl10
bar_p50_h60_tp15_sl10
bar_p50_h90_tp15_sl10
bar_p40_h60_tp15_sl10
bar_p60_h60_tp15_sl10
opc_p50_h30_tp15_sl10_label
opc_p50_h60_tp15_sl10_label
opc_p50_h90_tp15_sl10_label
opc_p40_h60_tp15_sl10_label
opc_p60_h60_tp15_sl10_label
opc_p50_h30_tp15_sl10
opc_p50_h60_tp15_sl10
opc_p50_h90_tp15_sl10
opc_p40_h60_tp15_sl10
opc_p60_h60_tp15_sl10
date
year
dataset_split

```

4.4 Columnas permitidas para construir features

Las features del Stage 05 deberán construirse únicamente a partir de columnas observables en t.

Esto incluye OHLCV, volumen, régimen intradiario, minuto del día y contrato.

A partir de estas columnas se podrán crear variables históricas mediante operaciones causales, por ejemplo:

```

returns pasados
rangos pasados
volatilidad rolling
momentum histórico
volumen relativo
posición del cierre dentro de rangos históricos
codificación de régimen
codificación de contrato

```


Toda operación rolling deberá usar información hasta t, nunca información futura.

```
In [28]: # 4.3 Columnas permitidas como fuente para features
# =====

feature_source_cols = [
    col for col in observable_at_t_cols
    if col in df.columns
]

print("Columnas permitidas como fuente para construir features")
print("=" * 80)
print(f"Cantidad: {len(feature_source_cols)}")

for col in feature_source_cols:
    print(col)

# Validación de columnas faltantes
missing_feature_source_cols = [
    col for col in observable_at_t_cols
    if col not in df.columns
]

print("\nColumnas esperadas no encontradas:")
print(missing_feature_source_cols)
```

Columnas permitidas como fuente para construir features

=====

Cantidad: 8

open

high

low

close

volume

minute_of_day

regime_id

contract

Columnas esperadas no encontradas:

[]

4.5 Auditoría de separación entre features y targets

```
In [29]: # 4.4 Auditoría de separación entre columnas permitidas y prohibidas
# =====

overlap_feature_target = sorted(set(feature_source_cols).intersection(set(target_cols)))
overlap_feature_forbidden = sorted(set(feature_source_cols).intersection(set(forbidden_as_features_cols)))

print("Auditoría de separación X_t / y_t")
print("=" * 80)

print(f"Overlap entre feature_source_cols y target_cols: {len(overlap_feature_target)}")
print(overlap_feature_target)

print(f"\nOverlap entre feature_source_cols y forbidden_as_features_cols: {len(overlap_feature_forbidden)}")
print(overlap_feature_forbidden)
```

```

if len(overlap_feature_target) == 0 and len(overlap_feature_forbidden) == 0:
    print("\nOK: no hay targets ni columnas prohibidas dentro de las fuentes de
else:
    print("\nATENCIÓN: revisar columnas superpuestas.")

```

Auditoría de separación X_t / y_t

=====

Overlap entre feature_source_cols y target_cols: 0

[]

Overlap entre feature_source_cols y forbidden_as_features_cols: 0

[]

OK: no hay targets ni columnas prohibidas dentro de las fuentes de features.

4.6 Registro formal de columnas del Stage 05

```

In [30]: # 4.5 Registro formal de columnas del Stage 05
# =====

stage05_column_registry = {
    "observable_at_t_cols": observable_at_t_cols,
    "feature_source_cols": feature_source_cols,
    "metadata_cols": metadata_cols,
    "dir_target_cols": dir_target_cols,
    "bar_target_cols": bar_target_cols,
    "opc_target_label_cols": opc_target_label_cols,
    "opc_target_numeric_cols": opc_target_numeric_cols,
    "target_cols": target_cols,
    "forbidden_as_features_cols": forbidden_as_features_cols,
}

print("Registro formal de columnas Stage 05")
print("=" * 80)

for group_name, cols in stage05_column_registry.items():
    print(f"{group_name:<30}: {len(cols)} columnas")

```

Registro formal de columnas Stage 05

=====

observable_at_t_cols	: 8 columnas
feature_source_cols	: 8 columnas
metadata_cols	: 3 columnas
dir_target_cols	: 5 columnas
bar_target_cols	: 5 columnas
opc_target_label_cols	: 5 columnas
opc_target_numeric_cols	: 5 columnas
target_cols	: 20 columnas
forbidden_as_features_cols	: 23 columnas

4.7 Conclusión del punto 4

Con esta separación queda establecida la regla de trabajo para el resto del Stage 05:

- Las features deberán construirse únicamente desde feature_source_cols.
- Los targets definidos en target_cols solo podrán usarse como variables objetivo y_t.

- Las columnas de metadata se conservarán para auditoría, particiones temporales y análisis por grupo, pero no como predictores iniciales.
- Las columnas prohibidas no podrán ingresar al set X_t .

A partir de este punto, cualquier nueva feature deberá ser revisada bajo dos condiciones:

1. Debe estar calculada únicamente con información disponible hasta t .
2. No debe derivarse de ningún target DIR, BAR u OPC.

5. Construcción de features causales base

En esta sección se construyen las primeras features predictivas del Stage 05.

Todas las variables se calculan a partir de información observable hasta el timestamp t :
open, high, low, close, volume, minute_of_day y regime_id.

La columna contract se conserva únicamente como metadata de control, pero no se utiliza para generar features en esta primera versión.

No se utiliza ningún target DIR, BAR u OPC como entrada. Tampoco se calculan ventanas históricas cruzando días de trading.

Esta primera versión busca generar un set base de features simples, interpretables y causalmente válidas.

5.1 Preparación del dataset de trabajo

```
In [31]: # 5. Construcción de features causales base
# =====

import numpy as np
import pandas as pd

df_feat = df_stage05_base.copy().sort_index()

print("Dataset recibido para feature engineering")
print("=" * 80)
print(f"Shape inicial: {df_feat.shape}")
print(f"Rango temporal: {df_feat.index.min()} -> {df_feat.index.max()}")
print(f"Índice único: {df_feat.index.is_unique}")
```

Dataset recibido para feature engineering

```
=====
Shape inicial: (1024062, 31)
Rango temporal: 2020-01-02 04:30:00-05:00 -> 2026-04-17 16:00:00-04:00
Índice único: True
```

5.2 Features básicas de vela en t

Estas variables usan únicamente la barra cerrada en t .

```
In [32]: # 5.1 Features básicas de vela en t
# =====

eps = 1e-9

df_feat["bar_range_pts"] = df_feat["high"] - df_feat["low"]
df_feat["bar_body_pts"] = df_feat["close"] - df_feat["open"]
df_feat["bar_body_abs_pts"] = df_feat["bar_body_pts"].abs()

df_feat["upper_wick_pts"] = df_feat["high"] - df_feat[["open", "close"]].max(axis=1)
df_feat["lower_wick_pts"] = df_feat[["open", "close"]].min(axis=1) - df_feat["close"]

df_feat["close_to_high_pts"] = df_feat["high"] - df_feat["close"]
df_feat["close_to_low_pts"] = df_feat["close"] - df_feat["low"]

# Versiones normalizadas contra close_t
df_feat["bar_range_pct"] = df_feat["bar_range_pts"] / (df_feat["close"] + eps)
df_feat["bar_body_pct"] = df_feat["bar_body_pts"] / (df_feat["close"] + eps)
df_feat["bar_body_abs_pct"] = df_feat["bar_body_abs_pts"] / (df_feat["close"] + eps)

print("Features básicas de vela creadas.")
print(f"Shape actual: {df_feat.shape}")
```

Features básicas de vela creadas.

Shape actual: (1024062, 41)

5.3 Funciones auxiliares para cálculos intradiarios causales

Estas funciones evitan que las ventanas históricas crucen de un día a otro.

```
In [33]: # 5.2 Funciones auxiliares para cálculos causales intradiarios
# =====

def group_shift(df, col, periods):
    """
    Shift dentro de cada día de trading.
    Evita que el último dato de un día contamine el primer dato del día siguiente.
    """
    return df.groupby("date", group_keys=False)[col].shift(periods)

def group_rolling(df, col, window, func, min_periods=None):
    """
    Rolling window dentro de cada día de trading.
    Evita cruzar jornadas.
    """
    if min_periods is None:
        min_periods = window

    return (
        df.groupby("date", group_keys=False)[col]
        .transform(lambda s: getattr(
            s.rolling(window=window, min_periods=min_periods), func
        ))
    )
```

```
print("Funciones auxiliares creadas.")
```

Funciones auxiliares creadas.

5.4 Returns y momentum históricos

```
In [34]: # 5.3 Returns y momentum históricos
# =====

lags = [1, 3, 5, 10, 15, 30, 60, 90]

for lag in lags:
    shifted_close = group_shift(df_feat, "close", lag)

    # Movimiento histórico en puntos
    df_feat[f"ret_pts_{lag}m"] = df_feat["close"] - shifted_close

    # Retorno histórico porcentual
    df_feat[f"ret_pct_{lag}m"] = (df_feat["close"] / shifted_close) - 1

print("Features de returns y momentum creadas.")
print(f"Cantidad de lags: {len(lags)}")
print(f"Shape actual: {df_feat.shape}")
```

Features de returns y momentum creadas.

Cantidad de lags: 8

Shape actual: (1024062, 57)

5.5 Volatilidad y rango histórico

```
In [35]: # 5.4 Volatilidad y rango histórico
# =====

windows = [5, 10, 15, 30, 60, 90]

for w in windows:
    # Volatilidad rolling de returns de 1 minuto
    df_feat[f"vol_ret_1m_{w}m"] = group_rolling(df_feat, "ret_pct_1m", w, "std")

    # Estadísticos rolling del rango de vela
    df_feat[f"bar_range_mean_{w}m"] = group_rolling(df_feat, "bar_range_pts", w,
    df_feat[f"bar_range_std_{w}m"] = group_rolling(df_feat, "bar_range_pts", w,

    # Máximo y mínimo histórico dentro de la ventana
    df_feat[f"rolling_high_{w}m"] = group_rolling(df_feat, "high", w, "max")
    df_feat[f"rolling_low_{w}m"] = group_rolling(df_feat, "low", w, "min")

    # Rango histórico
    df_feat[f"rolling_range_pts_{w}m"] = (
        df_feat[f"rolling_high_{w}m"] - df_feat[f"rolling_low_{w}m"]
    )

    df_feat[f"rolling_range_pct_{w}m"] = (
        df_feat[f"rolling_range_pts_{w}m"] / (df_feat["close"] + eps)
    )

print("Features de volatilidad y rango histórico creadas.")
```

```
print(f"Cantidad de ventanas: {len(windows)}")
print(f"Shape actual: {df_feat.shape}")
```

Features de volatilidad y rango histórico creadas.
 Cantidad de ventanas: 6
 Shape actual: (1024062, 99)

5.6 Posición del precio dentro del rango reciente

```
In [36]: # 5.5 Posición del close dentro del rango reciente
# =====

for w in windows:
    range_den = df_feat[f"rolling_range_pts_{w}m"].replace(0, np.nan)

    df_feat[f"close_pos_range_{w}m"] = (
        (df_feat["close"] - df_feat[f"rolling_low_{w}m"]) / range_den
    )

    df_feat[f"close_dist_high_{w}m"] = (
        (df_feat[f"rolling_high_{w}m"] - df_feat["close"]) / range_den
    )

    df_feat[f"close_dist_low_{w}m"] = (
        (df_feat["close"] - df_feat[f"rolling_low_{w}m"]) / range_den
    )

print("Features de posición dentro del rango reciente creadas.")
print(f"Shape actual: {df_feat.shape}")
```

Features de posición dentro del rango reciente creadas.
 Shape actual: (1024062, 117)

5.7 Features de volumen

```
In [37]: # 5.6 Features de volumen
# =====

df_feat["volume_log"] = np.log1p(df_feat["volume"])

for w in windows:
    df_feat[f"volume_mean_{w}m"] = group_rolling(df_feat, "volume", w, "mean")
    df_feat[f"volume_std_{w}m"] = group_rolling(df_feat, "volume", w, "std")

    df_feat[f"volume_rel_{w}m"] = (
        df_feat["volume"] / (df_feat[f"volume_mean_{w}m"] + eps)
    )

    df_feat[f"volume_zscore_{w}m"] = (
        (df_feat["volume"] - df_feat[f"volume_mean_{w}m"])
        / (df_feat[f"volume_std_{w}m"] + eps)
    )

print("Features de volumen creadas.")
print(f"Shape actual: {df_feat.shape}")
```

Features de volumen creadas.
 Shape actual: (1024062, 142)

5.8 Features temporales e intradiarias

No se generan features a partir de contract.

```
In [38]: # 5.7 Features temporales e intradiarias
# =====

# Día de la semana
df_feat["day_of_week"] = df_feat.index.dayofweek

# Hora y minuto
df_feat["hour"] = df_feat.index.hour
df_feat["minute"] = df_feat.index.minute

# Codificación cíclica del minuto operativo
minute_min = df_feat["minute_of_day"].min()
minute_max = df_feat["minute_of_day"].max()
minute_range = minute_max - minute_min

df_feat["minute_of_day_sin"] = np.sin(
    2 * np.pi * (df_feat["minute_of_day"] - minute_min) / minute_range
)

df_feat["minute_of_day_cos"] = np.cos(
    2 * np.pi * (df_feat["minute_of_day"] - minute_min) / minute_range
)

print("Features temporales e intradiarias creadas.")
print(f"Shape actual: {df_feat.shape}")
```

Features temporales e intradiarias creadas.

Shape actual: (1024062, 147)

5.9 Definición preliminar de features creadas

```
In [39]: # 5.8 Identificación de features creadas en Stage 05
# =====

original_cols = df_stage05_base.columns.tolist()

new_feature_cols = [
    col for col in df_feat.columns
    if col not in original_cols
]

print("Features nuevas creadas")
print("=" * 80)
print(f"Cantidad: {len(new_feature_cols)}")

for col in new_feature_cols:
    print(col)
```

Features nuevas creadas

```
=====
Cantidad: 116
bar_range_pts
bar_body_pts
bar_body_abs_pts
upper_wick_pts
lower_wick_pts
close_to_high_pts
close_to_low_pts
bar_range_pct
bar_body_pct
bar_body_abs_pct
ret_pts_1m
ret_pct_1m
ret_pts_3m
ret_pct_3m
ret_pts_5m
ret_pct_5m
ret_pts_10m
ret_pct_10m
ret_pts_15m
ret_pct_15m
ret_pts_30m
ret_pct_30m
ret_pts_60m
ret_pct_60m
ret_pts_90m
ret_pct_90m
vol_ret_1m_5m
bar_range_mean_5m
bar_range_std_5m
rolling_high_5m
rolling_low_5m
rolling_range_pts_5m
rolling_range_pct_5m
vol_ret_1m_10m
bar_range_mean_10m
bar_range_std_10m
rolling_high_10m
rolling_low_10m
rolling_range_pts_10m
rolling_range_pct_10m
vol_ret_1m_15m
bar_range_mean_15m
bar_range_std_15m
rolling_high_15m
rolling_low_15m
rolling_range_pts_15m
rolling_range_pct_15m
vol_ret_1m_30m
bar_range_mean_30m
bar_range_std_30m
rolling_high_30m
rolling_low_30m
rolling_range_pts_30m
rolling_range_pct_30m
vol_ret_1m_60m
bar_range_mean_60m
bar_range_std_60m
```


rolling_high_60m
rolling_low_60m
rolling_range_pts_60m
rolling_range_pct_60m
vol_ret_1m_90m
bar_range_mean_90m
bar_range_std_90m
rolling_high_90m
rolling_low_90m
rolling_range_pts_90m
rolling_range_pct_90m
close_pos_range_5m
close_dist_high_5m
close_dist_low_5m
close_pos_range_10m
close_dist_high_10m
close_dist_low_10m
close_pos_range_15m
close_dist_high_15m
close_dist_low_15m
close_pos_range_30m
close_dist_high_30m
close_dist_low_30m
close_pos_range_60m
close_dist_high_60m
close_dist_low_60m
close_pos_range_90m
close_dist_high_90m
close_dist_low_90m
volume_log
volume_mean_5m
volume_std_5m
volume_rel_5m
volume_zscore_5m
volume_mean_10m
volume_std_10m
volume_rel_10m
volume_zscore_10m
volume_mean_15m
volume_std_15m
volume_rel_15m
volume_zscore_15m
volume_mean_30m
volume_std_30m
volume_rel_30m
volume_zscore_30m
volume_mean_60m
volume_std_60m
volume_rel_60m
volume_zscore_60m
volume_mean_90m
volume_std_90m
volume_rel_90m
volume_zscore_90m
day_of_week
hour
minute
minute_of_day_sin
minute_of_day_cos

5.10 Auditoría inicial de features

```
In [40]: # 5.9 Auditoría inicial de features creadas
# =====

print("Auditoría de features nuevas")
print("=" * 80)

numeric_new_features = df_feat[new_feature_cols].select_dtypes(include=[np.number])

# Nulos
na_counts = df_feat[new_feature_cols].isna().sum()
na_counts = na_counts[na_counts > 0].sort_values(ascending=False)

print("\nFeatures con NaN:")
print(na_counts)

# Infinitos
inf_counts = np.isinf(df_feat[numeric_new_features]).sum()
inf_counts = inf_counts[inf_counts > 0].sort_values(ascending=False)

print("\nFeatures con infinitos:")
print(inf_counts)

# Columnas constantes
constant_cols = [
    col for col in new_feature_cols
    if df_feat[col].nunique(dropna=True) <= 1
]

print("\nFeatures constantes:")
print(constant_cols)

print("\nShape final con features:")
print(df_feat.shape)
```

Auditoría de features nuevas

=====

Features con NaN:

```
ret_pct_90m      133380
ret_pts_90m      133380
vol_ret_1m_90m   133380
rolling_high_90m 131898
close_dist_high_90m 131898
```

...

```
volume_zscore_5m 5928
ret_pct_3m       4446
ret_pts_3m       4446
ret_pct_1m       1482
ret_pts_1m       1482
Length: 100, dtype: int64
```

Features con infinitos:

Series([], dtype: int64)

Features constantes:

[]

Shape final con features:

(1024062, 147)

```
In [41]: # 5.10 Dataset resultante del punto 5
# =====

df_stage05_features = df_feat.copy()

print("Dataset con features causales base creado")
print("=" * 80)
print(f"Shape df_stage05_base:      {df_stage05_base.shape}")
print(f"Shape df_stage05_features: {df_stage05_features.shape}")
print(f"Features nuevas:             {len(new_feature_cols)}")

print("\nRango temporal:")
print(f"{df_stage05_features.index.min()} -> {df_stage05_features.index.max()}")

print("\nNota:")
print("La columna 'contract' se conserva en el dataset, pero no fue utilizada pa
```

Dataset con features causales base creado

=====

```
Shape df_stage05_base:      (1024062, 31)
Shape df_stage05_features: (1024062, 147)
Features nuevas:           116
```

Rango temporal:

2020-01-02 04:30:00-05:00 -> 2026-04-17 16:00:00-04:00

Nota:

La columna 'contract' se conserva en el dataset, pero no fue utilizada para crear features.

En este punto se construyó una primera versión de features causales basada en información observable hasta t.

Las features incluyen:

- estructura de la vela actual,
- retornos históricos,
- momentum,
- volatilidad rolling,
- rangos históricos,
- posición del precio dentro del rango reciente,
- volumen relativo,
- variables temporales e intradiarias.

No se construyeron features a partir de la columna contract. Esta variable se conserva únicamente como metadata de control para auditorías o análisis posteriores.

Todas las ventanas rolling se calcularon dentro de cada día de trading, evitando contaminación entre jornadas.

Los NaN iniciales generados por lags y ventanas rolling son esperados y serán tratados posteriormente durante la construcción del dataset predictivo final.

- 6. Validación de calidad de features
 - 6.1 NaN e infinitos
 - 6.2 Features constantes o casi constantes
 - 6.3 Correlación entre features
 - 6.4 Selección preliminar de features válidas
- 7. Análisis univariado de señal
 - 7.1 Spearman/IC para DIR y BAR
 - 7.2 Mutual Information para DIR, BAR y OPC
 - 7.3 Señal por régimen intradiario
 - 7.4 Señal por año dentro de development_2020_2024

6. Validación de calidad y diagnóstico de redundancia de features

En esta sección se valida la calidad inicial de las features construidas en el Stage 05.

El objetivo de este punto no es medir si las features predicen los targets, sino verificar si las variables creadas son utilizables desde el punto de vista estructural.

Se revisan:

- valores nulos,
- valores infinitos,
- features constantes,
- features casi constantes,
- features con demasiados NaN,
- correlación elevada entre features.

La correlación entre features se utiliza solo como diagnóstico de redundancia. En este punto no se eliminarán features por correlación, ya que todavía no se evaluó su señal predictiva ni su utilidad en modelos.

Las decisiones de calidad se toman usando únicamente el período development_2020_2024, para no contaminar el período final de evaluación 2025_2026.

6.1 Preparación del dataset para auditoría

```
In [54]: # 6. Validación de calidad y diagnóstico de redundancia
# =====

import numpy as np
import pandas as pd

df_quality = df_stage05_features.copy().sort_index()

print("Dataset recibido para validación de calidad")
print("=" * 80)
print(f"Shape: {df_quality.shape}")
print(f"Rango temporal: {df_quality.index.min()} -> {df_quality.index.max()}")
print(f"Índice único: {df_quality.index.is_unique}")
```

Dataset recibido para validación de calidad

```
=====
Shape: (1024062, 147)
Rango temporal: 2020-01-02 04:30:00-05:00 -> 2026-04-17 16:00:00-04:00
Índice único: True
```

6.2 Identificación de features nuevas y targets

```
In [55]: # 6.1 Identificación de columnas
# =====

# Columnas originales antes de crear features
base_stage05_cols = df_stage05_base.columns.tolist()

# Features creadas en el punto 5
new_feature_cols = [
    col for col in df_quality.columns
    if col not in base_stage05_cols
]

# Features numéricas
numeric_feature_cols = (
    df_quality[new_feature_cols]
    .select_dtypes(include=[np.number])
    .columns
    .tolist()
)

# Features no numéricas
non_numeric_feature_cols = [
    col for col in new_feature_cols
    if col not in numeric_feature_cols
]

# Targets DIR
dir_target_cols = [
    col for col in df_quality.columns
```

```

        if col.startswith("dir_")
    ]

    # Targets BAR
    bar_target_cols = [
        col for col in df_quality.columns
        if col.startswith("bar_")
    ]

    # Targets OPC Label
    opc_target_label_cols = [
        col for col in df_quality.columns
        if col.startswith("opc_") and col.endswith("_label")
    ]

    # Targets OPC numéricos
    opc_target_numeric_cols = [
        col for col in df_quality.columns
        if col.startswith("opc_") and not col.endswith("_label")
    ]

    target_cols = (
        dir_target_cols
        + bar_target_cols
        + opc_target_label_cols
        + opc_target_numeric_cols
    )

    print("Resumen de columnas")
    print("=" * 80)
    print(f"Targets detectados:           {len(target_cols)}")
    print(f"Features nuevas:                 {len(new_feature_cols)}")
    print(f"Features nuevas numéricas:       {len(numeric_feature_cols)}")
    print(f"Features nuevas no numéricas:    {len(non_numeric_feature_cols)}")

    print("\nFeatures no numéricas:")
    print(non_numeric_feature_cols)

```

Resumen de columnas

```

=====
Targets detectados:           38
Features nuevas:              116
Features nuevas numéricas:    116
Features nuevas no numéricas: 0

```

```

Features no numéricas:
[]

```

6.3 Separación temporal para auditoría

```

In [56]: # 6.2 Separación development / final test
# =====

df_dev_quality = df_quality[
    df_quality["dataset_split"] == "development_2020_2024"
].copy()

df_final_quality = df_quality[
    df_quality["dataset_split"] == "final_test_2025_2026"

```

```
].copy()

print("Separación temporal para auditoría")
print("=" * 80)

print(f"Development 2020-2024: {df_dev_quality.shape}")
print(f"Final test 2025-2026: {df_final_quality.shape}")

print("\nDistribución dataset_split:")
print(df_quality["dataset_split"].value_counts(dropna=False).sort_index())
```

Separación temporal para auditoría

```
=====
Development 2020-2024: (818835, 147)
Final test 2025-2026: (205227, 147)
```

Distribución dataset_split:

```
dataset_split
development_2020_2024    818835
final_test_2025_2026    205227
Name: count, dtype: int64
```

6.4 Auditoría de NaN e infinitos

```
In [57]: # 6.3 Auditoría de NaN e infinitos
# =====

feature_quality_summary = pd.DataFrame(index=numeric_feature_cols)

# NaN en development
feature_quality_summary["nan_count_dev"] = df_dev_quality[numeric_feature_cols].
feature_quality_summary["nan_ratio_dev"] = (
    feature_quality_summary["nan_count_dev"] / len(df_dev_quality)
)

# Infinitos en development
inf_mask_dev = np.isinf(df_dev_quality[numeric_feature_cols])
feature_quality_summary["inf_count_dev"] = inf_mask_dev.sum()
feature_quality_summary["inf_ratio_dev"] = (
    feature_quality_summary["inf_count_dev"] / len(df_dev_quality)
)

# Valores válidos
feature_quality_summary["valid_count_dev"] = (
    len(df_dev_quality)
    - feature_quality_summary["nan_count_dev"]
    - feature_quality_summary["inf_count_dev"]
)

print("Resumen de NaN e infinitos en development")
print("=" * 80)

print("\nTop features con mayor ratio de NaN:")
print(
    feature_quality_summary
    .sort_values("nan_ratio_dev", ascending=False)
    .head(30)
)
```

```
print("\nFeatures con infinitos:")
print(
    feature_quality_summary[
        feature_quality_summary["inf_count_dev"] > 0
    ].sort_values("inf_count_dev", ascending=False)
)
```


Resumen de NaN e infinitos en development

=====

Top features con mayor ratio de NaN:

	nan_count_dev	nan_ratio_dev	inf_count_dev	\
ret_pts_90m	106650	0.130246	0	
ret_pct_90m	106650	0.130246	0	
vol_ret_1m_90m	106650	0.130246	0	
rolling_high_90m	105465	0.128799	0	
close_pos_range_90m	105465	0.128799	0	
rolling_range_pts_90m	105465	0.128799	0	
close_dist_high_90m	105465	0.128799	0	
close_dist_low_90m	105465	0.128799	0	
volume_std_90m	105465	0.128799	0	
volume_rel_90m	105465	0.128799	0	
volume_zscore_90m	105465	0.128799	0	
volume_mean_90m	105465	0.128799	0	
rolling_range_pct_90m	105465	0.128799	0	
rolling_low_90m	105465	0.128799	0	
bar_range_std_90m	105465	0.128799	0	
bar_range_mean_90m	105465	0.128799	0	
ret_pts_60m	71100	0.086831	0	
vol_ret_1m_60m	71100	0.086831	0	
ret_pct_60m	71100	0.086831	0	
bar_range_std_60m	69915	0.085384	0	
volume_rel_60m	69915	0.085384	0	
volume_std_60m	69915	0.085384	0	
rolling_range_pct_60m	69915	0.085384	0	
bar_range_mean_60m	69915	0.085384	0	
rolling_low_60m	69915	0.085384	0	
close_dist_high_60m	69915	0.085384	0	
volume_mean_60m	69915	0.085384	0	
volume_zscore_60m	69915	0.085384	0	
rolling_high_60m	69915	0.085384	0	
rolling_range_pts_60m	69915	0.085384	0	

	inf_ratio_dev	valid_count_dev
ret_pts_90m	0.0	712185
ret_pct_90m	0.0	712185
vol_ret_1m_90m	0.0	712185
rolling_high_90m	0.0	713370
close_pos_range_90m	0.0	713370
rolling_range_pts_90m	0.0	713370
close_dist_high_90m	0.0	713370
close_dist_low_90m	0.0	713370
volume_std_90m	0.0	713370
volume_rel_90m	0.0	713370
volume_zscore_90m	0.0	713370
volume_mean_90m	0.0	713370
rolling_range_pct_90m	0.0	713370
rolling_low_90m	0.0	713370
bar_range_std_90m	0.0	713370
bar_range_mean_90m	0.0	713370
ret_pts_60m	0.0	747735
vol_ret_1m_60m	0.0	747735
ret_pct_60m	0.0	747735
bar_range_std_60m	0.0	748920
volume_rel_60m	0.0	748920
volume_std_60m	0.0	748920
rolling_range_pct_60m	0.0	748920

bar_range_mean_60m	0.0	748920
rolling_low_60m	0.0	748920
close_dist_high_60m	0.0	748920
volume_mean_60m	0.0	748920
volume_zscore_60m	0.0	748920
rolling_high_60m	0.0	748920
rolling_range_pts_60m	0.0	748920

Features con infinitos:

Empty DataFrame

Columns: [nan_count_dev, nan_ratio_dev, inf_count_dev, inf_ratio_dev, valid_count_dev]

Index: []

6.5 Features constantes o casi constantes

```
In [58]: # 6.4 Features constantes o casi constantes
# =====

# Cantidad de valores únicos
feature_quality_summary["nunique_dev"] = (
    df_dev_quality[numeric_feature_cols]
    .replace([np.inf, -np.inf], np.nan)
    .nunique(dropna=True)
)

# Ratio del valor más frecuente
dominant_ratios = {}

for col in numeric_feature_cols:
    vc = (
        df_dev_quality[col]
        .replace([np.inf, -np.inf], np.nan)
        .dropna()
        .value_counts(normalize=True)
    )
    dominant_ratios[col] = vc.iloc[0] if len(vc) > 0 else np.nan

feature_quality_summary["dominant_value_ratio_dev"] = pd.Series(dominant_ratios)

# Desviación estándar
feature_quality_summary["std_dev"] = (
    df_dev_quality[numeric_feature_cols]
    .replace([np.inf, -np.inf], np.nan)
    .std()
)

constant_feature_cols = feature_quality_summary[
    feature_quality_summary["nunique_dev"] <= 1
].index.tolist()

near_constant_feature_cols = feature_quality_summary[
    feature_quality_summary["dominant_value_ratio_dev"] >= 0.995
].index.tolist()

zero_std_feature_cols = feature_quality_summary[
    feature_quality_summary["std_dev"] == 0
].index.tolist()
```

```
print("Features constantes o casi constantes")
print("=" * 80)

print(f"Features constantes:      {len(constant_feature_cols)}")
print(constant_feature_cols)

print(f"\nFeatures casi constantes: {len(near_constant_feature_cols)}")
print(near_constant_feature_cols)

print(f"\nFeatures con std = 0:      {len(zero_std_feature_cols)}")
print(zero_std_feature_cols)
```

Features constantes o casi constantes

=====

Features constantes: 0

[]

Features casi constantes: 0

[]

Features con std = 0: 0

[]

6.6 Features con alto nivel de NaN

```
In [65]: # 6.5 Features con alto nivel de NaN
# =====

MAX_NAN_RATIO = 0.30

high_nan_feature_cols = feature_quality_summary[
    feature_quality_summary["nan_ratio_dev"] > MAX_NAN_RATIO
].index.tolist()

print("Features con alto nivel de NaN")
print("=" * 80)
print(f"Umbral máximo permitido: {MAX_NAN_RATIO:.0%}")
print(f"Cantidad detectada:      {len(high_nan_feature_cols)}")

for col in high_nan_feature_cols:
    ratio = feature_quality_summary.loc[col, "nan_ratio_dev"]
    print(f"{col:<40} nan_ratio_dev={ratio:.4f}")
```

Features con alto nivel de NaN

=====

Umbral máximo permitido: 30%

Cantidad detectada: 0

6.7 Features que pasan validación básica de calidad

Este bloque no selecciona features por capacidad predictiva.

Solo separa las features que no presentan problemas estructurales graves. Si una feature pasa este filtro, significa que puede continuar a la etapa de análisis de señal, no que sea una feature final para modelado.

```
In [60]: # 6.6 Features que pasan validación básica de calidad
# =====

features_to_exclude_quality = list(dict.fromkeys(
    high_nan_feature_cols
    + constant_feature_cols
    + near_constant_feature_cols
    + zero_std_feature_cols
))

feature_cols_quality_valid = [
    col for col in numeric_feature_cols
    if col not in features_to_exclude_quality
]

print("Validación básica de calidad")
print("=" * 80)

print(f"Features numéricas iniciales:      {len(numeric_feature_cols)}")
print(f"Features excluidas por calidad básica: {len(features_to_exclude_quality)}")
print(f"Features que pasan calidad básica:      {len(feature_cols_quality_valid)}")

print("\nFeatures excluidas por calidad básica:")
for col in features_to_exclude_quality:
    print(col)

print("\nNota:")
print("Estas features pasan controles básicos de NaN, infinitos y varianza.")
print("Todavía no fueron filtradas por correlación ni por señal predictiva.")
```

Validación básica de calidad

```
=====
Features numéricas iniciales:      116
Features excluidas por calidad básica: 0
Features que pasan calidad básica:  116
```

Features excluidas por calidad básica:

Nota:

Estas features pasan controles básicos de NaN, infinitos y varianza.
 Todavía no fueron filtradas por correlación ni por señal predictiva.

6.8 Diagnóstico de correlación entre features

La correlación entre features se calcula para detectar redundancia.

En este punto no se eliminan features por correlación. La razón es que dos features pueden estar muy correlacionadas, pero una puede ser más útil que otra para ciertos modelos o ciertos targets.

La reducción por correlación se decidirá más adelante, antes del modelado, comparando un set completo contra un set reducido.

```
In [61]: # 6.7 Diagnóstico de correlación entre features
# =====

corr_input_cols = feature_cols_quality_valid.copy()
```

```
df_corr = (
    df_dev_quality[corr_input_cols]
    .replace([np.inf, -np.inf], np.nan)
)

corr_matrix = df_corr.corr(method="spearman").abs()

# Triángulo superior de la matriz
upper_mask = np.triu(np.ones(corr_matrix.shape), k=1).astype(bool)

corr_pairs = (
    corr_matrix
    .where(upper_mask)
    .stack()
    .reset_index()
)

corr_pairs.columns = ["feature_1", "feature_2", "abs_spearman_corr"]

HIGH_CORR_THRESHOLD = 0.95

high_corr_pairs = corr_pairs[
    corr_pairs["abs_spearman_corr"] >= HIGH_CORR_THRESHOLD
].sort_values("abs_spearman_corr", ascending=False)

high_corr_feature_names = sorted(
    set(high_corr_pairs["feature_1"]).union(set(high_corr_pairs["feature_2"]))
)

print("Diagnóstico de correlación entre features")
print("=" * 80)
print(f"Features evaluadas: {len(corr_input_cols)}")
print(f"Umbral de correlación alta: {HIGH_CORR_THRESHOLD}")
print(f"Pares altamente correlacionados: {len(high_corr_pairs)}")
print(f"Features involucradas en alta correlación: {len(high_corr_feature_names)}")

print("\nTop pares correlacionados:")
print(high_corr_pairs.head(50))

print("\nNota:")
print("La correlación alta queda registrada como advertencia de redundancia.")
print("No se eliminan features por correlación en este punto.")
```

Diagnóstico de correlación entre features

```
=====
Features evaluadas:                116
Umbral de correlación alta:        0.95
Pares altamente correlacionados:   139
Features involucradas en alta correlación: 87
```

Top pares correlacionados:

	feature_1	feature_2	abs_spearman_corr
8776	close_dist_high_15m	close_dist_low_15m	1.000000
9127	close_dist_high_30m	close_dist_low_30m	1.000000
9011	close_pos_range_30m	close_dist_low_30m	1.000000
8074	close_dist_high_5m	close_dist_low_5m	1.000000
8308	close_pos_range_10m	close_dist_high_10m	1.000000
8425	close_dist_high_10m	close_dist_low_10m	1.000000
8309	close_pos_range_10m	close_dist_low_10m	1.000000
7957	close_pos_range_5m	close_dist_high_5m	1.000000
7958	close_pos_range_5m	close_dist_low_5m	1.000000
9712	close_pos_range_90m	close_dist_high_90m	1.000000
9478	close_dist_high_60m	close_dist_low_60m	1.000000
8660	close_pos_range_15m	close_dist_low_15m	1.000000
8659	close_pos_range_15m	close_dist_high_15m	1.000000
9713	close_pos_range_90m	close_dist_low_90m	1.000000
9010	close_pos_range_30m	close_dist_high_30m	1.000000
9361	close_pos_range_60m	close_dist_high_60m	1.000000
9362	close_pos_range_60m	close_dist_low_60m	1.000000
9829	close_dist_high_90m	close_dist_low_90m	1.000000
4336	rolling_low_10m	rolling_low_15m	0.999996
4219	rolling_high_10m	rolling_high_15m	0.999996
3517	rolling_low_5m	rolling_low_10m	0.999995
3400	rolling_high_5m	rolling_high_10m	0.999995
3524	rolling_low_5m	rolling_low_15m	0.999989
3407	rolling_high_5m	rolling_high_15m	0.999988
5155	rolling_low_15m	rolling_low_30m	0.999986
5038	rolling_high_15m	rolling_high_30m	0.999985
3394	rolling_high_5m	rolling_low_5m	0.999984
4343	rolling_low_10m	rolling_low_30m	0.999978
4226	rolling_high_10m	rolling_high_30m	0.999977
6793	rolling_low_60m	rolling_low_90m	0.999977
6676	rolling_high_60m	rolling_high_90m	0.999976
3401	rolling_high_5m	rolling_low_10m	0.999976
3516	rolling_low_5m	rolling_high_10m	0.999974
5974	rolling_low_30m	rolling_low_60m	0.999971
5857	rolling_high_30m	rolling_high_60m	0.999970
3531	rolling_low_5m	rolling_low_30m	0.999969
4213	rolling_high_10m	rolling_low_10m	0.999968
3408	rolling_high_5m	rolling_low_15m	0.999968
3414	rolling_high_5m	rolling_high_30m	0.999966
3523	rolling_low_5m	rolling_high_15m	0.999965
4220	rolling_high_10m	rolling_low_15m	0.999961
4335	rolling_low_10m	rolling_high_15m	0.999960
5032	rolling_high_15m	rolling_low_15m	0.999954
5162	rolling_low_15m	rolling_low_60m	0.999947
5045	rolling_high_15m	rolling_high_60m	0.999943
3415	rolling_high_5m	rolling_low_30m	0.999943
4227	rolling_high_10m	rolling_low_30m	0.999938
4350	rolling_low_10m	rolling_low_60m	0.999937
3530	rolling_low_5m	rolling_high_30m	0.999935
5981	rolling_low_30m	rolling_low_90m	0.999935

Nota:

La correlación alta queda registrada como advertencia de redundancia.

No se eliminan features por correlación en este punto.

6.9 Resumen interpretativo de redundancia

```
In [62]: # 6.8 Resumen interpretativo de redundancia
# =====

print("Resumen interpretativo de redundancia")
print("=" * 80)

print("Pares con correlación perfecta o casi perfecta suelen aparecer por transfo")
print("\nEjemplos típicos:")

print("\n1) Posición dentro del rango:")
print("    close_pos_range_Xm, close_dist_high_Xm y close_dist_low_Xm suelen cont")

print("\n2) Niveles rolling de precio:")
print("    rolling_high_Xm y rolling_low_Xm pueden estar altamente correlacionado")

print("\n3) Features temporales:")
print("    hour y minute pueden ser redundantes con minute_of_day_sin y minute_of")

print("\nDecisión del punto 6:")
print("    Se documenta la redundancia, pero no se eliminan features todavía.")
print("    El set que continúa al punto 7 será feature_cols_quality_valid.")
```

Resumen interpretativo de redundancia

=====

Pares con correlación perfecta o casi perfecta suelen aparecer por transformacion
es equivalentes.

Ejemplos típicos:

- 1) Posición dentro del rango:
close_pos_range_Xm, close_dist_high_Xm y close_dist_low_Xm suelen contener inf
ormación equivalente.
- 2) Niveles rolling de precio:
rolling_high_Xm y rolling_low_Xm pueden estar altamente correlacionados entre
ventanas cercanas.
- 3) Features temporales:
hour y minute pueden ser redundantes con minute_of_day_sin y minute_of_day_co
s.

Decisión del punto 6:

Se documenta la redundancia, pero no se eliminan features todavía.
El set que continúa al punto 7 será feature_cols_quality_valid.

6.10 Registro final del punto 6

```
In [63]: # 6.9 Registro final del punto 6
# =====

feature_quality_summary["excluded_by_quality"] = (
    feature_quality_summary.index.isin(features_to_exclude_quality)
```

```

)

feature_quality_summary["passes_quality_basic"] = (
    feature_quality_summary.index.isin(feature_cols_quality_valid)
)

feature_quality_summary["in_high_corr_pair"] = (
    feature_quality_summary.index.isin(high_corr_feature_names)
)

stage05_feature_quality_registry = {
    "numeric_feature_cols": numeric_feature_cols,
    "non_numeric_feature_cols": non_numeric_feature_cols,
    "features_to_exclude_quality": features_to_exclude_quality,
    "feature_cols_quality_valid": feature_cols_quality_valid,
    "high_nan_feature_cols": high_nan_feature_cols,
    "constant_feature_cols": constant_feature_cols,
    "near_constant_feature_cols": near_constant_feature_cols,
    "zero_std_feature_cols": zero_std_feature_cols,
    "high_corr_threshold": HIGH_CORR_THRESHOLD,
    "high_corr_pairs_count": len(high_corr_pairs),
    "high_corr_feature_names": high_corr_feature_names,
    "max_nan_ratio": MAX_NAN_RATIO,
}

print("Registro final de validación de features")
print("=" * 80)

print(f"Features numéricas iniciales: {len(numeric_feature_cols)}")
print(f"Features excluidas por calidad básica: {len(features_to_exclude_quality)}")
print(f"Features que pasan calidad básica: {len(feature_cols_quality_valid)}")
print(f"Pares con alta correlación: {len(high_corr_pairs)}")
print(f"Features en pares de alta correlación: {len(high_corr_feature_names)}")

print("\nObjeto principal para el punto 7:")
print("feature_cols_quality_valid")

```

Registro final de validación de features

```

=====
Features numéricas iniciales:      116
Features excluidas por calidad básica: 0
Features que pasan calidad básica:  116
Pares con alta correlación:        139
Features en pares de alta correlación: 87

```

Objeto principal para el punto 7:
feature_cols_quality_valid

6.11 Dataset resultante del punto 6

```

In [64]: # 6.10 Dataset resultante del punto 6
# =====

df_stage05_features_validated = df_quality.copy()

print("Dataset validado del Stage 05 creado")
print("=" * 80)

print(f"Shape df_stage05_features_validated: {df_stage05_features_validated.shape}

```



```
print(f"Features que pasan calidad básica:  {len(feature_cols_quality_valid)}")
print(f"Targets disponibles:                {len(target_cols)}")

print("\nImportante:")
print("Las features con alta correlación fueron registradas, pero no eliminadas.")
print("La reducción por redundancia se evaluará más adelante, antes del entrenam
```

Dataset validado del Stage 05 creado

```
=====
Shape df_stage05_features_validated: (1024062, 147)
Features que pasan calidad básica:    116
Targets disponibles:                  38
```

Importante:

Las features con alta correlación fueron registradas, pero no eliminadas.

La reducción por redundancia se evaluará más adelante, antes del entrenamiento de modelos.

Observaciones del punto 6

- La validación de calidad se realizó sobre el dataset completo del Stage 05, compuesto por 1,024,062 filas y 147 columnas, de las cuales 116 corresponden a features nuevas y 38 a targets disponibles.
- El análisis se realizó principalmente sobre el período development_2020_2024, que contiene 818,835 observaciones. El período final_test_2025_2026 se mantuvo separado para evitar contaminación del set final de evaluación.
- Las 116 features nuevas son numéricas. No se detectaron features no numéricas dentro del set creado en el punto 5.
- No se detectaron valores infinitos en las features. Tampoco se identificaron columnas constantes, casi constantes ni columnas con desviación estándar igual a cero.
- Las features con mayor proporción de valores nulos corresponden principalmente a ventanas largas de 60 y 90 minutos. Esto es esperable, ya que las ventanas rolling y los lags se calcularon sin cruzar días de trading. El máximo ratio de NaN observado fue aproximadamente 13.0%, por debajo del umbral definido de 30%.
- Por lo tanto, ninguna feature fue excluida por problemas básicos de calidad. Las 116 features pasan la validación estructural inicial.
- Sin embargo, se detectó una redundancia importante entre features: 139 pares presentan correlación de Spearman absoluta mayor o igual a 0.95, involucrando 87 features. Esto indica que el set actual contiene variables con información muy similar o transformaciones equivalentes.
- Los principales casos de redundancia aparecen en:
 - variables de posición dentro del rango, como close_pos_range, close_dist_high y close_dist_low;

- niveles rolling de precio, como rolling_high y rolling_low en ventanas cercanas;
- variables temporales potencialmente equivalentes, como hour, minute y las codificaciones cíclicas de minute_of_day.

Cierre del punto 6

En este punto se confirmó que el set de features construido en el Stage 05 es estructuralmente válido para continuar con el análisis de señal predictiva.

La auditoría no detectó problemas graves de calidad: no hubo infinitos, columnas constantes, columnas casi constantes ni niveles excesivos de valores nulos. Por este motivo, no se excluyó ninguna feature por criterios básicos de calidad.

Sin embargo, el set aún no debe considerarse definitivo para entrenamiento, ya que presenta una redundancia interna relevante. La correlación elevada entre features no mide capacidad predictiva, pero sí advierte que varias variables contienen información muy similar o derivada de una misma transformación.

En esta etapa se decide conservar las 116 features que pasaron la validación básica y registrar la redundancia como un aspecto a tratar posteriormente. La reducción definitiva del set de features se evaluará después del análisis de señal predictiva y antes del modelado.

Por lo tanto, el objeto que continúa al punto 7 es:

feature_cols_quality_valid

Este set contiene las features aptas para continuar el análisis, pero todavía no representa el conjunto final de features para entrenamiento.

7. Análisis univariado de señal predictiva

En este punto se evalúa, de forma exploratoria, si las features construidas presentan alguna relación univariada con los targets DIR, BAR y OPC.

El análisis se realiza únicamente sobre el período development_2020_2024. El período final_test_2025_2026 no se utiliza para evitar contaminación del set final de evaluación.

Este análisis no define todavía el set final de features ni implica que una feature sea suficiente para entrenar un modelo. Su objetivo es detectar señales preliminares, redundancias útiles y posibles diferencias por target, régimen intradiario y año.

7.0 Corrección de identificación de targets

In [66]:

```
# 7.0 Identificación correcta de targets para análisis de señal
# =====
```

```
# IMPORTANTE:
# Los targets se identifican desde df_stage05_base, no desde df_stage05_features
# para evitar confundir features nuevas como bar_range_pts con targets BAR.

base_cols_stage05 = df_stage05_base.columns.tolist()

dir_target_cols = [
    col for col in base_cols_stage05
    if col.startswith("dir_")
]

bar_target_cols = [
    col for col in base_cols_stage05
    if col.startswith("bar_p")
]

opc_target_label_cols = [
    col for col in base_cols_stage05
    if col.startswith("opc_") and col.endswith("_label")
]

opc_target_numeric_cols = [
    col for col in base_cols_stage05
    if col.startswith("opc_") and not col.endswith("_label")
]

target_cols_for_signal = (
    dir_target_cols
    + bar_target_cols
    + opc_target_numeric_cols
)

print("Targets correctamente identificados")
print("=" * 80)

print(f"DIR targets: {len(dir_target_cols)}")
print(dir_target_cols)

print(f"\nBAR targets: {len(bar_target_cols)}")
print(bar_target_cols)

print(f"\nOPC targets numéricos: {len(opc_target_numeric_cols)}")
print(opc_target_numeric_cols)

print(f"\nOPC labels: {len(opc_target_label_cols)}")
print(opc_target_label_cols)

print(f"\nTotal targets numéricos para análisis: {len(target_cols_for_signal)}")
```

Targets correctamente identificados

=====

DIR targets: 5

['dir_p50_h30', 'dir_p50_h60', 'dir_p50_h90', 'dir_p40_h60', 'dir_p60_h60']

BAR targets: 5

['bar_p50_h30_tp15_sl10', 'bar_p50_h60_tp15_sl10', 'bar_p50_h90_tp15_sl10', 'bar_p40_h60_tp15_sl10', 'bar_p60_h60_tp15_sl10']

OPC targets numéricos: 5

['opc_p50_h30_tp15_sl10', 'opc_p50_h60_tp15_sl10', 'opc_p50_h90_tp15_sl10', 'opc_p40_h60_tp15_sl10', 'opc_p60_h60_tp15_sl10']

OPC labels: 5

['opc_p50_h30_tp15_sl10_label', 'opc_p50_h60_tp15_sl10_label', 'opc_p50_h90_tp15_sl10_label', 'opc_p40_h60_tp15_sl10_label', 'opc_p60_h60_tp15_sl10_label']

Total targets numéricos para análisis: 15

7.1 Preparación del dataset de análisis

```
In [67]: # 7.1 Preparación del dataset de análisis univariado
# =====

df_signal = df_stage05_features_validated.copy().sort_index()

# Usamos solo development
df_signal_dev = df_signal[
    df_signal["dataset_split"] == "development_2020_2024"
].copy()

# Features que pasaron calidad básica en el punto 6
feature_cols_signal = feature_cols_quality_valid.copy()

print("Dataset para análisis de señal")
print("=" * 80)

print(f"Shape development:      {df_signal_dev.shape}")
print(f"Features a evaluar:      {len(feature_cols_signal)}")
print(f"Targets DIR:              {len(dir_target_cols)}")
print(f"Targets BAR:              {len(bar_target_cols)}")
print(f"Targets OPC numéricos:    {len(opc_target_numeric_cols)}")
print(f"Targets totales análisis: {len(target_cols_for_signal)}")

print("\nRango temporal development:")
print(f"{df_signal_dev.index.min()} -> {df_signal_dev.index.max()}")
```

Dataset para análisis de señal

=====

Shape development: (818835, 147)

Features a evaluar: 116

Targets DIR: 5

Targets BAR: 5

Targets OPC numéricos: 5

Targets totales análisis: 15

Rango temporal development:

2020-01-02 04:30:00-05:00 -> 2024-12-31 16:00:00-05:00

7.2 Spearman / IC para DIR y BAR

Para DIR y BAR se calcula una correlación de Spearman entre cada feature y cada target.

Este análisis se interpreta como una aproximación exploratoria al Information Coefficient, pero no como prueba definitiva de capacidad predictiva.

Para OPC no se usa Spearman como métrica principal, porque OPC es un target multiclase nominal. En ese caso se usará Mutual Information.

```
In [69]: # 7.2 Spearman / IC para targets DIR y BAR
# =====

def compute_spearman_ic(df, feature_cols, target_cols, min_obs=10_000):
    """
    Calcula Spearman IC entre cada feature y cada target.
    Usa solo filas donde el target no es NaN.
    """
    results = []

    for target in target_cols:
        df_target = df[feature_cols + [target]].copy()
        df_target = df_target[df_target[target].notna()]

        for feature in feature_cols:
            pair = df_target[[feature, target]].replace([np.inf, -np.inf], np.nan)
            n_obs = len(pair)

            if n_obs < min_obs:
                ic = np.nan
            else:
                ic = pair[feature].corr(pair[target], method="spearman")

            results.append({
                "target": target,
                "feature": feature,
                "spearman_ic": ic,
                "abs_spearman_ic": abs(ic) if pd.notna(ic) else np.nan,
                "n_obs": n_obs,
            })

    return pd.DataFrame(results)

ic_target_cols = dir_target_cols + bar_target_cols

df_ic_results = compute_spearman_ic(
    df=df_signal_dev,
    feature_cols=feature_cols_signal,
    target_cols=ic_target_cols,
    min_obs=10_000,
)

df_ic_results = df_ic_results.sort_values(
    ["target", "abs_spearman_ic"],
    ascending=[True, False]
)
```

```
print("Spearman / IC calculado para DIR y BAR")
print("=" * 80)
print(f"Resultados: {df_ic_results.shape}")

print("\nTop 30 relaciones feature-target por abs(IC):")
print(
    df_ic_results
    .sort_values("abs_spearman_ic", ascending=False)
    .head(30)
)
```

Spearman / IC calculado para DIR y BAR

Resultados: (1160, 5)

Top 30 relaciones feature-target por abs(IC):

	target	feature	spearman_ic \
614	bar_p50_h30_tp15_sl10	bar_range_mean_10m	0.120001
621	bar_p50_h30_tp15_sl10	bar_range_mean_15m	0.119306
607	bar_p50_h30_tp15_sl10	bar_range_mean_5m	0.118912
622	bar_p50_h30_tp15_sl10	bar_range_std_15m	0.117097
628	bar_p50_h30_tp15_sl10	bar_range_mean_30m	0.117006
629	bar_p50_h30_tp15_sl10	bar_range_std_30m	0.115776
615	bar_p50_h30_tp15_sl10	bar_range_std_10m	0.115135
627	bar_p50_h30_tp15_sl10	vol_ret_1m_30m	0.115054
635	bar_p50_h30_tp15_sl10	bar_range_mean_60m	0.114889
620	bar_p50_h30_tp15_sl10	vol_ret_1m_15m	0.114466
634	bar_p50_h30_tp15_sl10	vol_ret_1m_60m	0.113898
618	bar_p50_h30_tp15_sl10	rolling_range_pts_10m	0.113573
611	bar_p50_h30_tp15_sl10	rolling_range_pts_5m	0.112868
1085	bar_p60_h60_tp15_sl10	bar_range_mean_15m	0.112821
1078	bar_p60_h60_tp15_sl10	bar_range_mean_10m	0.112808
1092	bar_p60_h60_tp15_sl10	bar_range_mean_30m	0.112408
625	bar_p50_h30_tp15_sl10	rolling_range_pts_15m	0.112376
613	bar_p50_h30_tp15_sl10	vol_ret_1m_10m	0.112194
636	bar_p50_h30_tp15_sl10	bar_range_std_60m	0.111962
1071	bar_p60_h60_tp15_sl10	bar_range_mean_5m	0.111416
1099	bar_p60_h60_tp15_sl10	bar_range_mean_60m	0.110719
619	bar_p50_h30_tp15_sl10	rolling_range_pct_10m	0.110088
612	bar_p50_h30_tp15_sl10	rolling_range_pct_5m	0.109564
1091	bar_p60_h60_tp15_sl10	vol_ret_1m_30m	0.109166
626	bar_p50_h30_tp15_sl10	rolling_range_pct_15m	0.108923
632	bar_p50_h30_tp15_sl10	rolling_range_pts_30m	0.108878
1084	bar_p60_h60_tp15_sl10	vol_ret_1m_15m	0.108343
642	bar_p50_h30_tp15_sl10	bar_range_mean_90m	0.108262
580	bar_p50_h30_tp15_sl10	bar_range_pts	0.108085
1089	bar_p60_h60_tp15_sl10	rolling_range_pts_15m	0.107548

	abs_spearman_ic	n_obs
614	0.120001	388532
621	0.119306	385838
607	0.118912	391267
622	0.117097	385838
628	0.117006	377764
629	0.115776	377764
615	0.115135	388532
627	0.115054	377237
635	0.114889	362567
620	0.114466	385295
634	0.113898	362063
618	0.113573	388532
611	0.112868	391267
1085	0.112821	294296
1078	0.112808	296211
1092	0.112408	288719
625	0.112376	385838
613	0.112194	387984
636	0.111962	362567
1071	0.111416	298151
1099	0.110719	277788
619	0.110088	388532

612	0.109564	391267
1091	0.109166	288363
626	0.108923	385838
632	0.108878	377764
1084	0.108343	293917
642	0.108262	347053
580	0.108085	393525
1089	0.107548	294296

Observaciones preliminares del análisis Spearman / IC

- Los mayores valores de IC aparecen sobre targets BAR, especialmente en bar_p50_h30_tp15_sl10 y bar_p60_h60_tp15_sl10.
- Las features con mayor asociación son principalmente variables de rango, volatilidad y amplitud reciente: bar_range_mean, bar_range_std, vol_ret_1m y rolling_range.
- Esto es coherente con la naturaleza del target BAR, porque BAR depende de si el precio alcanza una barrera TP/SL dentro de una ventana futura. Si el mercado viene con más rango o volatilidad reciente, es más probable que alguna barrera sea alcanzada.
- El mayor IC observado es aproximadamente 0.12. No es una señal fuerte en términos absolutos, pero para datos financieros intradiarios es una señal univariada relevante para seguir investigando.
- La mayoría de las mejores relaciones no aparecen con features de volumen ni con variables temporales, sino con estructura de precio y volatilidad reciente.
- Los resultados también muestran redundancia: muchas features top son variantes muy parecidas de la misma idea, por ejemplo bar_range_mean_5m, bar_range_mean_10m, bar_range_mean_15m y bar_range_mean_30m.
- Por ahora no conviene interpretar estos resultados como selección final de features. Sirven para confirmar que el set construido contiene señal preliminar, especialmente relacionada con volatilidad/rango y targets de tipo BAR.
- Estos resultados no permiten todavía concluir que una feature predice rentabilidad operativa. Solo muestran asociación univariada con el target codificado.
- El siguiente paso, Mutual Information, ayudará a revisar si existen relaciones no lineales y si también aparece señal en DIR, BAR y OPC.

En resumen: buena señal inicial, pero concentrada en volatilidad/rango y muy ligada a targets BAR.

Todavía no seleccionaría features; esperaría los resultados de MI y luego estabilidad por régimen/año.

7.3 Mutual Information para DIR, BAR y OPC

Mutual Information permite evaluar dependencia no lineal entre una feature y un target de clasificación.

Se aplica a DIR, BAR y OPC numérico. Para mantener el análisis eficiente, se puede usar una muestra del período development.

```
In [72]: # 7.3 Mutual Information para DIR, BAR y OPC - versión rápida
# =====

import time
import inspect
import numpy as np
import pandas as pd

from sklearn.feature_selection import mutual_info_classif
from sklearn.impute import SimpleImputer

MI_SAMPLE_SIZE = 50_000
RANDOM_STATE = 42

def compute_mutual_information_fast(
    df,
    feature_cols,
    target_cols,
    sample_size=50_000,
    random_state=42,
    min_obs=10_000,
    n_neighbors=3
):
    """
    Calcula Mutual Information entre features numéricas y targets de clasificaci
    Versión rápida para análisis exploratorio inicial.
    """
    results = []

    # Verificar si la versión instalada de sklearn soporta n_jobs
    mi_signature = inspect.signature(mutual_info_classif)
    supports_n_jobs = "n_jobs" in mi_signature.parameters

    total_targets = len(target_cols)

    for i, target in enumerate(target_cols, start=1):
        start_time = time.time()

        print("=" * 80)
        print(f"Procesando target {i}/{total_targets}: {target}")

        df_target = df[feature_cols + [target]].copy()
        df_target = df_target[df_target[target].notna()]
        df_target = df_target.replace([np.inf, -np.inf], np.nan)

        n_obs_total = len(df_target)

        if n_obs_total < min_obs:
            print(f"Target omitido por pocas observaciones: {n_obs_total}")
            continue

        if n_obs_total > sample_size:
```

```

        df_target = df_target.sample(
            n=sample_size,
            random_state=random_state
        )

    X = df_target[feature_cols]
    y = df_target[target].astype(int)

    imputer = SimpleImputer(strategy="median")
    X_imp = imputer.fit_transform(X)

    mi_kwargs = {
        "discrete_features": False,
        "random_state": random_state,
        "n_neighbors": n_neighbors,
    }

    if supports_n_jobs:
        mi_kwargs["n_jobs"] = -1

    mi_values = mutual_info_classif(
        X_imp,
        y,
        **mi_kwargs
    )

    for feature, mi in zip(feature_cols, mi_values):
        results.append({
            "target": target,
            "feature": feature,
            "mutual_information": mi,
            "n_obs_total": n_obs_total,
            "n_obs_used": len(df_target),
            "n_classes": y.nunique(),
        })

    elapsed = time.time() - start_time

    print(f"Observaciones totales target: {n_obs_total}")
    print(f"Observaciones usadas: {len(df_target)}")
    print(f"Clases: {y.nunique()}")
    print(f"Tiempo target: {elapsed:.2f} segundos")

    return pd.DataFrame(results)

df_mi_results = compute_mutual_information_fast(
    df=df_signal_dev,
    feature_cols=feature_cols_signal,
    target_cols=target_cols_for_signal,
    sample_size=MI_SAMPLE_SIZE,
    random_state=RANDOM_STATE,
    min_obs=10_000,
    n_neighbors=3,
)

df_mi_results = df_mi_results.sort_values(
    ["target", "mutual_information"],
    ascending=[True, False]
)

```

```
print("\nMutual Information calculado")
print("=" * 80)
print(f"Resultados: {df_mi_results.shape}")

print("\nTop 30 relaciones feature-target por MI:")
print(
    df_mi_results
    .sort_values("mutual_information", ascending=False)
    .head(30)
)
```

```
=====
Procesando target 1/15: dir_p50_h30
Observaciones totales target: 783285
Observaciones usadas:          50000
Clases:                        3
Tiempo target:                  5.66 segundos
=====
Procesando target 2/15: dir_p50_h60
Observaciones totales target: 747735
Observaciones usadas:          50000
Clases:                        3
Tiempo target:                  2.19 segundos
=====
Procesando target 3/15: dir_p50_h90
Observaciones totales target: 712185
Observaciones usadas:          50000
Clases:                        3
Tiempo target:                  2.12 segundos
=====
Procesando target 4/15: dir_p40_h60
Observaciones totales target: 747735
Observaciones usadas:          50000
Clases:                        3
Tiempo target:                  2.17 segundos
=====
Procesando target 5/15: dir_p60_h60
Observaciones totales target: 747735
Observaciones usadas:          50000
Clases:                        3
Tiempo target:                  2.11 segundos
=====
Procesando target 6/15: bar_p50_h30_tp15_sl10
Observaciones totales target: 393525
Observaciones usadas:          50000
Clases:                        3
Tiempo target:                  1.87 segundos
=====
Procesando target 7/15: bar_p50_h60_tp15_sl10
Observaciones totales target: 374094
Observaciones usadas:          50000
Clases:                        3
Tiempo target:                  1.88 segundos
=====
Procesando target 8/15: bar_p50_h90_tp15_sl10
Observaciones totales target: 357726
Observaciones usadas:          50000
Clases:                        3
Tiempo target:                  1.86 segundos
=====
Procesando target 9/15: bar_p40_h60_tp15_sl10
Observaciones totales target: 450321
Observaciones usadas:          50000
Clases:                        3
Tiempo target:                  1.87 segundos
=====
Procesando target 10/15: bar_p60_h60_tp15_sl10
Observaciones totales target: 299771
Observaciones usadas:          50000
Clases:                        3
Tiempo target:                  1.80 segundos
```

```

=====
Procesando target 11/15: opc_p50_h30_tp15_sl10
Observaciones totales target: 783285
Observaciones usadas:          50000
Clases:                        5
Tiempo target:                  2.32 segundos
=====
Procesando target 12/15: opc_p50_h60_tp15_sl10
Observaciones totales target: 747735
Observaciones usadas:          50000
Clases:                        5
Tiempo target:                  2.37 segundos
=====
Procesando target 13/15: opc_p50_h90_tp15_sl10
Observaciones totales target: 712185
Observaciones usadas:          50000
Clases:                        5
Tiempo target:                  2.05 segundos
=====
Procesando target 14/15: opc_p40_h60_tp15_sl10
Observaciones totales target: 747735
Observaciones usadas:          50000
Clases:                        5
Tiempo target:                  2.13 segundos
=====
Procesando target 15/15: opc_p60_h60_tp15_sl10
Observaciones totales target: 747735
Observaciones usadas:          50000
Clases:                        5
Tiempo target:                  2.12 segundos

Mutual Information calculado
=====
Resultados: (1740, 6)

```

Top 30 relaciones feature-target por MI:

	target	feature	mutual_information \
296	dir_p50_h90	rolling_high_90m	0.295697
297	dir_p50_h90	rolling_low_90m	0.290775
289	dir_p50_h90	rolling_high_60m	0.283220
290	dir_p50_h90	rolling_low_60m	0.277617
412	dir_p40_h60	rolling_high_90m	0.264737
413	dir_p40_h60	rolling_low_90m	0.260785
180	dir_p50_h60	rolling_high_90m	0.255280
181	dir_p50_h60	rolling_low_90m	0.253424
406	dir_p40_h60	rolling_low_60m	0.252644
405	dir_p40_h60	rolling_high_60m	0.250816
173	dir_p50_h60	rolling_high_60m	0.245491
174	dir_p50_h60	rolling_low_60m	0.243461
1456	opc_p50_h90_tp15_sl10	rolling_high_90m	0.237963
529	dir_p60_h60	rolling_low_90m	0.236225
528	dir_p60_h60	rolling_high_90m	0.233162
1572	opc_p40_h60_tp15_sl10	rolling_high_90m	0.232914
1457	opc_p50_h90_tp15_sl10	rolling_low_90m	0.230442
1573	opc_p40_h60_tp15_sl10	rolling_low_90m	0.229547
1449	opc_p50_h90_tp15_sl10	rolling_high_60m	0.227777
521	dir_p60_h60	rolling_high_60m	0.225218
522	dir_p60_h60	rolling_low_60m	0.222682
869	bar_p50_h90_tp15_sl10	rolling_high_60m	0.222154
870	bar_p50_h90_tp15_sl10	rolling_low_60m	0.219345

876	bar_p50_h90_tp15_sl10	rolling_high_90m	0.219344
877	bar_p50_h90_tp15_sl10	rolling_low_90m	0.218302
1565	opc_p40_h60_tp15_sl10	rolling_high_60m	0.217963
1566	opc_p40_h60_tp15_sl10	rolling_low_60m	0.214883
282	dir_p50_h90	rolling_high_30m	0.214518
1450	opc_p50_h90_tp15_sl10	rolling_low_60m	0.212537
283	dir_p50_h90	rolling_low_30m	0.204042

	n_obs_total	n_obs_used	n_classes
296	712185	50000	3
297	712185	50000	3
289	712185	50000	3
290	712185	50000	3
412	747735	50000	3
413	747735	50000	3
180	747735	50000	3
181	747735	50000	3
406	747735	50000	3
405	747735	50000	3
173	747735	50000	3
174	747735	50000	3
1456	712185	50000	5
529	747735	50000	3
528	747735	50000	3
1572	747735	50000	5
1457	712185	50000	5
1573	747735	50000	5
1449	712185	50000	5
521	747735	50000	3
522	747735	50000	3
869	357726	50000	3
870	357726	50000	3
876	357726	50000	3
877	357726	50000	3
1565	747735	50000	5
1566	747735	50000	5
282	712185	50000	3
1450	712185	50000	5
283	712185	50000	3

Observaciones preliminares del análisis de Mutual Information

- El cálculo rápido funcionó correctamente: se evaluaron 15 targets, 116 features por target y se obtuvieron 1,740 relaciones feature-target.
- Usar 50,000 observaciones por target fue suficiente para una primera lectura exploratoria. El tiempo de ejecución fue razonable, entre 2 y 6 segundos por target.
- Los mayores valores de Mutual Information aparecen principalmente en targets DIR y OPC, especialmente en horizontes de 60 y 90 minutos.
- Las features dominantes son rolling_high_Xm y rolling_low_Xm, sobre todo en ventanas largas de 60 y 90 minutos.
- Esto sugiere que el modelo de MI está detectando una relación fuerte entre el nivel/rango histórico reciente del precio y los targets futuros.

- Sin embargo, hay que tener cuidado: `rolling_high` y `rolling_low` contienen mucho nivel absoluto de precio. Eso puede capturar efectos de año, tendencia general del mercado o cambios de régimen, no necesariamente una señal operativa robusta.
- Este resultado refuerza lo visto en el punto 6: existe alta redundancia entre `rolling_high`, `rolling_low` y otras variables de rango. Por eso no conviene seleccionar automáticamente estas features solo porque aparecen arriba en MI.
- A diferencia de Spearman/IC, donde destacaban más las features de rango y volatilidad para targets BAR, en Mutual Information aparecen con fuerza los niveles `rolling` de precio para DIR y OPC. Esto indica que MI está captando relaciones no lineales o dependencias más complejas.
- La señal en OPC es interesante, porque OPC es el target más operativo y multiclase. Que aparezcan relaciones con `opc_p50_h90` y `opc_p40_h60` sugiere que vale la pena seguir evaluando estos targets.
- Aun así, estos resultados todavía no prueban capacidad predictiva estable. Hay que verificar si estas relaciones se mantienen por régimen intradiario y por año dentro de development.

La observación más importante sería esta:

El análisis de Mutual Information muestra señal preliminar, pero está dominado por features de nivel absoluto como `rolling_high` y `rolling_low`. Por lo tanto, antes de usarlas para modelado, será necesario comprobar si esa señal es estable por año y régimen, o si simplemente refleja cambios de nivel del mercado a lo largo del tiempo.

En resumen: hay señal, pero todavía no confiaría ciegamente en las features top de MI. El siguiente paso clave es ver estabilidad por régimen y por año.

7.4 Ranking resumido por target

```
In [73]: # 7.4 Ranking resumido por target
# =====

TOP_N = 15

print("Top features por target según Spearman / IC")
print("=" * 80)

for target in ic_target_cols:
    print(f"\nTarget: {target}")
    top_ic = (
        df_ic_results[df_ic_results["target"] == target]
        .sort_values("abs_spearman_ic", ascending=False)
        .head(TOP_N)
    )
    print(top_ic[["feature", "spearman_ic", "abs_spearman_ic", "n_obs"]])

print("\n\nTop features por target según Mutual Information")
```

```
print("=" * 80)

for target in target_cols_for_signal:
    print(f"\nTarget: {target}")
    top_mi = (
        df_mi_results[df_mi_results["target"] == target]
        .sort_values("mutual_information", ascending=False)
        .head(TOP_N)
    )
    print(top_mi[["feature", "mutual_information", "n_obs_used", "n_classes"]])
```


Top features por target según Spearman / IC

=====

Target: dir_p50_h30

	feature	spearman_ic	abs_spearman_ic	n_obs
80	close_pos_range_60m	0.014899	0.014899	713370
81	close_dist_high_60m	-0.014899	0.014899	713370
82	close_dist_low_60m	0.014899	0.014899	713370
83	close_pos_range_90m	0.014816	0.014816	677820
84	close_dist_high_90m	-0.014816	0.014816	677820
85	close_dist_low_90m	0.014816	0.014816	677820
23	ret_pct_60m	0.013396	0.013396	712185
22	ret_pts_60m	0.012988	0.012988	712185
25	ret_pct_90m	0.011956	0.011956	676635
21	ret_pct_30m	0.011910	0.011910	747735
20	ret_pts_30m	0.011760	0.011760	747735
24	ret_pts_90m	0.011518	0.011518	676635
18	ret_pts_15m	0.008217	0.008217	765510
19	ret_pct_15m	0.007867	0.007867	765510
77	close_pos_range_30m	0.007643	0.007643	748920

Target: dir_p50_h60

	feature	spearman_ic	abs_spearman_ic	n_obs
199	close_pos_range_90m	0.014393	0.014393	642270
200	close_dist_high_90m	-0.014393	0.014393	642270
201	close_dist_low_90m	0.014393	0.014393	642270
196	close_pos_range_60m	0.012969	0.012969	677820
197	close_dist_high_60m	-0.012969	0.012969	677820
198	close_dist_low_60m	0.012969	0.012969	677820
136	ret_pts_30m	0.012486	0.012486	712185
137	ret_pct_30m	0.011942	0.011942	712185
138	ret_pts_60m	0.011505	0.011505	676635
139	ret_pct_60m	0.011178	0.011178	676635
141	ret_pct_90m	0.010788	0.010788	641085
140	ret_pts_90m	0.010748	0.010748	641085
134	ret_pts_15m	0.009521	0.009521	729960
167	rolling_low_30m	-0.009309	0.009309	713370
166	rolling_high_30m	-0.009266	0.009266	713370

Target: dir_p50_h90

	feature	spearman_ic	abs_spearman_ic	n_obs
315	close_pos_range_90m	0.014172	0.014172	606720
316	close_dist_high_90m	-0.014172	0.014172	606720
317	close_dist_low_90m	0.014172	0.014172	606720
283	rolling_low_30m	-0.013130	0.013130	677820
276	rolling_low_15m	-0.013092	0.013092	695595
282	rolling_high_30m	-0.013067	0.013067	677820
275	rolling_high_15m	-0.013049	0.013049	695595
297	rolling_low_90m	-0.012994	0.012994	606720
269	rolling_low_10m	-0.012972	0.012972	701520
268	rolling_high_10m	-0.012935	0.012935	701520
262	rolling_low_5m	-0.012917	0.012917	707445
261	rolling_high_5m	-0.012889	0.012889	707445
296	rolling_high_90m	-0.012795	0.012795	606720
290	rolling_low_60m	-0.012716	0.012716	642270
289	rolling_high_60m	-0.012592	0.012592	642270

Target: dir_p40_h60

	feature	spearman_ic	abs_spearman_ic	n_obs
431	close_pos_range_90m	0.014640	0.014640	642270

432	close_dist_high_90m	-0.014640	0.014640	642270
433	close_dist_low_90m	0.014640	0.014640	642270
428	close_pos_range_60m	0.013208	0.013208	677820
429	close_dist_high_60m	-0.013208	0.013208	677820
430	close_dist_low_60m	0.013208	0.013208	677820
368	ret_pts_30m	0.012109	0.012109	712185
369	ret_pct_30m	0.011761	0.011761	712185
373	ret_pct_90m	0.011717	0.011717	641085
372	ret_pts_90m	0.011492	0.011492	641085
370	ret_pts_60m	0.011365	0.011365	676635
371	ret_pct_60m	0.011277	0.011277	676635
409	vol_ret_1m_90m	0.009674	0.009674	641085
366	ret_pts_15m	0.009478	0.009478	729960
367	ret_pct_15m	0.008929	0.008929	729960

Target: dir_p60_h60

	feature	spearman_ic	abs_spearman_ic	n_obs
547	close_pos_range_90m	0.014425	0.014425	642270
548	close_dist_high_90m	-0.014425	0.014425	642270
549	close_dist_low_90m	0.014425	0.014425	642270
544	close_pos_range_60m	0.013103	0.013103	677820
545	close_dist_high_60m	-0.013103	0.013103	677820
546	close_dist_low_60m	0.013103	0.013103	677820
515	rolling_low_30m	-0.013018	0.013018	713370
514	rolling_high_30m	-0.012985	0.012985	713370
508	rolling_low_15m	-0.012945	0.012945	731145
507	rolling_high_15m	-0.012930	0.012930	731145
501	rolling_low_10m	-0.012723	0.012723	737070
500	rolling_high_10m	-0.012709	0.012709	737070
522	rolling_low_60m	-0.012667	0.012667	677820
529	rolling_low_90m	-0.012561	0.012561	642270
521	rolling_high_60m	-0.012555	0.012555	677820

Target: bar_p50_h30_tp15_sl10

	feature	spearman_ic	abs_spearman_ic	n_obs
614	bar_range_mean_10m	0.120001	0.120001	388532
621	bar_range_mean_15m	0.119306	0.119306	385838
607	bar_range_mean_5m	0.118912	0.118912	391267
622	bar_range_std_15m	0.117097	0.117097	385838
628	bar_range_mean_30m	0.117006	0.117006	377764
629	bar_range_std_30m	0.115776	0.115776	377764
615	bar_range_std_10m	0.115135	0.115135	388532
627	vol_ret_1m_30m	0.115054	0.115054	377237
635	bar_range_mean_60m	0.114889	0.114889	362567
620	vol_ret_1m_15m	0.114466	0.114466	385295
634	vol_ret_1m_60m	0.113898	0.113898	362063
618	rolling_range_pts_10m	0.113573	0.113573	388532
611	rolling_range_pts_5m	0.112868	0.112868	391267
625	rolling_range_pts_15m	0.112376	0.112376	385838
613	vol_ret_1m_10m	0.112194	0.112194	387984

Target: bar_p50_h60_tp15_sl10

	feature	spearman_ic	abs_spearman_ic	n_obs
730	bar_range_mean_10m	0.107532	0.107532	369509
737	bar_range_mean_15m	0.107391	0.107391	367053
723	bar_range_mean_5m	0.106646	0.106646	372027
744	bar_range_mean_30m	0.106152	0.106152	359700
741	rolling_range_pts_15m	0.102623	0.102623	367053
734	rolling_range_pts_10m	0.102520	0.102520	369509
751	bar_range_mean_60m	0.102420	0.102420	345134

727	rolling_range_pts_5m	0.102118	0.102118	372027
745	bar_range_std_30m	0.102013	0.102013	359700
738	bar_range_std_15m	0.101803	0.101803	367053
743	vol_ret_1m_30m	0.101593	0.101593	359219
748	rolling_range_pts_30m	0.101407	0.101407	359700
736	vol_ret_1m_15m	0.100934	0.100934	366562
729	vol_ret_1m_10m	0.098898	0.098898	369019
742	rolling_range_pct_15m	0.098620	0.098620	367053

Target: bar_p50_h90_tp15_sl10

	feature	spearman_ic	abs_spearman_ic	n_obs
846	bar_range_mean_10m	0.092713	0.092713	353707
853	bar_range_mean_15m	0.092357	0.092357	351505
839	bar_range_mean_5m	0.091577	0.091577	355932
860	bar_range_mean_30m	0.090470	0.090470	344924
857	rolling_range_pts_15m	0.088312	0.088312	351505
850	rolling_range_pts_10m	0.087855	0.087855	353707
867	bar_range_mean_60m	0.087491	0.087491	332134
854	bar_range_std_15m	0.086944	0.086944	351505
843	rolling_range_pts_5m	0.086758	0.086758	355932
864	rolling_range_pts_30m	0.086716	0.086716	344924
847	bar_range_std_10m	0.085548	0.085548	353707
852	vol_ret_1m_15m	0.084404	0.084404	351069
871	rolling_range_pts_60m	0.084180	0.084180	332134
861	bar_range_std_30m	0.083448	0.083448	344924
812	bar_range_pts	0.083391	0.083391	357726

Target: bar_p40_h60_tp15_sl10

	feature	spearman_ic	abs_spearman_ic	n_obs
962	bar_range_mean_10m	0.098774	0.098774	444640
969	bar_range_mean_15m	0.098262	0.098262	441573
955	bar_range_mean_5m	0.097522	0.097522	447748
976	bar_range_mean_30m	0.096908	0.096908	432418
977	bar_range_std_30m	0.095370	0.095370	432418
973	rolling_range_pts_15m	0.094906	0.094906	441573
970	bar_range_std_15m	0.094693	0.094693	441573
966	rolling_range_pts_10m	0.094529	0.094529	444640
980	rolling_range_pts_30m	0.093635	0.093635	432418
959	rolling_range_pts_5m	0.092542	0.092542	447748
963	bar_range_std_10m	0.092509	0.092509	444640
983	bar_range_mean_60m	0.092498	0.092498	414013
975	vol_ret_1m_30m	0.091041	0.091041	431803
968	vol_ret_1m_15m	0.090210	0.090210	440963
961	vol_ret_1m_10m	0.088518	0.088518	444026

Target: bar_p60_h60_tp15_sl10

	feature	spearman_ic	abs_spearman_ic	n_obs
1085	bar_range_mean_15m	0.112821	0.112821	294296
1078	bar_range_mean_10m	0.112808	0.112808	296211
1092	bar_range_mean_30m	0.112408	0.112408	288719
1071	bar_range_mean_5m	0.111416	0.111416	298151
1099	bar_range_mean_60m	0.110719	0.110719	277788
1091	vol_ret_1m_30m	0.109166	0.109166	288363
1084	vol_ret_1m_15m	0.108343	0.108343	293917
1089	rolling_range_pts_15m	0.107548	0.107548	294296
1082	rolling_range_pts_10m	0.107132	0.107132	296211
1098	vol_ret_1m_60m	0.106677	0.106677	277417
1086	bar_range_std_15m	0.106640	0.106640	294296
1090	rolling_range_pct_15m	0.106163	0.106163	294296
1075	rolling_range_pts_5m	0.105836	0.105836	298151

1083	rolling_range_pct_10m	0.105663	0.105663	296211
1077	vol_ret_1m_10m	0.105547	0.105547	295827

Top features por target según Mutual Information

=====

Target: dir_p50_h30

	feature	mutual_information	n_obs_used	n_classes
57	rolling_high_60m	0.177064	50000	3
64	rolling_high_90m	0.176706	50000	3
65	rolling_low_90m	0.173926	50000	3
58	rolling_low_60m	0.170913	50000	3
51	rolling_low_30m	0.146074	50000	3
50	rolling_high_30m	0.143488	50000	3
44	rolling_low_15m	0.107526	50000	3
43	rolling_high_15m	0.105678	50000	3
37	rolling_low_10m	0.090429	50000	3
36	rolling_high_10m	0.088956	50000	3
41	bar_range_mean_15m	0.077379	50000	3
27	bar_range_mean_5m	0.076882	50000	3
34	bar_range_mean_10m	0.076645	50000	3
60	rolling_range_pct_60m	0.075859	50000	3
67	rolling_range_pct_90m	0.074013	50000	3

Target: dir_p50_h60

	feature	mutual_information	n_obs_used	n_classes
180	rolling_high_90m	0.255280	50000	3
181	rolling_low_90m	0.253424	50000	3
173	rolling_high_60m	0.245491	50000	3
174	rolling_low_60m	0.243461	50000	3
166	rolling_high_30m	0.190084	50000	3
167	rolling_low_30m	0.184397	50000	3
159	rolling_high_15m	0.133716	50000	3
160	rolling_low_15m	0.126926	50000	3
152	rolling_high_10m	0.110934	50000	3
153	rolling_low_10m	0.107232	50000	3
145	rolling_high_5m	0.081952	50000	3
146	rolling_low_5m	0.077666	50000	3
176	rolling_range_pct_60m	0.077579	50000	3
183	rolling_range_pct_90m	0.076405	50000	3
150	bar_range_mean_10m	0.067515	50000	3

Target: dir_p50_h90

	feature	mutual_information	n_obs_used	n_classes
296	rolling_high_90m	0.295697	50000	3
297	rolling_low_90m	0.290775	50000	3
289	rolling_high_60m	0.283220	50000	3
290	rolling_low_60m	0.277617	50000	3
282	rolling_high_30m	0.214518	50000	3
283	rolling_low_30m	0.204042	50000	3
275	rolling_high_15m	0.142693	50000	3
276	rolling_low_15m	0.139683	50000	3
268	rolling_high_10m	0.118766	50000	3
269	rolling_low_10m	0.110931	50000	3
261	rolling_high_5m	0.088040	50000	3
262	rolling_low_5m	0.084628	50000	3
299	rolling_range_pct_90m	0.074929	50000	3
292	rolling_range_pct_60m	0.071529	50000	3
285	rolling_range_pct_30m	0.066035	50000	3

Target: dir_p40_h60

	feature	mutual_information	n_obs_used	n_classes
412	rolling_high_90m	0.264737	50000	3
413	rolling_low_90m	0.260785	50000	3
406	rolling_low_60m	0.252644	50000	3
405	rolling_high_60m	0.250816	50000	3
398	rolling_high_30m	0.197274	50000	3
399	rolling_low_30m	0.190322	50000	3
391	rolling_high_15m	0.137947	50000	3
392	rolling_low_15m	0.128178	50000	3
384	rolling_high_10m	0.117425	50000	3
385	rolling_low_10m	0.109589	50000	3
377	rolling_high_5m	0.085839	50000	3
378	rolling_low_5m	0.080870	50000	3
408	rolling_range_pct_60m	0.075073	50000	3
415	rolling_range_pct_90m	0.074815	50000	3
382	bar_range_mean_10m	0.069218	50000	3

Target: dir_p60_h60

	feature	mutual_information	n_obs_used	n_classes
529	rolling_low_90m	0.236225	50000	3
528	rolling_high_90m	0.233162	50000	3
521	rolling_high_60m	0.225218	50000	3
522	rolling_low_60m	0.222682	50000	3
514	rolling_high_30m	0.176377	50000	3
515	rolling_low_30m	0.165642	50000	3
507	rolling_high_15m	0.120340	50000	3
508	rolling_low_15m	0.112689	50000	3
500	rolling_high_10m	0.098525	50000	3
501	rolling_low_10m	0.091865	50000	3
493	rolling_high_5m	0.072963	50000	3
524	rolling_range_pct_60m	0.071666	50000	3
494	rolling_low_5m	0.070121	50000	3
531	rolling_range_pct_90m	0.069569	50000	3
505	bar_range_mean_15m	0.064590	50000	3

Target: bar_p50_h30_tp15_sl10

	feature	mutual_information	n_obs_used	n_classes
637	rolling_high_60m	0.130876	50000	3
644	rolling_high_90m	0.128645	50000	3
638	rolling_low_60m	0.121246	50000	3
645	rolling_low_90m	0.118836	50000	3
630	rolling_high_30m	0.113669	50000	3
631	rolling_low_30m	0.110022	50000	3
623	rolling_high_15m	0.096227	50000	3
624	rolling_low_15m	0.088989	50000	3
616	rolling_high_10m	0.075382	50000	3
617	rolling_low_10m	0.073045	50000	3
610	rolling_low_5m	0.054927	50000	3
647	rolling_range_pct_90m	0.054796	50000	3
609	rolling_high_5m	0.050065	50000	3
633	rolling_range_pct_30m	0.049531	50000	3
640	rolling_range_pct_60m	0.047566	50000	3

Target: bar_p50_h60_tp15_sl10

	feature	mutual_information	n_obs_used	n_classes
753	rolling_high_60m	0.188988	50000	3
760	rolling_high_90m	0.186169	50000	3
754	rolling_low_60m	0.182845	50000	3

761	rolling_low_90m	0.182622	50000	3
746	rolling_high_30m	0.160570	50000	3
747	rolling_low_30m	0.157983	50000	3
740	rolling_low_15m	0.123263	50000	3
739	rolling_high_15m	0.119773	50000	3
733	rolling_low_10m	0.099566	50000	3
732	rolling_high_10m	0.092260	50000	3
726	rolling_low_5m	0.066076	50000	3
763	rolling_range_pct_90m	0.060203	50000	3
725	rolling_high_5m	0.058345	50000	3
756	rolling_range_pct_60m	0.055749	50000	3
749	rolling_range_pct_30m	0.046844	50000	3

Target: bar_p50_h90_tp15_sl10

	feature	mutual_information	n_obs_used	n_classes
869	rolling_high_60m	0.222154	50000	3
870	rolling_low_60m	0.219345	50000	3
876	rolling_high_90m	0.219344	50000	3
877	rolling_low_90m	0.218302	50000	3
862	rolling_high_30m	0.193033	50000	3
863	rolling_low_30m	0.190705	50000	3
855	rolling_high_15m	0.140169	50000	3
856	rolling_low_15m	0.138918	50000	3
849	rolling_low_10m	0.112684	50000	3
848	rolling_high_10m	0.110090	50000	3
842	rolling_low_5m	0.079889	50000	3
841	rolling_high_5m	0.073432	50000	3
879	rolling_range_pct_90m	0.059477	50000	3
872	rolling_range_pct_60m	0.053555	50000	3
865	rolling_range_pct_30m	0.046339	50000	3

Target: bar_p40_h60_tp15_sl10

	feature	mutual_information	n_obs_used	n_classes
985	rolling_high_60m	0.170377	50000	3
992	rolling_high_90m	0.168143	50000	3
986	rolling_low_60m	0.166607	50000	3
993	rolling_low_90m	0.162161	50000	3
979	rolling_low_30m	0.146331	50000	3
978	rolling_high_30m	0.140202	50000	3
972	rolling_low_15m	0.107094	50000	3
971	rolling_high_15m	0.104251	50000	3
964	rolling_high_10m	0.087703	50000	3
965	rolling_low_10m	0.085476	50000	3
957	rolling_high_5m	0.060991	50000	3
958	rolling_low_5m	0.060755	50000	3
995	rolling_range_pct_90m	0.059743	50000	3
988	rolling_range_pct_60m	0.055718	50000	3
994	rolling_range_pts_90m	0.040961	50000	3

Target: bar_p60_h60_tp15_sl10

	feature	mutual_information	n_obs_used	n_classes
1108	rolling_high_90m	0.202693	50000	3
1102	rolling_low_60m	0.201993	50000	3
1109	rolling_low_90m	0.201907	50000	3
1101	rolling_high_60m	0.196585	50000	3
1095	rolling_low_30m	0.182806	50000	3
1094	rolling_high_30m	0.179816	50000	3
1087	rolling_high_15m	0.140657	50000	3
1088	rolling_low_15m	0.135536	50000	3
1081	rolling_low_10m	0.112472	50000	3

1080	rolling_high_10m	0.110767	50000	3
1074	rolling_low_5m	0.077578	50000	3
1073	rolling_high_5m	0.072476	50000	3
1111	rolling_range_pct_90m	0.066820	50000	3
1104	rolling_range_pct_60m	0.064457	50000	3
1097	rolling_range_pct_30m	0.046055	50000	3

Target: opc_p50_h30_tp15_sl10

	feature	mutual_information	n_obs_used	n_classes
1224	rolling_high_90m	0.140043	50000	5
1218	rolling_low_60m	0.139128	50000	5
1225	rolling_low_90m	0.137732	50000	5
1217	rolling_high_60m	0.135344	50000	5
1210	rolling_high_30m	0.113100	50000	5
1211	rolling_low_30m	0.111469	50000	5
1203	rolling_high_15m	0.086198	50000	5
1204	rolling_low_15m	0.085176	50000	5
1196	rolling_high_10m	0.071506	50000	5
1227	rolling_range_pct_90m	0.070636	50000	5
1220	rolling_range_pct_60m	0.069513	50000	5
1194	bar_range_mean_10m	0.068246	50000	5
1187	bar_range_mean_5m	0.067703	50000	5
1197	rolling_low_10m	0.067384	50000	5
1201	bar_range_mean_15m	0.067020	50000	5

Target: opc_p50_h60_tp15_sl10

	feature	mutual_information	n_obs_used	n_classes
1341	rolling_low_90m	0.198269	50000	5
1340	rolling_high_90m	0.198178	50000	5
1333	rolling_high_60m	0.184012	50000	5
1334	rolling_low_60m	0.183671	50000	5
1326	rolling_high_30m	0.146414	50000	5
1327	rolling_low_30m	0.136638	50000	5
1319	rolling_high_15m	0.097051	50000	5
1320	rolling_low_15m	0.093733	50000	5
1312	rolling_high_10m	0.085121	50000	5
1313	rolling_low_10m	0.078366	50000	5
1343	rolling_range_pct_90m	0.066203	50000	5
1336	rolling_range_pct_60m	0.065360	50000	5
1306	rolling_low_5m	0.063537	50000	5
1305	rolling_high_5m	0.062634	50000	5
1342	rolling_range_pts_90m	0.059828	50000	5

Target: opc_p50_h90_tp15_sl10

	feature	mutual_information	n_obs_used	n_classes
1456	rolling_high_90m	0.237963	50000	5
1457	rolling_low_90m	0.230442	50000	5
1449	rolling_high_60m	0.227777	50000	5
1450	rolling_low_60m	0.212537	50000	5
1442	rolling_high_30m	0.169140	50000	5
1443	rolling_low_30m	0.160785	50000	5
1435	rolling_high_15m	0.117535	50000	5
1436	rolling_low_15m	0.112777	50000	5
1428	rolling_high_10m	0.095698	50000	5
1429	rolling_low_10m	0.092516	50000	5
1421	rolling_high_5m	0.076054	50000	5
1422	rolling_low_5m	0.071207	50000	5
1459	rolling_range_pct_90m	0.069091	50000	5
1452	rolling_range_pct_60m	0.061647	50000	5
1458	rolling_range_pts_90m	0.059588	50000	5

Target: opc_p40_h60_tp15_sl10

	feature	mutual_information	n_obs_used	n_classes
1572	rolling_high_90m	0.232914	50000	5
1573	rolling_low_90m	0.229547	50000	5
1565	rolling_high_60m	0.217963	50000	5
1566	rolling_low_60m	0.214883	50000	5
1558	rolling_high_30m	0.169321	50000	5
1559	rolling_low_30m	0.164794	50000	5
1551	rolling_high_15m	0.117224	50000	5
1552	rolling_low_15m	0.114689	50000	5
1544	rolling_high_10m	0.098613	50000	5
1545	rolling_low_10m	0.094130	50000	5
1568	rolling_range_pct_60m	0.077630	50000	5
1575	rolling_range_pct_90m	0.075388	50000	5
1537	rolling_high_5m	0.075171	50000	5
1538	rolling_low_5m	0.074771	50000	5
1561	rolling_range_pct_30m	0.068910	50000	5

Target: opc_p60_h60_tp15_sl10

	feature	mutual_information	n_obs_used	n_classes
1688	rolling_high_90m	0.158247	50000	5
1689	rolling_low_90m	0.157862	50000	5
1681	rolling_high_60m	0.151967	50000	5
1682	rolling_low_60m	0.144550	50000	5
1674	rolling_high_30m	0.117682	50000	5
1675	rolling_low_30m	0.108407	50000	5
1667	rolling_high_15m	0.080452	50000	5
1668	rolling_low_15m	0.076537	50000	5
1660	rolling_high_10m	0.067926	50000	5
1661	rolling_low_10m	0.066110	50000	5
1691	rolling_range_pct_90m	0.057685	50000	5
1684	rolling_range_pct_60m	0.055120	50000	5
1654	rolling_low_5m	0.051407	50000	5
1690	rolling_range_pts_90m	0.050864	50000	5
1653	rolling_high_5m	0.050139	50000	5

Observaciones del ranking resumido por target

- Los resultados muestran una diferencia importante entre los targets DIR y BAR cuando se usa Spearman / IC.
- Para los targets DIR, los valores de Spearman / IC son muy bajos. Los mejores valores se ubican alrededor de 0.01 a 0.015, lo que sugiere que la relación monotónica univariada entre las features actuales y la dirección futura es débil.
- En DIR aparecen principalmente variables como close_pos_range, close_dist_high, close_dist_low, retornos pasados y algunos rolling_high/rolling_low. Sin embargo, la magnitud de la señal es baja, por lo que no conviene interpretar estos resultados como una señal direccional fuerte.
- Para los targets BAR, la señal univariada es más clara. Los mejores valores de Spearman / IC se ubican aproximadamente entre 0.09 y 0.12.
- Las features más relevantes para BAR son principalmente variables asociadas a rango y volatilidad reciente: bar_range_mean, bar_range_std, vol_ret_1m y

rolling_range_pts.

- Este resultado es coherente con la naturaleza del target BAR, ya que BAR mide si una barrera operativa fue alcanzada dentro de una ventana futura. Por lo tanto, un mercado con mayor rango o volatilidad reciente tiene más probabilidad de tocar TP o SL.
- El target bar_p50_h30_tp15_sl10 muestra el mayor IC observado, cercano a 0.12, seguido por bar_p60_h60_tp15_sl10. Esto sugiere que los targets BAR de 30 y 60 minutos capturan mejor la relación con la volatilidad reciente que los targets de 90 minutos.
- En Mutual Information aparece un patrón distinto: las features dominantes son rolling_high y rolling_low, especialmente en ventanas de 60 y 90 minutos.
- Este comportamiento se observa en DIR, BAR y OPC. Los valores de MI son especialmente altos en targets DIR y OPC de horizontes 60 y 90 minutos.
- Sin embargo, este resultado debe interpretarse con cautela. rolling_high y rolling_low contienen nivel absoluto de precio, por lo que podrían estar capturando efectos de tendencia, año, régimen de mercado o cambios estructurales del dataset, y no necesariamente una señal operativa robusta.
- En OPC también aparece señal relevante por MI, especialmente en opc_p50_h60, opc_p50_h90 y opc_p40_h60. Esto es importante porque OPC es el target más cercano a una lógica operativa completa.
- En conjunto, Spearman / IC y Mutual Information no están contando exactamente la misma historia. Spearman destaca rango y volatilidad reciente para BAR, mientras que MI destaca niveles rolling de precio para DIR, BAR y OPC.
- Por este motivo, no conviene seleccionar features automáticamente solo por el ranking de MI. Antes de tomar decisiones, es necesario revisar estabilidad por régimen intradiario y por año dentro del período development.

Conclusión parcial

- El análisis univariado muestra que las features construidas contienen señal preliminar, pero la señal no es homogénea entre targets.
- Los targets BAR presentan la relación más interpretable bajo Spearman / IC, principalmente asociada a volatilidad y rango reciente.
- Los targets DIR muestran una señal monotónica débil, aunque Mutual Information detecta dependencias no lineales más fuertes, dominadas por rolling_high y rolling_low.
- Los targets OPC muestran resultados prometedores en Mutual Information, pero todavía requieren validación por régimen y por año.

- Por lo tanto, estos rankings deben usarse como diagnóstico exploratorio, no como selección final de features.

La alerta principal es esta: BAR parece más interpretable con features de volatilidad/rango; MI favorece demasiado rolling_high/rolling_low, que podrían estar capturando nivel de mercado y no señal operativa limpia.

7.5 Señal por régimen intradiario

Este análisis permite revisar si una feature mantiene señal en distintos regímenes intradiarios o si su relación con el target depende fuertemente del horario operativo.

```
In [74]: # 7.5 Spearman / IC por régimen intradiario para DIR y BAR
# =====

def compute_grouped_spearman_ic(
    df,
    feature_cols,
    target_cols,
    group_col,
    min_obs=5_000
):
    """
    Calcula Spearman IC por grupo.
    """
    results = []

    for group_value, df_group in df.groupby(group_col):
        for target in target_cols:
            df_target = df_group[feature_cols + [target]].copy()
            df_target = df_target[df_target[target].notna()]

            for feature in feature_cols:
                pair = (
                    df_target[[feature, target]]
                    .replace([np.inf, -np.inf], np.nan)
                    .dropna()
                )

                n_obs = len(pair)

                if n_obs < min_obs:
                    ic = np.nan
                else:
                    ic = pair[feature].corr(pair[target], method="spearman")

                results.append({
                    group_col: group_value,
                    "target": target,
                    "feature": feature,
                    "spearman_ic": ic,
                    "abs_spearman_ic": abs(ic) if pd.notna(ic) else np.nan,
                    "n_obs": n_obs,
                })

    return pd.DataFrame(results)
```

```
df_ic_by_regime = compute_grouped_spearman_ic(  
    df=df_signal_dev,  
    feature_cols=feature_cols_signal,  
    target_cols=ic_target_cols,  
    group_col="regime_id",  
    min_obs=5_000,  
)  
  
df_ic_by_regime = df_ic_by_regime.sort_values(  
    ["target", "regime_id", "abs_spearman_ic"],  
    ascending=[True, True, False]  
)  
  
print("IC por régimen intradiario calculado")  
print("=" * 80)  
print(f"Resultados: {df_ic_by_regime.shape}")  
  
print("\nTop 50 relaciones feature-target-régimen:")  
print(  
    df_ic_by_regime  
    .sort_values("abs_spearman_ic", ascending=False)  
    .head(50)  
)
```

IC por régimen intradiario calculado

Resultados: (5800, 6)

Top 50 relaciones feature-target-régimen:

	regime_id	target	feature	spearman_ic \
4108	3	bar_p50_h30_tp15_sl10	bar_range_mean_30m	0.212350
4101	3	bar_p50_h30_tp15_sl10	bar_range_mean_15m	0.212149
4094	3	bar_p50_h30_tp15_sl10	bar_range_mean_10m	0.210830
4115	3	bar_p50_h30_tp15_sl10	bar_range_mean_60m	0.207515
4087	3	bar_p50_h30_tp15_sl10	bar_range_mean_5m	0.203477
4572	3	bar_p60_h60_tp15_sl10	bar_range_mean_30m	0.203227
4565	3	bar_p60_h60_tp15_sl10	bar_range_mean_15m	0.201363
4122	3	bar_p50_h30_tp15_sl10	bar_range_mean_90m	0.199443
4558	3	bar_p60_h60_tp15_sl10	bar_range_mean_10m	0.197980
4579	3	bar_p60_h60_tp15_sl10	bar_range_mean_60m	0.196597
4116	3	bar_p50_h30_tp15_sl10	bar_range_std_60m	0.196131
4224	3	bar_p50_h60_tp15_sl10	bar_range_mean_30m	0.194941
4217	3	bar_p50_h60_tp15_sl10	bar_range_mean_15m	0.192428
4109	3	bar_p50_h30_tp15_sl10	bar_range_std_30m	0.192160
4551	3	bar_p60_h60_tp15_sl10	bar_range_mean_5m	0.191640
4231	3	bar_p50_h60_tp15_sl10	bar_range_mean_60m	0.190160
4123	3	bar_p50_h30_tp15_sl10	bar_range_std_90m	0.189510
4210	3	bar_p50_h60_tp15_sl10	bar_range_mean_10m	0.189460
4586	3	bar_p60_h60_tp15_sl10	bar_range_mean_90m	0.188895
4114	3	bar_p50_h30_tp15_sl10	vol_ret_1m_60m	0.185006
4580	3	bar_p60_h60_tp15_sl10	bar_range_std_60m	0.183765
4107	3	bar_p50_h30_tp15_sl10	vol_ret_1m_30m	0.183621
4203	3	bar_p50_h60_tp15_sl10	bar_range_mean_5m	0.183591
4238	3	bar_p50_h60_tp15_sl10	bar_range_mean_90m	0.183222
621	0	bar_p50_h30_tp15_sl10	bar_range_mean_15m	0.182469
4573	3	bar_p60_h60_tp15_sl10	bar_range_std_30m	0.182379
614	0	bar_p50_h30_tp15_sl10	bar_range_mean_10m	0.181236
628	0	bar_p50_h30_tp15_sl10	bar_range_mean_30m	0.179826
4102	3	bar_p50_h30_tp15_sl10	bar_range_std_15m	0.179436
4098	3	bar_p50_h30_tp15_sl10	rolling_range_pts_10m	0.178990
4121	3	bar_p50_h30_tp15_sl10	vol_ret_1m_90m	0.178630
4105	3	bar_p50_h30_tp15_sl10	rolling_range_pts_15m	0.178203
4571	3	bar_p60_h60_tp15_sl10	vol_ret_1m_30m	0.178010
4578	3	bar_p60_h60_tp15_sl10	vol_ret_1m_60m	0.177438
635	0	bar_p50_h30_tp15_sl10	bar_range_mean_60m	0.177341
4456	3	bar_p40_h60_tp15_sl10	bar_range_mean_30m	0.176575
4587	3	bar_p60_h60_tp15_sl10	bar_range_std_90m	0.176567
4340	3	bar_p50_h90_tp15_sl10	bar_range_mean_30m	0.176299
4091	3	bar_p50_h30_tp15_sl10	rolling_range_pts_5m	0.176091
4232	3	bar_p50_h60_tp15_sl10	bar_range_std_60m	0.175995
4100	3	bar_p50_h30_tp15_sl10	vol_ret_1m_15m	0.175713
4112	3	bar_p50_h30_tp15_sl10	rolling_range_pts_30m	0.175293
4449	3	bar_p40_h60_tp15_sl10	bar_range_mean_15m	0.174490
4119	3	bar_p50_h30_tp15_sl10	rolling_range_pts_60m	0.173929
4126	3	bar_p50_h30_tp15_sl10	rolling_range_pts_90m	0.173876
607	0	bar_p50_h30_tp15_sl10	bar_range_mean_5m	0.173849
4333	3	bar_p50_h90_tp15_sl10	bar_range_mean_15m	0.173713
2927	2	bar_p50_h30_tp15_sl10	bar_range_mean_5m	0.173241
4347	3	bar_p50_h90_tp15_sl10	bar_range_mean_60m	0.173049
4225	3	bar_p50_h60_tp15_sl10	bar_range_std_30m	0.172860

	abs_spearman_ic	n_obs
4108	0.212350	179346
4101	0.212149	179346

4094	0.210830	179346
4115	0.207515	179346
4087	0.203477	179346
4572	0.203227	128748
4565	0.201363	128748
4122	0.199443	179346
4558	0.197980	128748
4579	0.196597	128748
4116	0.196131	179346
4224	0.194941	160539
4217	0.192428	160539
4109	0.192160	179346
4551	0.191640	128748
4231	0.190160	160539
4123	0.189510	179346
4210	0.189460	160539
4586	0.188895	128748
4114	0.185006	179346
4580	0.183765	128748
4107	0.183621	179346
4203	0.183591	160539
4238	0.183222	160539
621	0.182469	134451
4573	0.182379	128748
614	0.181236	137145
628	0.179826	126377
4102	0.179436	179346
4098	0.178990	179346
4121	0.178630	179346
4105	0.178203	179346
4571	0.178010	128748
4578	0.177438	128748
635	0.177341	111180
4456	0.176575	193763
4587	0.176567	128748
4340	0.176299	143216
4091	0.176091	179346
4232	0.175995	160539
4100	0.175713	179346
4112	0.175293	179346
4449	0.174490	193763
4119	0.173929	179346
4126	0.173876	179346
607	0.173849	139880
4333	0.173713	143216
2927	0.173241	35628
4347	0.173049	143216
4225	0.172860	160539

Observaciones del análisis por régimen intradiario

- La señal más fuerte aparece claramente en el régimen 3, Regular, especialmente para targets BAR.
- El IC máximo sube hasta aproximadamente 0.21, bastante más alto que el IC global observado antes, que estaba cerca de 0.12. Esto indica que la señal mejora cuando se analiza por régimen.

- Los targets que más aparecen son bar_p50_h30_tp15_sl10, bar_p50_h60_tp15_sl10, bar_p50_h90_tp15_sl10, bar_p40_h60_tp15_sl10 y bar_p60_h60_tp15_sl10.
- Las features dominantes siguen siendo variables de rango y volatilidad reciente: bar_range_mean, bar_range_std, vol_ret_1m y rolling_range_pts.
- Esto refuerza la idea de que los targets BAR están más relacionados con la expansión de rango/volatilidad que con una dirección pura del mercado.
- No aparecen targets DIR entre las principales relaciones. Esto confirma que la señal direccional sigue siendo débil a nivel univariado.
- También aparecen señales relevantes en el régimen 0, Overnight, y en menor medida en el régimen 2, Opening, pero el régimen Regular domina claramente el ranking.
- El resultado es importante porque muestra que la señal no es homogénea durante todo el día. Hay regímenes donde las features parecen tener más capacidad explicativa.
- Operativamente, esto sugiere que más adelante podría ser conveniente entrenar/evaluar modelos segmentando por régimen o, como mínimo, incluir regime_id como variable de contexto.
- Todavía no es una decisión operativa final. Es una evidencia a favor de analizar BAR y OPC con especial atención al régimen Regular.

Conclusión corta:

El análisis por régimen confirma que la señal más clara está en targets BAR y se concentra principalmente en el régimen Regular. La señal está asociada a volatilidad y rango reciente, no a dirección pura.

7.6 Señal por año dentro de development

Este análisis permite revisar estabilidad temporal dentro del período de desarrollo.

Una feature puede parecer útil en el agregado, pero si solo funciona en un año específico, puede tratarse de una señal inestable.

```
In [75]: # 7.6 Spearman / IC por año para DIR y BAR
# =====

df_ic_by_year = compute_grouped_spearman_ic(
    df=df_signal_dev,
    feature_cols=feature_cols_signal,
    target_cols=ic_target_cols,
    group_col="year",
    min_obs=5_000,
)

df_ic_by_year = df_ic_by_year.sort_values(
    ["target", "year", "abs_spearman_ic"],
```

```
        ascending=[True, True, False]
    )

    print("IC por año calculado dentro de development")
    print("=" * 80)
    print(f"Resultados: {df_ic_by_year.shape}")

    print("\nTop 50 relaciones feature-target-año:")
    print(
        df_ic_by_year
        .sort_values("abs_spearman_ic", ascending=False)
        .head(50)
    )
```

IC por año calculado dentro de development

Resultados: (5800, 6)

Top 50 relaciones feature-target-año:

	year	target	feature	spearman_ic	\
1091	2020	bar_p60_h60_tp15_sl10	vol_ret_1m_30m	0.182028	
1111	2020	bar_p60_h60_tp15_sl10	rolling_range_pct_90m	0.180849	
1098	2020	bar_p60_h60_tp15_sl10	vol_ret_1m_60m	0.178898	
1084	2020	bar_p60_h60_tp15_sl10	vol_ret_1m_15m	0.178448	
1104	2020	bar_p60_h60_tp15_sl10	rolling_range_pct_60m	0.178239	
1105	2020	bar_p60_h60_tp15_sl10	vol_ret_1m_90m	0.175966	
1097	2020	bar_p60_h60_tp15_sl10	rolling_range_pct_30m	0.175857	
1077	2020	bar_p60_h60_tp15_sl10	vol_ret_1m_10m	0.173380	
1110	2020	bar_p60_h60_tp15_sl10	rolling_range_pts_90m	0.173128	
736	2020	bar_p50_h60_tp15_sl10	vol_ret_1m_15m	0.172099	
743	2020	bar_p50_h60_tp15_sl10	vol_ret_1m_30m	0.171896	
1090	2020	bar_p60_h60_tp15_sl10	rolling_range_pct_15m	0.171887	
1096	2020	bar_p60_h60_tp15_sl10	rolling_range_pts_30m	0.171544	
1092	2020	bar_p60_h60_tp15_sl10	bar_range_mean_30m	0.171396	
763	2020	bar_p50_h60_tp15_sl10	rolling_range_pct_90m	0.171356	
1103	2020	bar_p60_h60_tp15_sl10	rolling_range_pts_60m	0.171096	
1085	2020	bar_p60_h60_tp15_sl10	bar_range_mean_15m	0.170729	
757	2020	bar_p50_h60_tp15_sl10	vol_ret_1m_90m	0.170538	
1083	2020	bar_p60_h60_tp15_sl10	rolling_range_pct_10m	0.170365	
1078	2020	bar_p60_h60_tp15_sl10	bar_range_mean_10m	0.169926	
729	2020	bar_p50_h60_tp15_sl10	vol_ret_1m_10m	0.169343	
750	2020	bar_p50_h60_tp15_sl10	vol_ret_1m_60m	0.168971	
730	2020	bar_p50_h60_tp15_sl10	bar_range_mean_10m	0.168055	
749	2020	bar_p50_h60_tp15_sl10	rolling_range_pct_30m	0.168031	
756	2020	bar_p50_h60_tp15_sl10	rolling_range_pct_60m	0.167916	
737	2020	bar_p50_h60_tp15_sl10	bar_range_mean_15m	0.167678	
1099	2020	bar_p60_h60_tp15_sl10	bar_range_mean_60m	0.167498	
1089	2020	bar_p60_h60_tp15_sl10	rolling_range_pts_15m	0.167294	
762	2020	bar_p50_h60_tp15_sl10	rolling_range_pts_90m	0.166099	
1071	2020	bar_p60_h60_tp15_sl10	bar_range_mean_5m	0.166000	
1082	2020	bar_p60_h60_tp15_sl10	rolling_range_pts_10m	0.165672	
735	2020	bar_p50_h60_tp15_sl10	rolling_range_pct_10m	0.165594	
744	2020	bar_p50_h60_tp15_sl10	bar_range_mean_30m	0.165471	
723	2020	bar_p50_h60_tp15_sl10	bar_range_mean_5m	0.165419	
1093	2020	bar_p60_h60_tp15_sl10	bar_range_std_30m	0.165416	
1076	2020	bar_p60_h60_tp15_sl10	rolling_range_pct_5m	0.165385	
748	2020	bar_p50_h60_tp15_sl10	rolling_range_pts_30m	0.164352	
742	2020	bar_p50_h60_tp15_sl10	rolling_range_pct_15m	0.164019	
728	2020	bar_p50_h60_tp15_sl10	rolling_range_pct_5m	0.163699	
755	2020	bar_p50_h60_tp15_sl10	rolling_range_pts_60m	0.163326	
1106	2020	bar_p60_h60_tp15_sl10	bar_range_mean_90m	0.162109	
1086	2020	bar_p60_h60_tp15_sl10	bar_range_std_15m	0.161974	
734	2020	bar_p50_h60_tp15_sl10	rolling_range_pts_10m	0.161548	
751	2020	bar_p50_h60_tp15_sl10	bar_range_mean_60m	0.161512	
1075	2020	bar_p60_h60_tp15_sl10	rolling_range_pts_5m	0.160752	
738	2020	bar_p50_h60_tp15_sl10	bar_range_std_15m	0.160523	
741	2020	bar_p50_h60_tp15_sl10	rolling_range_pts_15m	0.159912	
727	2020	bar_p50_h60_tp15_sl10	rolling_range_pts_5m	0.159676	
1070	2020	bar_p60_h60_tp15_sl10	vol_ret_1m_5m	0.159433	
758	2020	bar_p50_h60_tp15_sl10	bar_range_mean_90m	0.159342	

	abs_spearman_ic	n_obs
1091	0.182028	43811
1111	0.180849	39455

1098	0.178898	41595
1084	0.178448	44826
1104	0.178239	41672
1105	0.175966	39385
1097	0.175857	43884
1077	0.173380	45161
1110	0.173128	39455
736	0.172099	56988
743	0.171896	55733
1090	0.171887	44895
1096	0.171544	43884
1092	0.171396	43884
763	0.171356	50141
1103	0.171096	41672
1085	0.170729	44895
757	0.170538	50050
1083	0.170365	45230
1078	0.169926	45230
729	0.169343	57404
750	0.168971	52890
730	0.168055	57488
749	0.168031	55820
756	0.167916	52981
737	0.167678	57072
1099	0.167498	41672
1089	0.167294	44895
762	0.166099	50141
1071	0.166000	45582
1082	0.165672	45230
735	0.165594	57488
744	0.165471	55820
723	0.165419	57926
1093	0.165416	43884
1076	0.165385	45582
748	0.164352	55820
742	0.164019	57072
728	0.163699	57926
755	0.163326	52981
1106	0.162109	39455
1086	0.161974	44895
734	0.161548	57488
751	0.161512	52981
1075	0.160752	45582
738	0.160523	57072
741	0.159912	57072
727	0.159676	57926
1070	0.159433	45511
758	0.159342	50141

Observaciones del análisis por año

- El Top 50 está dominado completamente por el año 2020.
- La señal más fuerte aparece en targets BAR, especialmente: bar_p60_h60_tp15_sl10
bar_p50_h60_tp15_sl10
- Las features dominantes vuelven a ser de volatilidad/rango: vol_ret_1m,
rolling_range_pct, rolling_range_pts y bar_range_mean.

- Los IC más altos están alrededor de 0.16 a 0.18, lo cual es relevante, pero el hecho de que aparezcan concentrados en 2020 es una alerta.
- Esto sugiere que la señal puede estar muy influenciada por un año excepcionalmente volátil, no necesariamente por una relación estable en todo el período development.
- No aparecen en el Top 50 relaciones de 2021, 2022, 2023 o 2024. Esto no significa que no tengan señal, pero sí indica que la señal más fuerte está concentrada en 2020.
- La lectura es consistente con lo visto antes: BAR responde mejor a condiciones de rango y volatilidad que DIR.
- Pero para tomar decisiones, ahora necesitamos ver si esas mismas features mantienen IC razonable en otros años. Si solo funcionan en 2020, no son confiables para modelado general.

Conclusión puntual:

La señal por año confirma que BAR tiene relación con volatilidad/rango, pero también muestra una posible dependencia fuerte del año 2020. Antes de avanzar a una decisión operativa o selección de features, hay que verificar estabilidad entre 2021 y 2024.

7.7 Consolidación de outputs del punto 7

```
In [77]: # 7.7 Consolidación de outputs del punto 7
# =====

stage05_signal_analysis_registry = {
    "feature_cols_signal": feature_cols_signal,
    "dir_target_cols": dir_target_cols,
    "bar_target_cols": bar_target_cols,
    "opc_target_numeric_cols": opc_target_numeric_cols,
    "target_cols_for_signal": target_cols_for_signal,
    "ic_target_cols": ic_target_cols,
    "mi_sample_size": MI_SAMPLE_SIZE,
    "random_state": RANDOM_STATE,
}

print("Outputs del análisis univariado de señal")
print("=" * 80)

print("DataFrames creados:")
print("- df_ic_results")
print("- df_mi_results")
print("- df_ic_by_regime")
print("- df_ic_by_year")
print("- stage05_signal_analysis_registry")

print("\nFeatures evaluadas:")
print(len(feature_cols_signal))

print("\nTargets evaluados en IC:")
```

```
print(len(ic_target_cols))

print("\nTargets evaluados en MI:")
print(len(target_cols_for_signal))
```

Outputs del análisis univariado de señal

=====

DataFrames creados:

- df_ic_results
- df_mi_results
- df_ic_by_regime
- df_ic_by_year
- stage05_signal_analysis_registry

Features evaluadas:

116

Targets evaluados en IC:

10

Targets evaluados en MI:

15

En este punto se consolidaron los objetos principales generados durante el análisis univariado de señal.

Los outputs quedan registrados para ser utilizados en la interpretación final del punto 7 y en las etapas posteriores del Stage 05.

Esta consolidación no implica selección final de features ni decisión operativa.

7.8 Interpretación por tipo de target 7.9 Cierre del punto 7

Luego: 8. Construcción del dataset predictivo final para modelado

7.8 Robustez de señal BAR controlando régimen y 2020

```
In [78]: # 7.8 Robustez de señal BAR controlando régimen y año 2020
# =====

bar_targets_to_check = bar_target_cols.copy()

bar_signal_scenarios = {
    "dev_all": df_signal_dev,
    "dev_no_2020": df_signal_dev[df_signal_dev["year"] != 2020],
    "regime_3_all": df_signal_dev[df_signal_dev["regime_id"] == 3],
    "regime_3_no_2020": df_signal_dev[
        (df_signal_dev["regime_id"] == 3)
        & (df_signal_dev["year"] != 2020)
    ],
}

print("Escenarios para robustez BAR")
print("=" * 80)
```

```
for name, df_tmp in bar_signal_scenarios.items():
    print(f"{name:<20} shape={df_tmp.shape}")
```

Escenarios para robustez BAR

```
=====
dev_all          shape=(818835, 147)
dev_no_2020      shape=(663360, 147)
regime_3_all     shape=(355500, 147)
regime_3_no_2020 shape=(288000, 147)
```

```
In [79]: # 7.8.1 Cálculo de IC BAR por escenario
# =====

def compute_bar_ic_by_scenario(
    scenarios,
    feature_cols,
    target_cols,
    min_obs=10_000
):
    results = []

    for scenario_name, df_scenario in scenarios.items():
        print("=" * 80)
        print(f"Procesando escenario: {scenario_name}")

        df_ic_tmp = compute_spearman_ic(
            df=df_scenario,
            feature_cols=feature_cols,
            target_cols=target_cols,
            min_obs=min_obs,
        )

        df_ic_tmp["scenario"] = scenario_name
        results.append(df_ic_tmp)

        print(f"Resultados: {df_ic_tmp.shape}")

    return pd.concat(results, ignore_index=True)

df_bar_ic_robustness = compute_bar_ic_by_scenario(
    scenarios=bar_signal_scenarios,
    feature_cols=feature_cols_signal,
    target_cols=bar_targets_to_check,
    min_obs=10_000,
)

print("\nIC BAR por escenario calculado")
print("=" * 80)
print(df_bar_ic_robustness.shape)
```

```
=====
Procesando escenario: dev_all
```

```
Resultados: (580, 6)
=====
```

```
Procesando escenario: dev_no_2020
```

```
Resultados: (580, 6)
=====
```

```
Procesando escenario: regime_3_all
```

```
Resultados: (580, 6)
=====
```

```
Procesando escenario: regime_3_no_2020
```

```
Resultados: (580, 6)
=====
```

```
IC BAR por escenario calculado
```

```
=====
(2320, 6)
```

```
In [80]: # 7.8.2 Resumen por target y escenario
# =====

TOP_N = 10

summary_rows = []

for scenario in df_bar_ic_robustness["scenario"].unique():
    for target in bar_targets_to_check:
        df_tmp = (
            df_bar_ic_robustness[
                (df_bar_ic_robustness["scenario"] == scenario)
                & (df_bar_ic_robustness["target"] == target)
            ]
            .dropna(subset=["abs_spearman_ic"])
            .sort_values("abs_spearman_ic", ascending=False)
        )

        if len(df_tmp) == 0:
            continue

        top_row = df_tmp.iloc[0]
        top_n = df_tmp.head(TOP_N)

        summary_rows.append({
            "scenario": scenario,
            "target": target,
            "top_feature": top_row["feature"],
            "top_ic": top_row["spearman_ic"],
            "top_abs_ic": top_row["abs_spearman_ic"],
            "top10_mean_abs_ic": top_n["abs_spearman_ic"].mean(),
            "top10_median_abs_ic": top_n["abs_spearman_ic"].median(),
            "n_obs_top": top_row["n_obs"],
        })

df_bar_ic_robustness_summary = pd.DataFrame(summary_rows)

print("Resumen de robustez BAR")
print("=" * 80)

print(
    df_bar_ic_robustness_summary
```

```
.sort_values(["target", "scenario"])
)
```

Resumen de robustez BAR

	scenario	target	top_feature	top_ic \
3	dev_all	bar_p40_h60_tp15_sl10	bar_range_mean_10m	0.098774
8	dev_no_2020	bar_p40_h60_tp15_sl10	bar_range_mean_10m	0.090039
13	regime_3_all	bar_p40_h60_tp15_sl10	bar_range_mean_30m	0.176575
18	regime_3_no_2020	bar_p40_h60_tp15_sl10	bar_range_mean_30m	0.167701
0	dev_all	bar_p50_h30_tp15_sl10	bar_range_mean_10m	0.120001
5	dev_no_2020	bar_p50_h30_tp15_sl10	bar_range_mean_10m	0.115910
10	regime_3_all	bar_p50_h30_tp15_sl10	bar_range_mean_30m	0.212350
15	regime_3_no_2020	bar_p50_h30_tp15_sl10	bar_range_mean_30m	0.206884
1	dev_all	bar_p50_h60_tp15_sl10	bar_range_mean_10m	0.107532
6	dev_no_2020	bar_p50_h60_tp15_sl10	bar_range_mean_10m	0.095957
11	regime_3_all	bar_p50_h60_tp15_sl10	bar_range_mean_30m	0.194941
16	regime_3_no_2020	bar_p50_h60_tp15_sl10	bar_range_mean_30m	0.186178
2	dev_all	bar_p50_h90_tp15_sl10	bar_range_mean_10m	0.092713
7	dev_no_2020	bar_p50_h90_tp15_sl10	bar_range_mean_10m	0.083624
12	regime_3_all	bar_p50_h90_tp15_sl10	bar_range_mean_30m	0.176299
17	regime_3_no_2020	bar_p50_h90_tp15_sl10	bar_range_mean_30m	0.166107
4	dev_all	bar_p60_h60_tp15_sl10	bar_range_mean_15m	0.112821
9	dev_no_2020	bar_p60_h60_tp15_sl10	bar_range_mean_10m	0.102449
14	regime_3_all	bar_p60_h60_tp15_sl10	bar_range_mean_30m	0.203227
19	regime_3_no_2020	bar_p60_h60_tp15_sl10	bar_range_mean_30m	0.195881
	top_abs_ic	top10_mean_abs_ic	top10_median_abs_ic	n_obs_top
3	0.098774	0.095714	0.095138	444640
8	0.090039	0.087781	0.087434	373852
13	0.176575	0.165055	0.165840	193763
18	0.167701	0.157413	0.156143	163509
0	0.120001	0.116764	0.116391	388532
5	0.115910	0.113402	0.113385	326552
10	0.212350	0.200857	0.201460	179346
15	0.206884	0.196758	0.196056	151341
1	0.107532	0.104122	0.102571	369509
6	0.095957	0.093537	0.093557	312021
11	0.194941	0.181939	0.183407	160539
16	0.186178	0.174720	0.174152	136113
2	0.092713	0.089119	0.088083	353707
7	0.083624	0.080467	0.079581	299357
12	0.176299	0.163583	0.166583	143216
17	0.166107	0.154650	0.156273	121686
4	0.112821	0.109904	0.109943	294296
9	0.102449	0.100292	0.100484	250981
14	0.203227	0.190129	0.190267	128748
19	0.195881	0.184355	0.183078	109557

```
In [81]: # 7.8.3 Comparación rápida contra development completo
# =====

df_bar_pivot = df_bar_ic_robustness_summary.pivot_table(
    index="target",
    columns="scenario",
    values="top10_mean_abs_ic"
)

# Ratios de estabilidad
df_bar_pivot["ratio_no_2020_vs_all"] = (
```

```

df_bar_pivot["dev_no_2020"] / df_bar_pivot["dev_all"]
)

df_bar_pivot["ratio_regime3_no_2020_vs_regime3"] = (
    df_bar_pivot["regime_3_no_2020"] / df_bar_pivot["regime_3_all"]
)

print("Comparación de robustez BAR")
print("=" * 80)

print(df_bar_pivot.sort_values("dev_all", ascending=False))

```

Comparación de robustez BAR

```

=====
scenario          dev_all  dev_no_2020  regime_3_all  regime_3_no_2020  \
target
bar_p50_h30_tp15_sl10  0.116764    0.113402    0.200857    0.196758
bar_p60_h60_tp15_sl10  0.109904    0.100292    0.190129    0.184355
bar_p50_h60_tp15_sl10  0.104122    0.093537    0.181939    0.174720
bar_p40_h60_tp15_sl10  0.095714    0.087781    0.165055    0.157413
bar_p50_h90_tp15_sl10  0.089119    0.080467    0.163583    0.154650

scenario          ratio_no_2020_vs_all  ratio_regime3_no_2020_vs_regime3
target
bar_p50_h30_tp15_sl10          0.971203          0.979592
bar_p60_h60_tp15_sl10          0.912542          0.969631
bar_p50_h60_tp15_sl10          0.898343          0.960319
bar_p40_h60_tp15_sl10          0.917118          0.953703
bar_p50_h90_tp15_sl10          0.902917          0.945396

```

```

In [82]: # 7.8.4 Lectura automática simple
# =====

print("Lectura rápida por target BAR")
print("=" * 80)

for target, row in df_bar_pivot.iterrows():
    ratio_no_2020 = row["ratio_no_2020_vs_all"]
    ratio_regime3_no_2020 = row["ratio_regime3_no_2020_vs_regime3"]

    print(f"\nTarget: {target}")
    print(f"Ratio sin 2020 vs development completo: {ratio_no_2020:.3f}")
    print(f"Ratio régimen 3 sin 2020 vs régimen 3 completo: {ratio_regime3_no_2020:.3f}")

    if ratio_no_2020 >= 0.70 and ratio_regime3_no_2020 >= 0.70:
        print("Lectura: señal relativamente robusta.")
    elif ratio_no_2020 >= 0.50 or ratio_regime3_no_2020 >= 0.50:
        print("Lectura: señal parcial, requiere cautela.")
    else:
        print("Lectura: señal muy dependiente de 2020 o del régimen.")

```

Lectura rápida por target BAR

=====

Target: bar_p40_h60_tp15_sl10

Ratio sin 2020 vs development completo: 0.917

Ratio régimen 3 sin 2020 vs régimen 3 completo: 0.954

Lectura: señal relativamente robusta.

Target: bar_p50_h30_tp15_sl10

Ratio sin 2020 vs development completo: 0.971

Ratio régimen 3 sin 2020 vs régimen 3 completo: 0.980

Lectura: señal relativamente robusta.

Target: bar_p50_h60_tp15_sl10

Ratio sin 2020 vs development completo: 0.898

Ratio régimen 3 sin 2020 vs régimen 3 completo: 0.960

Lectura: señal relativamente robusta.

Target: bar_p50_h90_tp15_sl10

Ratio sin 2020 vs development completo: 0.903

Ratio régimen 3 sin 2020 vs régimen 3 completo: 0.945

Lectura: señal relativamente robusta.

Target: bar_p60_h60_tp15_sl10

Ratio sin 2020 vs development completo: 0.913

Ratio régimen 3 sin 2020 vs régimen 3 completo: 0.970

Lectura: señal relativamente robusta.

Observaciones rápidas 7.8

- La señal BAR no depende exclusivamente del año 2020.
- Al excluir 2020, todos los targets BAR conservan entre 89.8% y 97.1% de su señal promedio Top 10.
- La señal del régimen 3 también se mantiene al excluir 2020. Los ratios se ubican entre 94.5% y 98.0%.
- Esto confirma que la señal BAR es relativamente robusta y no está explicada solamente por un año excepcionalmente volátil.
- El régimen 3 sigue siendo el contexto con mayor señal. Los IC en régimen Regular son claramente superiores a los del development completo.
- El target más robusto y fuerte parece ser bar_p50_h30_tp15_sl10:
 - Top10 IC global: 0.1168
 - Top10 IC sin 2020: 0.1134
 - Top10 IC régimen 3: 0.2009
 - Ratio sin 2020: 0.971
 - Ratio régimen 3 sin 2020: 0.980
- También son buenos candidatos bar_p60_h60_tp15_sl10 y bar_p50_h60_tp15_sl10.
- El target bar_p50_h90_tp15_sl10 conserva señal, pero es el más débil del grupo.

Conclusión del 7.8

La señal BAR es robusta al excluir 2020 y se fortalece claramente dentro del régimen Regular.

Por lo tanto, BAR debe continuar como target principal de investigación predictiva.

Dentro de BAR, el candidato más fuerte es bar_p50_h30_tp15_sl10, seguido por bar_p60_h60_tp15_sl10 y bar_p50_h60_tp15_sl10.

DIR queda como benchmark direccional débil.

OPC sigue como target operativo avanzado, pero debe evaluarse después con mayor cuidado.

7.9 Robustez de señal OPC controlando régimen y 2020

```
In [83]: # 7.9 Robustez de señal OPC controlando régimen y año 2020
# =====

opc_targets_to_check = opc_target_numeric_cols.copy()

opc_signal_scenarios = {
    "dev_all": df_signal_dev,
    "dev_no_2020": df_signal_dev[df_signal_dev["year"] != 2020],
    "regime_3_all": df_signal_dev[df_signal_dev["regime_id"] == 3],
    "regime_3_no_2020": df_signal_dev[
        (df_signal_dev["regime_id"] == 3)
        & (df_signal_dev["year"] != 2020)
    ],
}

print("Escenarios para robustez OPC")
print("=" * 80)

for name, df_tmp in opc_signal_scenarios.items():
    print(f"{name:<20} shape={df_tmp.shape}")
```

Escenarios para robustez OPC

```
=====
dev_all          shape=(818835, 147)
dev_no_2020      shape=(663360, 147)
regime_3_all     shape=(355500, 147)
regime_3_no_2020 shape=(288000, 147)
```

```
In [84]: # 7.9.1 Cálculo de Mutual Information OPC por escenario
# =====

opc_mi_results = []

for scenario_name, df_scenario in opc_signal_scenarios.items():
    print("=" * 80)
    print(f"Procesando escenario OPC: {scenario_name}")

    df_mi_tmp = compute_mutual_information_fast(
        df=df_scenario,
        feature_cols=feature_cols_signal,
```

```
        target_cols=opc_targets_to_check,  
        sample_size=MI_SAMPLE_SIZE,  
        random_state=RANDOM_STATE,  
        min_obs=10_000,  
        n_neighbors=3,  
    )  
  
    df_mi_tmp["scenario"] = scenario_name  
    opc_mi_results.append(df_mi_tmp)  
  
df_opc_mi_robustness = pd.concat(opc_mi_results, ignore_index=True)  
  
print("\nMutual Information OPC por escenario calculado")  
print("=" * 80)  
print(df_opc_mi_robustness.shape)
```

```
=====
Procesando escenario OPC: dev_all
=====
Procesando target 1/5: opc_p50_h30_tp15_sl10
Observaciones totales target: 783285
Observaciones usadas:          50000
Clases:                        5
Tiempo target:                  5.03 segundos
=====
Procesando target 2/5: opc_p50_h60_tp15_sl10
Observaciones totales target: 747735
Observaciones usadas:          50000
Clases:                        5
Tiempo target:                  2.45 segundos
=====
Procesando target 3/5: opc_p50_h90_tp15_sl10
Observaciones totales target: 712185
Observaciones usadas:          50000
Clases:                        5
Tiempo target:                  2.27 segundos
=====
Procesando target 4/5: opc_p40_h60_tp15_sl10
Observaciones totales target: 747735
Observaciones usadas:          50000
Clases:                        5
Tiempo target:                  2.33 segundos
=====
Procesando target 5/5: opc_p60_h60_tp15_sl10
Observaciones totales target: 747735
Observaciones usadas:          50000
Clases:                        5
Tiempo target:                  2.27 segundos
=====
Procesando escenario OPC: dev_no_2020
=====
Procesando target 1/5: opc_p50_h30_tp15_sl10
Observaciones totales target: 634560
Observaciones usadas:          50000
Clases:                        5
Tiempo target:                  2.07 segundos
=====
Procesando target 2/5: opc_p50_h60_tp15_sl10
Observaciones totales target: 605760
Observaciones usadas:          50000
Clases:                        5
Tiempo target:                  2.06 segundos
=====
Procesando target 3/5: opc_p50_h90_tp15_sl10
Observaciones totales target: 576960
Observaciones usadas:          50000
Clases:                        5
Tiempo target:                  2.04 segundos
=====
Procesando target 4/5: opc_p40_h60_tp15_sl10
Observaciones totales target: 605760
Observaciones usadas:          50000
Clases:                        5
Tiempo target:                  2.06 segundos
=====
Procesando target 5/5: opc_p60_h60_tp15_sl10
```

Observaciones totales target: 605760
Observaciones usadas: 50000
Clases: 5
Tiempo target: 2.06 segundos

=====
Procesando escenario OPC: regime_3_all
=====

Procesando target 1/5: opc_p50_h30_tp15_sl10
Observaciones totales target: 355500
Observaciones usadas: 50000
Clases: 5
Tiempo target: 1.67 segundos

=====
Procesando target 2/5: opc_p50_h60_tp15_sl10
Observaciones totales target: 321135
Observaciones usadas: 50000
Clases: 5
Tiempo target: 1.75 segundos

=====
Procesando target 3/5: opc_p50_h90_tp15_sl10
Observaciones totales target: 285585
Observaciones usadas: 50000
Clases: 5
Tiempo target: 1.74 segundos

=====
Procesando target 4/5: opc_p40_h60_tp15_sl10
Observaciones totales target: 321135
Observaciones usadas: 50000
Clases: 5
Tiempo target: 1.79 segundos

=====
Procesando target 5/5: opc_p60_h60_tp15_sl10
Observaciones totales target: 321135
Observaciones usadas: 50000
Clases: 5
Tiempo target: 1.73 segundos

=====
Procesando escenario OPC: regime_3_no_2020
=====

Procesando target 1/5: opc_p50_h30_tp15_sl10
Observaciones totales target: 288000
Observaciones usadas: 50000
Clases: 5
Tiempo target: 1.63 segundos

=====
Procesando target 2/5: opc_p50_h60_tp15_sl10
Observaciones totales target: 260160
Observaciones usadas: 50000
Clases: 5
Tiempo target: 1.69 segundos

=====
Procesando target 3/5: opc_p50_h90_tp15_sl10
Observaciones totales target: 231360
Observaciones usadas: 50000
Clases: 5
Tiempo target: 1.74 segundos

=====
Procesando target 4/5: opc_p40_h60_tp15_sl10
Observaciones totales target: 260160
Observaciones usadas: 50000

Clases: 5
 Tiempo target: 1.74 segundos

=====
 Procesando target 5/5: opc_p60_h60_tp15_sl10
 Observaciones totales target: 260160
 Observaciones usadas: 50000
 Clases: 5
 Tiempo target: 1.88 segundos

Mutual Information OPC por escenario calculado
 =====
 (2320, 7)

```
In [85]: # 7.9.2 Resumen de robustez OPC por target y escenario
# =====

TOP_N = 10

summary_rows = []

for scenario in df_opc_mi_robustness["scenario"].unique():
    for target in opc_targets_to_check:
        df_tmp = (
            df_opc_mi_robustness[
                (df_opc_mi_robustness["scenario"] == scenario)
                & (df_opc_mi_robustness["target"] == target)
            ]
            .sort_values("mutual_information", ascending=False)
        )

        if len(df_tmp) == 0:
            continue

        top_row = df_tmp.iloc[0]
        top_n = df_tmp.head(TOP_N)

        summary_rows.append({
            "scenario": scenario,
            "target": target,
            "top_feature": top_row["feature"],
            "top_mi": top_row["mutual_information"],
            "top10_mean_mi": top_n["mutual_information"].mean(),
            "top10_median_mi": top_n["mutual_information"].median(),
            "n_obs_used": top_row["n_obs_used"],
            "n_classes": top_row["n_classes"],
        })

df_opc_mi_robustness_summary = pd.DataFrame(summary_rows)

print("Resumen de robustez OPC")
print("=" * 80)

print(
    df_opc_mi_robustness_summary
    .sort_values(["target", "scenario"])
)
```

Resumen de robustez OPC

	scenario	target	top_feature	top_mi \
3	dev_all	opc_p40_h60_tp15_sl10	rolling_high_90m	0.232914
8	dev_no_2020	opc_p40_h60_tp15_sl10	rolling_high_90m	0.257522
13	regime_3_all	opc_p40_h60_tp15_sl10	rolling_low_90m	0.363245
18	regime_3_no_2020	opc_p40_h60_tp15_sl10	rolling_low_90m	0.397367
0	dev_all	opc_p50_h30_tp15_sl10	rolling_high_90m	0.140043
5	dev_no_2020	opc_p50_h30_tp15_sl10	rolling_low_90m	0.157856
10	regime_3_all	opc_p50_h30_tp15_sl10	rolling_high_90m	0.241556
15	regime_3_no_2020	opc_p50_h30_tp15_sl10	rolling_low_90m	0.260946
1	dev_all	opc_p50_h60_tp15_sl10	rolling_low_90m	0.198269
6	dev_no_2020	opc_p50_h60_tp15_sl10	rolling_low_90m	0.223107
11	regime_3_all	opc_p50_h60_tp15_sl10	rolling_high_90m	0.311870
16	regime_3_no_2020	opc_p50_h60_tp15_sl10	rolling_low_90m	0.342606
2	dev_all	opc_p50_h90_tp15_sl10	rolling_high_90m	0.237963
7	dev_no_2020	opc_p50_h90_tp15_sl10	rolling_low_90m	0.266182
12	regime_3_all	opc_p50_h90_tp15_sl10	rolling_high_90m	0.368494
17	regime_3_no_2020	opc_p50_h90_tp15_sl10	rolling_high_90m	0.396755
4	dev_all	opc_p60_h60_tp15_sl10	rolling_high_90m	0.158247
9	dev_no_2020	opc_p60_h60_tp15_sl10	rolling_low_90m	0.186990
14	regime_3_all	opc_p60_h60_tp15_sl10	rolling_low_90m	0.257780
19	regime_3_no_2020	opc_p60_h60_tp15_sl10	rolling_low_90m	0.287139
	top10_mean_mi	top10_median_mi	n_obs_used	n_classes
3	0.165408	0.167058	50000	5
8	0.187547	0.192303	50000	5
13	0.277572	0.296522	50000	5
18	0.309594	0.337264	50000	5
0	0.109033	0.112284	50000	5
5	0.121594	0.127983	50000	5
10	0.196846	0.202271	50000	5
15	0.210792	0.227116	50000	5
1	0.140145	0.141526	50000	5
6	0.161533	0.165301	50000	5
11	0.237048	0.251200	50000	5
16	0.267021	0.291314	50000	5
2	0.165717	0.164963	50000	5
7	0.187785	0.192276	50000	5
12	0.281396	0.299282	50000	5
17	0.314597	0.337731	50000	5
4	0.112974	0.113045	50000	5
9	0.134266	0.135787	50000	5
14	0.196013	0.207387	50000	5
19	0.222198	0.240909	50000	5

```
In [86]: # 7.9.3 Comparación rápida OPC
# =====

df_opc_pivot = df_opc_mi_robustness_summary.pivot_table(
    index="target",
    columns="scenario",
    values="top10_mean_mi"
)

df_opc_pivot["ratio_no_2020_vs_all"] = (
    df_opc_pivot["dev_no_2020"] / df_opc_pivot["dev_all"]
)

df_opc_pivot["ratio_regime3_no_2020_vs_regime3"] = (
```

```

df_opc_pivot["regime_3_no_2020"] / df_opc_pivot["regime_3_all"]
)

print("Comparación de robustez OPC")
print("=" * 80)

print(df_opc_pivot.sort_values("dev_all", ascending=False))

```

Comparación de robustez OPC

```

=====
scenario          dev_all  dev_no_2020  regime_3_all  regime_3_no_2020  \
target
opc_p50_h90_tp15_sl10  0.165717    0.187785    0.281396    0.314597
opc_p40_h60_tp15_sl10  0.165408    0.187547    0.277572    0.309594
opc_p50_h60_tp15_sl10  0.140145    0.161533    0.237048    0.267021
opc_p60_h60_tp15_sl10  0.112974    0.134266    0.196013    0.222198
opc_p50_h30_tp15_sl10  0.109033    0.121594    0.196846    0.210792

scenario          ratio_no_2020_vs_all  ratio_regime3_no_2020_vs_regime3
target
opc_p50_h90_tp15_sl10                1.133166                1.117988
opc_p40_h60_tp15_sl10                1.133847                1.115364
opc_p50_h60_tp15_sl10                1.152612                1.126445
opc_p60_h60_tp15_sl10                1.188467                1.133588
opc_p50_h30_tp15_sl10                1.115200                1.070851

```

```

In [87]: # 7.9.4 Lectura rápida por target OPC
# =====

print("Lectura rápida por target OPC")
print("=" * 80)

for target, row in df_opc_pivot.iterrows():
    ratio_no_2020 = row["ratio_no_2020_vs_all"]
    ratio_regime3_no_2020 = row["ratio_regime3_no_2020_vs_regime3"]

    print(f"\nTarget: {target}")
    print(f"Ratio sin 2020 vs development completo: {ratio_no_2020:.3f}")
    print(f"Ratio régimen 3 sin 2020 vs régimen 3 completo: {ratio_regime3_no_2020:.3f}")

    if ratio_no_2020 >= 0.70 and ratio_regime3_no_2020 >= 0.70:
        print("Lectura: señal OPC relativamente robusta.")
    elif ratio_no_2020 >= 0.50 or ratio_regime3_no_2020 >= 0.50:
        print("Lectura: señal OPC parcial, requiere cautela.")
    else:
        print("Lectura: señal OPC muy dependiente de 2020 o del régimen.")

```

Lectura rápida por target OPC

=====

Target: opc_p40_h60_tp15_sl10

Ratio sin 2020 vs development completo: 1.134

Ratio régimen 3 sin 2020 vs régimen 3 completo: 1.115

Lectura: señal OPC relativamente robusta.

Target: opc_p50_h30_tp15_sl10

Ratio sin 2020 vs development completo: 1.115

Ratio régimen 3 sin 2020 vs régimen 3 completo: 1.071

Lectura: señal OPC relativamente robusta.

Target: opc_p50_h60_tp15_sl10

Ratio sin 2020 vs development completo: 1.153

Ratio régimen 3 sin 2020 vs régimen 3 completo: 1.126

Lectura: señal OPC relativamente robusta.

Target: opc_p50_h90_tp15_sl10

Ratio sin 2020 vs development completo: 1.133

Ratio régimen 3 sin 2020 vs régimen 3 completo: 1.118

Lectura: señal OPC relativamente robusta.

Target: opc_p60_h60_tp15_sl10

Ratio sin 2020 vs development completo: 1.188

Ratio régimen 3 sin 2020 vs régimen 3 completo: 1.134

Lectura: señal OPC relativamente robusta.

Observaciones rápidas OPC

- La señal OPC es robusta. Al excluir 2020, la señal no cae; al contrario, mejora.
- Los ratios sin 2020 son mayores a 1 en todos los targets OPC. Esto significa que 2020 no explica artificialmente la señal OPC.
- La señal OPC se fortalece mucho en régimen 3, Regular.
- Los mejores candidatos OPC por MI son:
 1. opc_p50_h90_tp15_sl10
 2. opc_p40_h60_tp15_sl10
 3. opc_p50_h60_tp15_sl10
- opc_p50_h30_tp15_sl10 también es robusto, pero tiene menor MI que los de 60 y 90 minutos.
- opc_p60_h60_tp15_sl10 mejora mucho al excluir 2020, pero su MI base es menor que p50_h90, p40_h60 y p50_h60.
- La señal OPC está dominada por rolling_high_90m, rolling_low_90m, rolling_high_60m y rolling_low_60m.
- Esto es positivo porque hay señal, pero también exige cautela: esas features contienen nivel absoluto de precio y podrían capturar estructura temporal o régimen de mercado.

Lectura metodológica

- OPC no depende de 2020.
- OPC mejora en régimen 3.
- OPC tiene señal más fuerte que DIR.
- OPC es más operativo que BAR, pero también más complejo.

Decisión preliminar

- DIR:
 - Benchmark débil. No foco principal.
- BAR:
 - Target principal interpretable.
 - Muy buena señal con rango/volatilidad.
 - Robusto sin 2020.
 - Fuerte en régimen 3.
- OPC:
 - Target operativo avanzado.
 - Señal robusta por ML.
 - Debe seguir como segundo foco principal, especialmente:
 - opc_p50_h90_tp15_sl10
 - opc_p40_h60_tp15_sl10
 - opc_p50_h60_tp15_sl10

Conclusión directa

Ya podemos cerrar el punto 7 con esta decisión:

Los targets que continúan al siguiente análisis son BAR y OPC.

DIR queda solo como benchmark.

BAR se conserva como target principal por interpretabilidad. OPC se conserva como target operativo avanzado por robustez y cercanía a una lógica real de trading.

El régimen 3 debe tratarse como contexto clave en modelado.

7.10 Interpretación por tipo de target y cierre del punto 7

DIR

- Los targets DIR muestran señal univariada débil. Los valores de Spearman / IC son bajos y no aparece una señal direccional clara ni suficientemente robusta.
- Decisión: DIR no será el foco principal del modelado. Se conserva solo como benchmark direccional.

BAR

- Los targets BAR muestran la señal más clara e interpretable. La señal está asociada principalmente a rango y volatilidad reciente.
- Además, la señal BAR se mantiene al excluir 2020 y se fortalece claramente en el régimen 3.
- Decisión: BAR queda como target principal de investigación predictiva.

OPC

- Los targets OPC muestran señal robusta por Mutual Information. La señal mejora al excluir 2020 y también se fortalece en el régimen 3.
- OPC es más complejo que BAR, pero es más cercano a una lógica operativa real.
- Decisión: OPC queda como target operativo avanzado para seguir investigando.

Targets que continúan

```
In [88]: # 7.10 Targets preseleccionados al cierre del punto 7
# =====

selected_targets_stage05 = {
    "DIR_benchmark": [
        "dir_p50_h60",
    ],

    "BAR_main": [
        "bar_p50_h30_tp15_sl10",
        "bar_p50_h60_tp15_sl10",
        "bar_p60_h60_tp15_sl10",
    ],

    "OPC_advanced": [
        "opc_p50_h90_tp15_sl10",
        "opc_p40_h60_tp15_sl10",
        "opc_p50_h60_tp15_sl10",
    ],
}

print("Targets preseleccionados al cierre del punto 7")
print("=" * 80)

for group, targets in selected_targets_stage05.items():
    print(f"\n{group}:")
    for target in targets:
        print(f"- {target}")
```

Targets preseleccionados al cierre del punto 7

=====

DIR_benchmark:

- dir_p50_h60

BAR_main:

- bar_p50_h30_tp15_sl10

- bar_p50_h60_tp15_sl10

- bar_p60_h60_tp15_sl10

OPC_advanced:

- opc_p50_h90_tp15_sl10

- opc_p40_h60_tp15_sl10

- opc_p50_h60_tp15_sl10

```

In [89]: # 7.10.1 Features preseleccionadas para targets BAR
# =====

bar_selected_feature_cols = [
    # Rango promedio reciente
    "bar_range_mean_5m",
    "bar_range_mean_10m",
    "bar_range_mean_15m",
    "bar_range_mean_30m",
    "bar_range_mean_60m",

    # Dispersión del rango reciente
    "bar_range_std_10m",
    "bar_range_std_15m",
    "bar_range_std_30m",
    "bar_range_std_60m",

    # Volatilidad de retornos
    "vol_ret_1m_10m",
    "vol_ret_1m_15m",
    "vol_ret_1m_30m",
    "vol_ret_1m_60m",

    # Rango rolling en puntos
    "rolling_range_pts_5m",
    "rolling_range_pts_10m",
    "rolling_range_pts_15m",
    "rolling_range_pts_30m",
    "rolling_range_pts_60m",

    # Rango rolling normalizado
    "rolling_range_pct_10m",
    "rolling_range_pct_15m",
    "rolling_range_pct_30m",
    "rolling_range_pct_60m",
]

bar_selected_feature_cols = [
    col for col in bar_selected_feature_cols
    if col in feature_cols_signal
]

print("Features preseleccionadas para BAR")
print("=" * 80)

```

```
print(f"Cantidad: {len(bar_selected_feature_cols)}")

for col in bar_selected_feature_cols:
    print(col)
```

Features preseleccionadas para BAR

```
=====
Cantidad: 22
bar_range_mean_5m
bar_range_mean_10m
bar_range_mean_15m
bar_range_mean_30m
bar_range_mean_60m
bar_range_std_10m
bar_range_std_15m
bar_range_std_30m
bar_range_std_60m
vol_ret_1m_10m
vol_ret_1m_15m
vol_ret_1m_30m
vol_ret_1m_60m
rolling_range_pts_5m
rolling_range_pts_10m
rolling_range_pts_15m
rolling_range_pts_30m
rolling_range_pts_60m
rolling_range_pct_10m
rolling_range_pct_15m
rolling_range_pct_30m
rolling_range_pct_60m
```

```
In [90]: # 7.10.2 Features preseleccionadas para targets OPC
# =====

opc_selected_feature_cols = [
    # Niveles rolling detectados por Mutual Information
    "rolling_high_30m",
    "rolling_low_30m",
    "rolling_high_60m",
    "rolling_low_60m",
    "rolling_high_90m",
    "rolling_low_90m",

    # Rango rolling normalizado
    "rolling_range_pct_30m",
    "rolling_range_pct_60m",
    "rolling_range_pct_90m",

    # Rango rolling en puntos
    "rolling_range_pts_30m",
    "rolling_range_pts_60m",
    "rolling_range_pts_90m",

    # Rango/volatilidad reciente como soporte
    "bar_range_mean_10m",
    "bar_range_mean_15m",
    "bar_range_mean_30m",
    "vol_ret_1m_30m",
    "vol_ret_1m_60m",
    "vol_ret_1m_90m",
```

```

]

opc_selected_feature_cols = [
    col for col in opc_selected_feature_cols
    if col in feature_cols_signal
]

print("Features preseleccionadas para OPC")
print("=" * 80)
print(f"Cantidad: {len(opc_selected_feature_cols)}")

for col in opc_selected_feature_cols:
    print(col)

print("\nNota:")
print("rolling_high y rolling_low quedan preseleccionadas por MI, pero deben tra
print("porque contienen nivel absoluto de precio.")

```

Features preseleccionadas para OPC

=====

Cantidad: 18

rolling_high_30m

rolling_low_30m

rolling_high_60m

rolling_low_60m

rolling_high_90m

rolling_low_90m

rolling_range_pct_30m

rolling_range_pct_60m

rolling_range_pct_90m

rolling_range_pts_30m

rolling_range_pts_60m

rolling_range_pts_90m

bar_range_mean_10m

bar_range_mean_15m

bar_range_mean_30m

vol_ret_1m_30m

vol_ret_1m_60m

vol_ret_1m_90m

Nota:

rolling_high y rolling_low quedan preseleccionadas por MI, pero deben tratarse co
n cautela

porque contienen nivel absoluto de precio.

In [91]: # 7.10.3 Registro final del punto 7

```

# =====

stage05_signal_decision_registry = {
    "selected_targets_stage05": selected_targets_stage05,
    "bar_selected_feature_cols": bar_selected_feature_cols,
    "opc_selected_feature_cols": opc_selected_feature_cols,
    "dir_role": "benchmark_only",
    "bar_role": "main_predictive_target",
    "opc_role": "advanced_operational_target",
    "main_regime_context": 3,
    "main_regime_name": "Regular",
}

print("Registro de decisión del punto 7")

```

```

print("=" * 80)

print("DIR:")
print(stage05_signal_decision_registry["dir_role"])

print("\nBAR:")
print(stage05_signal_decision_registry["bar_role"])
print(f"Features BAR: {len(bar_selected_feature_cols)}")

print("\nOPC:")
print(stage05_signal_decision_registry["opc_role"])
print(f"Features OPC: {len(opc_selected_feature_cols)}")

print("\nRégimen clave:")
print(stage05_signal_decision_registry["main_regime_context"])

```

Registro de decisión del punto 7

=====

DIR:

benchmark_only

BAR:

main_predictive_target

Features BAR: 22

OPC:

advanced_operational_target

Features OPC: 18

Régimen clave:

3

Cierre del punto 7

El análisis univariado de señal muestra que los targets DIR presentan señal débil y no serán el foco principal del modelado. Se conservarán únicamente como benchmark direccional.

Los targets BAR muestran la señal más clara, estable e interpretable. La señal está asociada principalmente a features de rango y volatilidad reciente, se mantiene al excluir el año 2020 y se fortalece dentro del régimen 3. Por este motivo, BAR queda como target principal de investigación predictiva.

Los targets OPC muestran señal robusta mediante Mutual Information, también mejoran al excluir 2020 y se fortalecen en régimen 3. Dado que representan una lógica más cercana a una operación completa, quedan como targets operativos avanzados para seguir investigando.

Al cierre del punto 7, los targets que continúan son:

DIR benchmark:

- dir_p50_h60

BAR principales:

- bar_p50_h30_tp15_sl10

- bar_p50_h60_tp15_sl10
- bar_p60_h60_tp15_sl10

OPC avanzados:

- opc_p50_h90_tp15_sl10
- opc_p40_h60_tp15_sl10
- opc_p50_h60_tp15_sl10

El régimen 3, correspondiente al período Regular, queda identificado como el contexto intradiario con mayor señal y deberá ser considerado explícitamente en los siguientes pasos del Stage 05.

7.11 Correlación entre features preseleccionadas BAR y OPC

```
In [92]: # 7.11 Correlación entre features preseleccionadas BAR y OPC
# =====

def get_high_corr_pairs(df, feature_cols, threshold=0.95, method="spearman"):
    df_tmp = (
        df[feature_cols]
        .replace([np.inf, -np.inf], np.nan)
    )

    corr_matrix = df_tmp.corr(method=method).abs()

    upper_mask = np.triu(np.ones(corr_matrix.shape), k=1).astype(bool)

    corr_pairs = (
        corr_matrix
        .where(upper_mask)
        .stack()
        .reset_index()
    )

    corr_pairs.columns = ["feature_1", "feature_2", "abs_corr"]

    high_corr_pairs = corr_pairs[
        corr_pairs["abs_corr"] >= threshold
    ].sort_values("abs_corr", ascending=False)

    return high_corr_pairs

HIGH_CORR_SELECTED_THRESHOLD = 0.95

bar_high_corr_pairs = get_high_corr_pairs(
    df=df_signal_dev,
    feature_cols=bar_selected_feature_cols,
    threshold=HIGH_CORR_SELECTED_THRESHOLD,
)

opc_high_corr_pairs = get_high_corr_pairs(
    df=df_signal_dev,
```

```
        feature_cols=opc_selected_feature_cols,
        threshold=HIGH_CORR_SELECTED_THRESHOLD,
    )

combined_selected_feature_cols = list(dict.fromkeys(
    bar_selected_feature_cols + opc_selected_feature_cols
))

combined_high_corr_pairs = get_high_corr_pairs(
    df=df_signal_dev,
    feature_cols=combined_selected_feature_cols,
    threshold=HIGH_CORR_SELECTED_THRESHOLD,
)

print("Correlación en features preseleccionadas")
print("=" * 80)

print(f"Features BAR: {len(bar_selected_feature_cols)}")
print(f"Pares BAR con correlación >= {HIGH_CORR_SELECTED_THRESHOLD}: {len(bar_high_corr_pairs)}")

print(f"\nFeatures OPC: {len(opc_selected_feature_cols)}")
print(f"Pares OPC con correlación >= {HIGH_CORR_SELECTED_THRESHOLD}: {len(opc_high_corr_pairs)}")

print(f"\nFeatures combinadas BAR + OPC: {len(combined_selected_feature_cols)}")
print(f"Pares combinados con correlación >= {HIGH_CORR_SELECTED_THRESHOLD}: {len(combined_high_corr_pairs)}")

print("\nTop pares BAR:")
print(bar_high_corr_pairs.head(30))

print("\nTop pares OPC:")
print(opc_high_corr_pairs.head(30))

print("\nTop pares combinados:")
print(combined_high_corr_pairs.head(30))
```


Correlación en features preseleccionadas

=====

Features BAR: 22

Pares BAR con correlación ≥ 0.95 : 10

Features OPC: 18

Pares OPC con correlación ≥ 0.95 : 20

Features combinadas BAR + OPC: 31

Pares combinados con correlación ≥ 0.95 : 26

Top pares BAR:

	feature_1	feature_2	abs_corr
24	bar_range_mean_10m	bar_range_mean_15m	0.989502
47	bar_range_mean_15m	bar_range_mean_30m	0.980574
70	bar_range_mean_30m	bar_range_mean_60m	0.976265
1	bar_range_mean_5m	bar_range_mean_10m	0.975120
254	vol_ret_1m_30m	vol_ret_1m_60m	0.966802
208	vol_ret_1m_10m	vol_ret_1m_15m	0.965209
25	bar_range_mean_10m	bar_range_mean_30m	0.964308
2	bar_range_mean_5m	bar_range_mean_15m	0.960572
231	vol_ret_1m_15m	vol_ret_1m_30m	0.958411
415	rolling_range_pct_10m	rolling_range_pct_15m	0.950760

Top pares OPC:

	feature_1	feature_2	abs_corr
59	rolling_low_60m	rolling_low_90m	0.999977
40	rolling_high_60m	rolling_high_90m	0.999976
21	rolling_low_30m	rolling_low_60m	0.999971
2	rolling_high_30m	rolling_high_60m	0.999970
23	rolling_low_30m	rolling_low_90m	0.999935
4	rolling_high_30m	rolling_high_90m	0.999931
1	rolling_high_30m	rolling_low_30m	0.999912
3	rolling_high_30m	rolling_low_60m	0.999871
20	rolling_low_30m	rolling_high_60m	0.999858
39	rolling_high_60m	rolling_low_60m	0.999832
5	rolling_high_30m	rolling_low_90m	0.999828
22	rolling_low_30m	rolling_high_90m	0.999802
41	rolling_high_60m	rolling_low_90m	0.999797
58	rolling_low_60m	rolling_high_90m	0.999782
77	rolling_high_90m	rolling_low_90m	0.999758
229	bar_range_mean_10m	bar_range_mean_15m	0.989502
305	vol_ret_1m_60m	vol_ret_1m_90m	0.983699
248	bar_range_mean_15m	bar_range_mean_30m	0.980574
286	vol_ret_1m_30m	vol_ret_1m_60m	0.966802
230	bar_range_mean_10m	bar_range_mean_30m	0.964308

Top pares combinados:

	feature_1	feature_2	abs_corr
802	rolling_low_60m	rolling_low_90m	0.999977
770	rolling_high_60m	rolling_high_90m	0.999976
738	rolling_low_30m	rolling_low_60m	0.999971
706	rolling_high_30m	rolling_high_60m	0.999970
740	rolling_low_30m	rolling_low_90m	0.999935
708	rolling_high_30m	rolling_high_90m	0.999931
705	rolling_high_30m	rolling_low_30m	0.999912
707	rolling_high_30m	rolling_low_60m	0.999871
737	rolling_low_30m	rolling_high_60m	0.999858
769	rolling_high_60m	rolling_low_60m	0.999832
709	rolling_high_30m	rolling_low_90m	0.999828

739	rolling_low_30m	rolling_high_90m	0.999802
771	rolling_high_60m	rolling_low_90m	0.999797
801	rolling_low_60m	rolling_high_90m	0.999782
833	rolling_high_90m	rolling_low_90m	0.999758
33	bar_range_mean_10m	bar_range_mean_15m	0.989502
402	vol_ret_1m_60m	vol_ret_1m_90m	0.983699
65	bar_range_mean_15m	bar_range_mean_30m	0.980574
97	bar_range_mean_30m	bar_range_mean_60m	0.976265
1	bar_range_mean_5m	bar_range_mean_10m	0.975120
353	vol_ret_1m_30m	vol_ret_1m_60m	0.966802
289	vol_ret_1m_10m	vol_ret_1m_15m	0.965209
34	bar_range_mean_10m	bar_range_mean_30m	0.964308
2	bar_range_mean_5m	bar_range_mean_15m	0.960572
321	vol_ret_1m_15m	vol_ret_1m_30m	0.958411
577	rolling_range_pct_10m	rolling_range_pct_15m	0.950760

Features reducidas recomendadas para BAR

```
In [93]: bar_selected_feature_cols_reduced = [
    "bar_range_mean_10m",
    "bar_range_mean_30m",
    "bar_range_std_30m",
    "vol_ret_1m_30m",
    "rolling_range_pts_15m",
    "rolling_range_pts_30m",
    "rolling_range_pct_30m",
    "rolling_range_pct_60m",
]

bar_selected_feature_cols_reduced = [
    col for col in bar_selected_feature_cols_reduced
    if col in feature_cols_signal
]
```

Features reducidas recomendadas para OPC

```
In [94]: opc_selected_feature_cols_reduced_level = [
    "rolling_high_90m",
    "rolling_range_pct_60m",
    "rolling_range_pct_90m",
    "rolling_range_pts_60m",
    "rolling_range_pts_90m",
    "bar_range_mean_30m",
    "vol_ret_1m_60m",
    "vol_ret_1m_90m",
]

opc_selected_feature_cols_reduced_level = [
    col for col in opc_selected_feature_cols_reduced_level
    if col in feature_cols_signal
]
```

```
In [95]: opc_selected_feature_cols_reduced_no_level = [
    "rolling_range_pct_60m",
    "rolling_range_pct_90m",
    "rolling_range_pts_60m",
    "rolling_range_pts_90m",
]
```

```

        "bar_range_mean_30m",
        "vol_ret_1m_60m",
        "vol_ret_1m_90m",
    ]

opc_selected_feature_cols_reduced_no_level = [
    col for col in opc_selected_feature_cols_reduced_no_level
    if col in feature_cols_signal
]

```

```

In [96]: # 7.11 Sets finales de features preseleccionadas
# =====

# -----
# BAR - set full
# Todas las features preseleccionadas para BAR
# -----

bar_feature_set_full = bar_selected_feature_cols.copy()

# -----
# BAR - set reducido
# Versión compacta para reducir redundancia entre ventanas cercanas
# -----

bar_feature_set_reduced = [
    "bar_range_mean_10m",
    "bar_range_mean_30m",
    "bar_range_std_30m",
    "vol_ret_1m_30m",
    "rolling_range_pts_15m",
    "rolling_range_pts_30m",
    "rolling_range_pct_30m",
    "rolling_range_pct_60m",
]

bar_feature_set_reduced = [
    col for col in bar_feature_set_reduced
    if col in feature_cols_signal
]

# -----
# OPC - set full
# Todas las features preseleccionadas para OPC
# -----

opc_feature_set_full = opc_selected_feature_cols.copy()

# -----
# OPC - set reducido con nivel de precio
# Conserva una feature de nivel rolling porque fue dominante en MI
# -----

opc_feature_set_reduced_level = [
    "rolling_high_90m",
    "rolling_range_pct_60m",
    "rolling_range_pct_90m",
]

```

```

    "rolling_range_pts_60m",
    "rolling_range_pts_90m",
    "bar_range_mean_30m",
    "vol_ret_1m_60m",
    "vol_ret_1m_90m",
]

opc_feature_set_reduced_level = [
    col for col in opc_feature_set_reduced_level
    if col in feature_cols_signal
]

# -----
# OPC - set reducido sin nivel de precio
# Versión más conservadora, evita rolling_high y rolling_low
# -----

opc_feature_set_reduced_no_level = [
    "rolling_range_pct_60m",
    "rolling_range_pct_90m",
    "rolling_range_pts_60m",
    "rolling_range_pts_90m",
    "bar_range_mean_30m",
    "vol_ret_1m_60m",
    "vol_ret_1m_90m",
]

opc_feature_set_reduced_no_level = [
    col for col in opc_feature_set_reduced_no_level
    if col in feature_cols_signal
]

# -----
# Registro
# -----

stage05_feature_sets_registry = {
    "BAR": {
        "full": bar_feature_set_full,
        "reduced": bar_feature_set_reduced,
    },
    "OPC": {
        "full": opc_feature_set_full,
        "reduced_level": opc_feature_set_reduced_level,
        "reduced_no_level": opc_feature_set_reduced_no_level,
    },
}

print("Sets de features preseleccionadas al cierre del punto 7")
print("=" * 80)

print("\nBAR:")
print(f"full:    {len(bar_feature_set_full)} features")
print(f"reduced:  {len(bar_feature_set_reduced)} features")

print("\nOPC:")
print(f"full:      {len(opc_feature_set_full)} features")

```

```
print(f"reduced_level: {len(opc_feature_set_reduced_level)} features")
print(f"reduced_no_level: {len(opc_feature_set_reduced_no_level)} features")
```

Sets de features preseleccionadas al cierre del punto 7

=====

BAR:

full: 22 features

reduced: 8 features

OPC:

full: 18 features

reduced_level: 8 features

reduced_no_level: 7 features

Interpretación final de los sets

BAR full: Contiene las 22 features preseleccionadas de rango, volatilidad y rolling range.

BAR reduced: Contiene una versión compacta de 8 features, eliminando redundancia entre ventanas muy correlacionadas.

OPC full: Contiene las 18 features preseleccionadas, incluyendo rolling_high y rolling_low.

OPC reduced_level: Contiene 8 features y conserva una variable de nivel absoluto de precio, rolling_high_90m, porque fue dominante en Mutual Information.

OPC reduced_no_level: Contiene 7 features y elimina rolling_high/rolling_low. Es la versión más conservadora para evitar dependencia del nivel absoluto del mercado.

8. Construcción del dataset predictivo final

En este punto se construyen los datasets predictivos finales del Stage 05.

A partir de los targets y features preseleccionados en el punto 7, se generan datasets supervisados listos para el Stage 06.

Cada dataset tendrá:

- X: features causales seleccionadas.
- y: target a predecir.
- metadata: date, year, dataset_split, regime_id, minute_of_day, contract.

Se construirán datasets separados para BAR y OPC, considerando versiones full y reduced de features.

También se generarán versiones all_regimes y regime_3, ya que el análisis previo mostró que la señal se fortalece en el régimen Regular.

8.1 Verificación de objetos necesarios

```
In [97]: # 8.1 Verificación de objetos necesarios
# =====

required_objects_stage08 = [
    "df_stage05_features_validated",
    "selected_targets_stage05",
    "stage05_feature_sets_registry",
]

missing_objects_stage08 = [
    obj for obj in required_objects_stage08
    if obj not in globals()
]

print("Verificación de objetos para el punto 8")
print("=" * 80)

if len(missing_objects_stage08) == 0:
    print("Todos los objetos requeridos están disponibles.")
else:
    print("Faltan objetos requeridos:")
    for obj in missing_objects_stage08:
        print(f"- {obj}")
```

Verificación de objetos para el punto 8

=====

Todos los objetos requeridos están disponibles.

8.2 Definición de metadata y configuraciones

```
In [98]: # 8.2 Definición de metadata y configuraciones
# =====

df_stage08_base = df_stage05_features_validated.copy().sort_index()

metadata_cols_stage08 = [
    "date",
    "year",
    "dataset_split",
    "regime_id",
    "minute_of_day",
    "contract",
]

metadata_cols_stage08 = [
    col for col in metadata_cols_stage08
    if col in df_stage08_base.columns
]

bar_targets_stage08 = selected_targets_stage05["BAR_main"]
opc_targets_stage08 = selected_targets_stage05["OPC_advanced"]

feature_sets_stage08 = {
    "BAR_full": stage05_feature_sets_registry["BAR"]["full"],
    "BAR_reduced": stage05_feature_sets_registry["BAR"]["reduced"],
    "OPC_full": stage05_feature_sets_registry["OPC"]["full"],
    "OPC_reduced_level": stage05_feature_sets_registry["OPC"]["reduced_level"],
    "OPC_reduced_no_level": stage05_feature_sets_registry["OPC"]["reduced_no_level"]
}
```

```

}

print("Configuración del punto 8")
print("=" * 80)

print("\nMetadata:")
print(metadata_cols_stage08)

print("\nTargets BAR:")
for target in bar_targets_stage08:
    print(f"- {target}")

print("\nTargets OPC:")
for target in opc_targets_stage08:
    print(f"- {target}")

print("\nFeature sets:")
for name, cols in feature_sets_stage08.items():
    print(f"{name:<25} {len(cols)} features")

```

Configuración del punto 8

=====

Metadata:

['date', 'year', 'dataset_split', 'regime_id', 'minute_of_day', 'contract']

Targets BAR:

- bar_p50_h30_tp15_sl10
- bar_p50_h60_tp15_sl10
- bar_p60_h60_tp15_sl10

Targets OPC:

- opc_p50_h90_tp15_sl10
- opc_p40_h60_tp15_sl10
- opc_p50_h60_tp15_sl10

Feature sets:

BAR_full	22 features
BAR_reduced	8 features
OPC_full	18 features
OPC_reduced_level	8 features
OPC_reduced_no_level	7 features

8.3 Función para construir datasets supervisados

```

In [99]: # 8.3 Función para construir datasets supervisados
# =====

def build_supervised_dataset(
    df,
    target_col,
    feature_cols,
    metadata_cols,
    regime_filter=None,
    dropna=True,
):
    """
    Construye un dataset supervisado para un target específico.

```

```

Incluye:
- metadata
- features seleccionadas
- target

Si regime_filter no es None, filtra por regime_id.
Si dropna=True, elimina filas con NaN en features o target.
"""

required_cols = metadata_cols + feature_cols + [target_col]

missing_cols = [
    col for col in required_cols
    if col not in df.columns
]

if len(missing_cols) > 0:
    raise ValueError(f"Columnas faltantes: {missing_cols}")

df_out = df[required_cols].copy()

if regime_filter is not None:
    df_out = df_out[df_out["regime_id"] == regime_filter].copy()

rows_before_dropna = len(df_out)

if dropna:
    df_out = df_out.dropna(subset=feature_cols + [target_col]).copy()

rows_after_dropna = len(df_out)

return df_out, {
    "target": target_col,
    "n_features": len(feature_cols),
    "regime_filter": regime_filter,
    "rows_before_dropna": rows_before_dropna,
    "rows_after_dropna": rows_after_dropna,
    "rows_removed_by_dropna": rows_before_dropna - rows_after_dropna,
}

print("Función build_supervised_dataset creada.")

```

Función build_supervised_dataset creada.

8.4 Construcción de datasets BAR

In [100...

```

# 8.4 Construcción de datasets BAR
# =====

stage08_datasets = {}
stage08_dataset_summary = []

bar_feature_set_names = [
    "BAR_full",
    "BAR_reduced",
]

regime_versions = {
    "all_regimes": None,

```



```

    "regime_3": 3,
}

for target in bar_targets_stage08:
    for feature_set_name in bar_feature_set_names:
        for regime_name, regime_filter in regime_versions.items():

            dataset_name = f"{target}__{feature_set_name}__{regime_name}"

            df_dataset, summary = build_supervised_dataset(
                df=df_stage08_base,
                target_col=target,
                feature_cols=feature_sets_stage08[feature_set_name],
                metadata_cols=metadata_cols_stage08,
                regime_filter=regime_filter,
                dropna=True,
            )

            stage08_datasets[dataset_name] = df_dataset

            summary.update({
                "dataset_name": dataset_name,
                "target_type": "BAR",
                "feature_set_name": feature_set_name,
                "regime_name": regime_name,
            })

            stage08_dataset_summary.append(summary)

print("Datasets BAR contruidos")
print("=" * 80)
print(f"Cantidad de datasets BAR: {len([k for k in stage08_datasets if k.startsw

```

Datasets BAR contruidos

=====
 Cantidad de datasets BAR: 12

8.5 Construcción de datasets OPC

In [101...

```

# 8.5 Construcción de datasets OPC
# =====

opc_feature_set_names = [
    "OPC_full",
    "OPC_reduced_level",
    "OPC_reduced_no_level",
]

for target in opc_targets_stage08:
    for feature_set_name in opc_feature_set_names:
        for regime_name, regime_filter in regime_versions.items():

            dataset_name = f"{target}__{feature_set_name}__{regime_name}"

            df_dataset, summary = build_supervised_dataset(
                df=df_stage08_base,
                target_col=target,
                feature_cols=feature_sets_stage08[feature_set_name],
                metadata_cols=metadata_cols_stage08,

```

```

        regime_filter=regime_filter,
        dropna=True,
    )

    stage08_datasets[dataset_name] = df_dataset

    summary.update({
        "dataset_name": dataset_name,
        "target_type": "OPC",
        "feature_set_name": feature_set_name,
        "regime_name": regime_name,
    })

    stage08_dataset_summary.append(summary)

print("Datasets OPC contruidos")
print("=" * 80)
print(f"Total datasets contruidos: {len(stage08_datasets)}")

```

Datasets OPC contruidos

=====

Total datasets contruidos: 30

8.6 Resumen de datasets contruidos

In [102...

```

# 8.6 Resumen de datasets contruidos
# =====

df_stage08_dataset_summary = pd.DataFrame(stage08_dataset_summary)

cols_order = [
    "dataset_name",
    "target_type",
    "target",
    "feature_set_name",
    "regime_name",
    "regime_filter",
    "n_features",
    "rows_before_dropna",
    "rows_after_dropna",
    "rows_removed_by_dropna",
]

df_stage08_dataset_summary = df_stage08_dataset_summary[cols_order]

print("Resumen de datasets predictivos contruidos")
print("=" * 80)

print(df_stage08_dataset_summary)

print("\nCantidad total de datasets:")
print(len(stage08_datasets))

```

Resumen de datasets predictivos construidos

	dataset_name	target_type	\
0	bar_p50_h30_tp15_sl10_BAR_full_all_regimes	BAR	
1	bar_p50_h30_tp15_sl10_BAR_full_regime_3	BAR	
2	bar_p50_h30_tp15_sl10_BAR_reduced_all_regimes	BAR	
3	bar_p50_h30_tp15_sl10_BAR_reduced_regime_3	BAR	
4	bar_p50_h60_tp15_sl10_BAR_full_all_regimes	BAR	
5	bar_p50_h60_tp15_sl10_BAR_full_regime_3	BAR	
6	bar_p50_h60_tp15_sl10_BAR_reduced_all_regimes	BAR	
7	bar_p50_h60_tp15_sl10_BAR_reduced_regime_3	BAR	
8	bar_p60_h60_tp15_sl10_BAR_full_all_regimes	BAR	
9	bar_p60_h60_tp15_sl10_BAR_full_regime_3	BAR	
10	bar_p60_h60_tp15_sl10_BAR_reduced_all_regimes	BAR	
11	bar_p60_h60_tp15_sl10_BAR_reduced_regime_3	BAR	
12	opc_p50_h90_tp15_sl10_OPC_full_all_regimes	OPC	
13	opc_p50_h90_tp15_sl10_OPC_full_regime_3	OPC	
14	opc_p50_h90_tp15_sl10_OPC_reduced_level_all...	OPC	
15	opc_p50_h90_tp15_sl10_OPC_reduced_level_regi...	OPC	
16	opc_p50_h90_tp15_sl10_OPC_reduced_no_level_a...	OPC	
17	opc_p50_h90_tp15_sl10_OPC_reduced_no_level_r...	OPC	
18	opc_p40_h60_tp15_sl10_OPC_full_all_regimes	OPC	
19	opc_p40_h60_tp15_sl10_OPC_full_regime_3	OPC	
20	opc_p40_h60_tp15_sl10_OPC_reduced_level_all...	OPC	
21	opc_p40_h60_tp15_sl10_OPC_reduced_level_regi...	OPC	
22	opc_p40_h60_tp15_sl10_OPC_reduced_no_level_a...	OPC	
23	opc_p40_h60_tp15_sl10_OPC_reduced_no_level_r...	OPC	
24	opc_p50_h60_tp15_sl10_OPC_full_all_regimes	OPC	
25	opc_p50_h60_tp15_sl10_OPC_full_regime_3	OPC	
26	opc_p50_h60_tp15_sl10_OPC_reduced_level_all...	OPC	
27	opc_p50_h60_tp15_sl10_OPC_reduced_level_regi...	OPC	
28	opc_p50_h60_tp15_sl10_OPC_reduced_no_level_a...	OPC	
29	opc_p50_h60_tp15_sl10_OPC_reduced_no_level_r...	OPC	

	target	feature_set_name	regime_name	regime_filter	\
0	bar_p50_h30_tp15_sl10	BAR_full	all_regimes	NaN	
1	bar_p50_h30_tp15_sl10	BAR_full	regime_3	3.0	
2	bar_p50_h30_tp15_sl10	BAR_reduced	all_regimes	NaN	
3	bar_p50_h30_tp15_sl10	BAR_reduced	regime_3	3.0	
4	bar_p50_h60_tp15_sl10	BAR_full	all_regimes	NaN	
5	bar_p50_h60_tp15_sl10	BAR_full	regime_3	3.0	
6	bar_p50_h60_tp15_sl10	BAR_reduced	all_regimes	NaN	
7	bar_p50_h60_tp15_sl10	BAR_reduced	regime_3	3.0	
8	bar_p60_h60_tp15_sl10	BAR_full	all_regimes	NaN	
9	bar_p60_h60_tp15_sl10	BAR_full	regime_3	3.0	
10	bar_p60_h60_tp15_sl10	BAR_reduced	all_regimes	NaN	
11	bar_p60_h60_tp15_sl10	BAR_reduced	regime_3	3.0	
12	opc_p50_h90_tp15_sl10	OPC_full	all_regimes	NaN	
13	opc_p50_h90_tp15_sl10	OPC_full	regime_3	3.0	
14	opc_p50_h90_tp15_sl10	OPC_reduced_level	all_regimes	NaN	
15	opc_p50_h90_tp15_sl10	OPC_reduced_level	regime_3	3.0	
16	opc_p50_h90_tp15_sl10	OPC_reduced_no_level	all_regimes	NaN	
17	opc_p50_h90_tp15_sl10	OPC_reduced_no_level	regime_3	3.0	
18	opc_p40_h60_tp15_sl10	OPC_full	all_regimes	NaN	
19	opc_p40_h60_tp15_sl10	OPC_full	regime_3	3.0	
20	opc_p40_h60_tp15_sl10	OPC_reduced_level	all_regimes	NaN	
21	opc_p40_h60_tp15_sl10	OPC_reduced_level	regime_3	3.0	
22	opc_p40_h60_tp15_sl10	OPC_reduced_no_level	all_regimes	NaN	
23	opc_p40_h60_tp15_sl10	OPC_reduced_no_level	regime_3	3.0	
24	opc_p50_h60_tp15_sl10	OPC_full	all_regimes	NaN	

25	opc_p50_h60_tp15_sl10	OPC_full	regime_3	3.0
26	opc_p50_h60_tp15_sl10	OPC_reduced_level	all_regimes	NaN
27	opc_p50_h60_tp15_sl10	OPC_reduced_level	regime_3	3.0
28	opc_p50_h60_tp15_sl10	OPC_reduced_no_level	all_regimes	NaN
29	opc_p50_h60_tp15_sl10	OPC_reduced_no_level	regime_3	3.0

	n_features	rows_before_dropna	rows_after_dropna	rows_removed_by_dropna
0	22	1024062	486761	537301
1	22	444600	239801	204799
2	8	1024062	487435	536627
3	8	444600	239801	204799
4	22	1024062	464458	559604
5	22	444600	215022	229578
6	8	1024062	465102	558960
7	8	444600	215022	229578
8	22	1024062	382185	641877
9	22	444600	176358	268242
10	8	1024062	382693	641369
11	8	444600	176358	268242
12	18	1024062	757302	266760
13	18	444600	357162	87438
14	8	1024062	757302	266760
15	8	444600	357162	87438
16	7	1024062	757302	266760
17	7	444600	357162	87438
18	18	1024062	801762	222300
19	18	444600	401622	42978
20	8	1024062	801762	222300
21	8	444600	401622	42978
22	7	1024062	801762	222300
23	7	444600	401622	42978
24	18	1024062	801762	222300
25	18	444600	401622	42978
26	8	1024062	801762	222300
27	8	444600	401622	42978
28	7	1024062	801762	222300
29	7	444600	401622	42978

Cantidad total de datasets:

30

8.7.

In [103...

```
# 8.7 Validación de distribución de clases por dataset y split
# =====

def summarize_class_distribution(stage08_datasets, df_dataset_summary):
    """
    Calcula distribución de clases para cada dataset construido,
    tanto global como por dataset_split.
    """
    rows = []
    balance_rows = []

    summary_lookup = (
        df_dataset_summary
        .set_index("dataset_name")
        .to_dict(orient="index")
    )
```

```

for dataset_name, df_tmp in stage08_datasets.items():
    info = summary_lookup[dataset_name]
    target_col = info["target"]

    splits_to_check = ["ALL"] + sorted(df_tmp["dataset_split"].dropna().unique())

    for split_name in splits_to_check:
        if split_name == "ALL":
            df_split = df_tmp.copy()
        else:
            df_split = df_tmp[df_tmp["dataset_split"] == split_name].copy()

        counts = df_split[target_col].value_counts(dropna=False).sort_index()
        ratios = counts / counts.sum()

        for class_value in counts.index:
            rows.append({
                "dataset_name": dataset_name,
                "target_type": info["target_type"],
                "target": target_col,
                "feature_set_name": info["feature_set_name"],
                "regime_name": info["regime_name"],
                "dataset_split": split_name,
                "class_value": class_value,
                "class_count": counts.loc[class_value],
                "class_ratio": ratios.loc[class_value],
                "total_rows": len(df_split),
            })

        min_ratio = ratios.min()
        max_ratio = ratios.max()

        balance_rows.append({
            "dataset_name": dataset_name,
            "target_type": info["target_type"],
            "target": target_col,
            "feature_set_name": info["feature_set_name"],
            "regime_name": info["regime_name"],
            "dataset_split": split_name,
            "total_rows": len(df_split),
            "n_classes": counts.shape[0],
            "min_class_ratio": min_ratio,
            "max_class_ratio": max_ratio,
            "imbalance_ratio": max_ratio / min_ratio if min_ratio > 0 else n

        })

    df_class_distribution = pd.DataFrame(rows)
    df_class_balance_summary = pd.DataFrame(balance_rows)

    return df_class_distribution, df_class_balance_summary

df_stage08_class_distribution, df_stage08_class_balance_summary = summarize_class(
    stage08_datasets=stage08_datasets,
    df_dataset_summary=df_stage08_dataset_summary,
)

print("Distribución de clases calculada")
print("=" * 80)

```

```
print(f"Detalle distribución: {df_stage08_class_distribution.shape}")
print(f"Resumen balance:      {df_stage08_class_balance_summary.shape}")
```

Distribución de clases calculada

=====

Detalle distribución: (378, 10)

Resumen balance: (90, 11)

In [104...

```
# 8.7.1 Resumen compacto de balance por dataset
# =====

cols_view = [
    "dataset_name",
    "target_type",
    "target",
    "feature_set_name",
    "regime_name",
    "dataset_split",
    "total_rows",
    "n_classes",
    "min_class_ratio",
    "max_class_ratio",
    "imbalance_ratio",
]

print("Resumen compacto de balance de clases")
print("=" * 80)

print(
    df_stage08_class_balance_summary[cols_view]
    .sort_values(["target_type", "target", "feature_set_name", "regime_name", "d
])
```

Resumen compacto de balance de clases

```
=====
dataset_name target_type \
0 bar_p50_h30_tp15_sl10__BAR_full_all_regimes BAR
1 bar_p50_h30_tp15_sl10__BAR_full_all_regimes BAR
2 bar_p50_h30_tp15_sl10__BAR_full_all_regimes BAR
3 bar_p50_h30_tp15_sl10__BAR_full_regime_3 BAR
4 bar_p50_h30_tp15_sl10__BAR_full_regime_3 BAR
.. ...
49 opc_p50_h90_tp15_sl10__OPC_reduced_no_level_a... OPC
50 opc_p50_h90_tp15_sl10__OPC_reduced_no_level_a... OPC
51 opc_p50_h90_tp15_sl10__OPC_reduced_no_level_r... OPC
52 opc_p50_h90_tp15_sl10__OPC_reduced_no_level_r... OPC
53 opc_p50_h90_tp15_sl10__OPC_reduced_no_level_r... OPC

target feature_set_name regime_name \
0 bar_p50_h30_tp15_sl10 BAR_full all_regimes
1 bar_p50_h30_tp15_sl10 BAR_full all_regimes
2 bar_p50_h30_tp15_sl10 BAR_full all_regimes
3 bar_p50_h30_tp15_sl10 BAR_full regime_3
4 bar_p50_h30_tp15_sl10 BAR_full regime_3
.. ...
49 opc_p50_h90_tp15_sl10 OPC_reduced_no_level all_regimes
50 opc_p50_h90_tp15_sl10 OPC_reduced_no_level all_regimes
51 opc_p50_h90_tp15_sl10 OPC_reduced_no_level regime_3
52 opc_p50_h90_tp15_sl10 OPC_reduced_no_level regime_3
53 opc_p50_h90_tp15_sl10 OPC_reduced_no_level regime_3

dataset_split total_rows n_classes min_class_ratio \
0 ALL 486761 3 0.058014
1 development_2020_2024 362063 3 0.045503
2 final_test_2025_2026 124698 3 0.094340
3 ALL 239801 3 0.054649
4 development_2020_2024 179346 3 0.042794
.. ...
49 development_2020_2024 605535 5 0.012293
50 final_test_2025_2026 151767 5 0.036042
51 ALL 357162 5 0.014072
52 development_2020_2024 285585 5 0.009839
53 final_test_2025_2026 71577 5 0.030024

max_class_ratio imbalance_ratio
0 0.539495 9.299373
1 0.515341 11.325402
2 0.609625 6.462003
3 0.534230 9.775582
4 0.508548 11.883518
.. ...
49 0.709876 57.745164
50 0.486983 13.511517
51 0.690597 49.075806
52 0.731113 74.304270
53 0.528941 17.617497
```

[90 rows x 11 columns]

In [105... *# 8.7.2 Datasets con mayor desbalance*

```
# =====
print("Datasets más desbalanceados")
```

```
print("=" * 80)

print(
    df_stage08_class_balance_summary
    .sort_values("imbalance_ratio", ascending=False)
    .head(30)[cols_view]
)

print("\nDatasets con clase minoritaria menor al 5%:")
print("=" * 80)

print(
    df_stage08_class_balance_summary[
        df_stage08_class_balance_summary["min_class_ratio"] < 0.05
    ][cols_view]
    .sort_values("min_class_ratio")
)
```


Datasets más desbalanceados

		dataset_name	target_type	\
46	opc_p50_h90_tp15_sl10__OPC_reduced_level_regi...		OPC	
40	opc_p50_h90_tp15_sl10__OPC_full_regime_3		OPC	
52	opc_p50_h90_tp15_sl10__OPC_reduced_no_level_r...		OPC	
82	opc_p50_h60_tp15_sl10__OPC_reduced_level_regi...		OPC	
76	opc_p50_h60_tp15_sl10__OPC_full_regime_3		OPC	
88	opc_p50_h60_tp15_sl10__OPC_reduced_no_level_r...		OPC	
79	opc_p50_h60_tp15_sl10__OPC_reduced_level_all...		OPC	
73	opc_p50_h60_tp15_sl10__OPC_full_all_regimes		OPC	
85	opc_p50_h60_tp15_sl10__OPC_reduced_no_level_a...		OPC	
49	opc_p50_h90_tp15_sl10__OPC_reduced_no_level_a...		OPC	
37	opc_p50_h90_tp15_sl10__OPC_full_all_regimes		OPC	
43	opc_p50_h90_tp15_sl10__OPC_reduced_level_all...		OPC	
51	opc_p50_h90_tp15_sl10__OPC_reduced_no_level_r...		OPC	
45	opc_p50_h90_tp15_sl10__OPC_reduced_level_regi...		OPC	
39	opc_p50_h90_tp15_sl10__OPC_full_regime_3		OPC	
87	opc_p50_h60_tp15_sl10__OPC_reduced_no_level_r...		OPC	
75	opc_p50_h60_tp15_sl10__OPC_full_regime_3		OPC	
81	opc_p50_h60_tp15_sl10__OPC_reduced_level_regi...		OPC	
78	opc_p50_h60_tp15_sl10__OPC_reduced_level_all...		OPC	
72	opc_p50_h60_tp15_sl10__OPC_full_all_regimes		OPC	
84	opc_p50_h60_tp15_sl10__OPC_reduced_no_level_a...		OPC	
48	opc_p50_h90_tp15_sl10__OPC_reduced_no_level_a...		OPC	
36	opc_p50_h90_tp15_sl10__OPC_full_all_regimes		OPC	
42	opc_p50_h90_tp15_sl10__OPC_reduced_level_all...		OPC	
58	opc_p40_h60_tp15_sl10__OPC_full_regime_3		OPC	
64	opc_p40_h60_tp15_sl10__OPC_reduced_level_regi...		OPC	
70	opc_p40_h60_tp15_sl10__OPC_reduced_no_level_r...		OPC	
61	opc_p40_h60_tp15_sl10__OPC_reduced_level_all...		OPC	
55	opc_p40_h60_tp15_sl10__OPC_full_all_regimes		OPC	
67	opc_p40_h60_tp15_sl10__OPC_reduced_no_level_a...		OPC	
		target	feature_set_name	regime_name \
46	opc_p50_h90_tp15_sl10	OPC_reduced_level	regime_3	
40	opc_p50_h90_tp15_sl10	OPC_full	regime_3	
52	opc_p50_h90_tp15_sl10	OPC_reduced_no_level	regime_3	
82	opc_p50_h60_tp15_sl10	OPC_reduced_level	regime_3	
76	opc_p50_h60_tp15_sl10	OPC_full	regime_3	
88	opc_p50_h60_tp15_sl10	OPC_reduced_no_level	regime_3	
79	opc_p50_h60_tp15_sl10	OPC_reduced_level	all_regimes	
73	opc_p50_h60_tp15_sl10	OPC_full	all_regimes	
85	opc_p50_h60_tp15_sl10	OPC_reduced_no_level	all_regimes	
49	opc_p50_h90_tp15_sl10	OPC_reduced_no_level	all_regimes	
37	opc_p50_h90_tp15_sl10	OPC_full	all_regimes	
43	opc_p50_h90_tp15_sl10	OPC_reduced_level	all_regimes	
51	opc_p50_h90_tp15_sl10	OPC_reduced_no_level	regime_3	
45	opc_p50_h90_tp15_sl10	OPC_reduced_level	regime_3	
39	opc_p50_h90_tp15_sl10	OPC_full	regime_3	
87	opc_p50_h60_tp15_sl10	OPC_reduced_no_level	regime_3	
75	opc_p50_h60_tp15_sl10	OPC_full	regime_3	
81	opc_p50_h60_tp15_sl10	OPC_reduced_level	regime_3	
78	opc_p50_h60_tp15_sl10	OPC_reduced_level	all_regimes	
72	opc_p50_h60_tp15_sl10	OPC_full	all_regimes	
84	opc_p50_h60_tp15_sl10	OPC_reduced_no_level	all_regimes	
48	opc_p50_h90_tp15_sl10	OPC_reduced_no_level	all_regimes	
36	opc_p50_h90_tp15_sl10	OPC_full	all_regimes	
42	opc_p50_h90_tp15_sl10	OPC_reduced_level	all_regimes	
58	opc_p40_h60_tp15_sl10	OPC_full	regime_3	

64	opc_p40_h60_tp15_sl10	OPC_reduced_level	regime_3
70	opc_p40_h60_tp15_sl10	OPC_reduced_no_level	regime_3
61	opc_p40_h60_tp15_sl10	OPC_reduced_level	all_regimes
55	opc_p40_h60_tp15_sl10	OPC_full	all_regimes
67	opc_p40_h60_tp15_sl10	OPC_reduced_no_level	all_regimes

	dataset_split	total_rows	n_classes	min_class_ratio	\
46	development_2020_2024	285585	5	0.009839	
40	development_2020_2024	285585	5	0.009839	
52	development_2020_2024	285585	5	0.009839	
82	development_2020_2024	321135	5	0.009877	
76	development_2020_2024	321135	5	0.009877	
88	development_2020_2024	321135	5	0.009877	
79	development_2020_2024	641085	5	0.010775	
73	development_2020_2024	641085	5	0.010775	
85	development_2020_2024	641085	5	0.010775	
49	development_2020_2024	605535	5	0.012293	
37	development_2020_2024	605535	5	0.012293	
43	development_2020_2024	605535	5	0.012293	
51	ALL	357162	5	0.014072	
45	ALL	357162	5	0.014072	
39	ALL	357162	5	0.014072	
87	ALL	401622	5	0.014568	
75	ALL	401622	5	0.014568	
81	ALL	401622	5	0.014568	
78	ALL	801762	5	0.015740	
72	ALL	801762	5	0.015740	
84	ALL	801762	5	0.015740	
48	ALL	757302	5	0.017389	
36	ALL	757302	5	0.017389	
42	ALL	757302	5	0.017389	
58	development_2020_2024	321135	5	0.017167	
64	development_2020_2024	321135	5	0.017167	
70	development_2020_2024	321135	5	0.017167	
61	development_2020_2024	641085	5	0.018110	
55	development_2020_2024	641085	5	0.018110	
67	development_2020_2024	641085	5	0.018110	

	max_class_ratio	imbalance_ratio
46	0.731113	74.304270
40	0.731113	74.304270
52	0.731113	74.304270
82	0.733212	74.230769
76	0.733212	74.230769
88	0.733212	74.230769
79	0.719451	66.767371
73	0.719451	66.767371
85	0.719451	66.767371
49	0.709876	57.745164
37	0.709876	57.745164
43	0.709876	57.745164
51	0.690597	49.075806
45	0.690597	49.075806
39	0.690597	49.075806
87	0.692709	47.548624
75	0.692709	47.548624
81	0.692709	47.548624
78	0.675234	42.898336
72	0.675234	42.898336
84	0.675234	42.898336

48	0.665208	38.253702
36	0.665208	38.253702
42	0.665208	38.253702
58	0.638980	37.220932
64	0.638980	37.220932
70	0.638980	37.220932
61	0.624195	34.467011
55	0.624195	34.467011
67	0.624195	34.467011

Datasets con clase minoritaria menor al 5%:

=====

	dataset_name	target_type	\
46	opc_p50_h90_tp15_sl10__OPC_reduced_level_regi...	OPC	
40	opc_p50_h90_tp15_sl10__OPC_full_regime_3	OPC	
52	opc_p50_h90_tp15_sl10__OPC_reduced_no_level_r...	OPC	
82	opc_p50_h60_tp15_sl10__OPC_reduced_level_regi...	OPC	
76	opc_p50_h60_tp15_sl10__OPC_full_regime_3	OPC	
..	...	...	
59	opc_p40_h60_tp15_sl10__OPC_full_regime_3	OPC	
71	opc_p40_h60_tp15_sl10__OPC_reduced_no_level_r...	OPC	
56	opc_p40_h60_tp15_sl10__OPC_full_all_regimes	OPC	
68	opc_p40_h60_tp15_sl10__OPC_reduced_no_level_a...	OPC	
62	opc_p40_h60_tp15_sl10__OPC_reduced_level_all...	OPC	

	target	feature_set_name	regime_name	\
46	opc_p50_h90_tp15_sl10	OPC_reduced_level	regime_3	
40	opc_p50_h90_tp15_sl10	OPC_full	regime_3	
52	opc_p50_h90_tp15_sl10	OPC_reduced_no_level	regime_3	
82	opc_p50_h60_tp15_sl10	OPC_reduced_level	regime_3	
76	opc_p50_h60_tp15_sl10	OPC_full	regime_3	
..	...	...	...	
59	opc_p40_h60_tp15_sl10	OPC_full	regime_3	
71	opc_p40_h60_tp15_sl10	OPC_reduced_no_level	regime_3	
56	opc_p40_h60_tp15_sl10	OPC_full	all_regimes	
68	opc_p40_h60_tp15_sl10	OPC_reduced_no_level	all_regimes	
62	opc_p40_h60_tp15_sl10	OPC_reduced_level	all_regimes	

	dataset_split	total_rows	n_classes	min_class_ratio	\
46	development_2020_2024	285585	5	0.009839	
40	development_2020_2024	285585	5	0.009839	
52	development_2020_2024	285585	5	0.009839	
82	development_2020_2024	321135	5	0.009877	
76	development_2020_2024	321135	5	0.009877	
..	...	...	...	...	
59	final_test_2025_2026	80487	5	0.045548	
71	final_test_2025_2026	80487	5	0.045548	
56	final_test_2025_2026	160677	5	0.048240	
68	final_test_2025_2026	160677	5	0.048240	
62	final_test_2025_2026	160677	5	0.048240	

	max_class_ratio	imbalance_ratio
46	0.731113	74.304270
40	0.731113	74.304270
52	0.731113	74.304270
82	0.733212	74.230769
76	0.733212	74.230769
..	...	...
59	0.438245	9.621658
71	0.438245	9.621658

```

56      0.403611      8.366791
68      0.403611      8.366791
62      0.403611      8.366791

```

[70 rows x 11 columns]

```

In [106... # 8.7.3 Distribución detallada de clases
# =====

print("Distribución detallada de clases")
print("=" * 80)

print(
    df_stage08_class_distribution
    .sort_values(["dataset_name", "dataset_split", "class_value"])
)

```

Distribución detallada de clases

```

=====
                                dataset_name target_type \
0      bar_p50_h30_tp15_sl10__BAR_full__all_regimes      BAR
1      bar_p50_h30_tp15_sl10__BAR_full__all_regimes      BAR
2      bar_p50_h30_tp15_sl10__BAR_full__all_regimes      BAR
3      bar_p50_h30_tp15_sl10__BAR_full__all_regimes      BAR
4      bar_p50_h30_tp15_sl10__BAR_full__all_regimes      BAR
..
193  opc_p50_h90_tp15_sl10__OPC_reduced_no_level__r...      OPC
194  opc_p50_h90_tp15_sl10__OPC_reduced_no_level__r...      OPC
195  opc_p50_h90_tp15_sl10__OPC_reduced_no_level__r...      OPC
196  opc_p50_h90_tp15_sl10__OPC_reduced_no_level__r...      OPC
197  opc_p50_h90_tp15_sl10__OPC_reduced_no_level__r...      OPC

```

```

                                target      feature_set_name regime_name \
0      bar_p50_h30_tp15_sl10      BAR_full  all_regimes
1      bar_p50_h30_tp15_sl10      BAR_full  all_regimes
2      bar_p50_h30_tp15_sl10      BAR_full  all_regimes
3      bar_p50_h30_tp15_sl10      BAR_full  all_regimes
4      bar_p50_h30_tp15_sl10      BAR_full  all_regimes
..
193  opc_p50_h90_tp15_sl10  OPC_reduced_no_level  regime_3
194  opc_p50_h90_tp15_sl10  OPC_reduced_no_level  regime_3
195  opc_p50_h90_tp15_sl10  OPC_reduced_no_level  regime_3
196  opc_p50_h90_tp15_sl10  OPC_reduced_no_level  regime_3
197  opc_p50_h90_tp15_sl10  OPC_reduced_no_level  regime_3

```

```

                                dataset_split class_value class_count class_ratio total_rows
0                                ALL          -1.0         28239      0.058014      486761
1                                ALL           0.0        195917      0.402491      486761
2                                ALL           1.0        262605      0.539495      486761
3      development_2020_2024          -1.0         16475      0.045503      362063
4      development_2020_2024           0.0        159002      0.439156      362063
..
193  final_test_2025_2026           0.0         37860      0.528941       71577
194  final_test_2025_2026           1.0         12633      0.176495       71577
195  final_test_2025_2026           2.0          2488      0.034760       71577
196  final_test_2025_2026           3.0         16447      0.229781       71577
197  final_test_2025_2026           4.0          2149      0.030024       71577

```

[378 rows x 10 columns]

Observaciones sobre distribución de clases

- Los datasets se construyeron correctamente, pero hay desbalance importante.
- BAR tiene desbalance moderado. No es ideal, pero es trabajable. Ejemplo: bar_p50_h30 en development tiene clase minoritaria de 4.55% y clase mayoritaria de 51.5%.
- OPC tiene desbalance fuerte/severo. En varios datasets OPC, la clase minoritaria está cerca de 1% en development, especialmente en regime_3.
- El desbalance más crítico aparece en: opc_p50_h90_tp15_sl10
opc_p50_h60_tp15_sl10
- En OPC, una clase dominante llega a representar 70%–73% en algunos casos de development/regime_3. Eso puede hacer que un modelo aparente buen accuracy sin aprender bien las clases minoritarias.
- Las versiones full/reduced/reduced_no_level casi no cambian la distribución de clases. Eso confirma que el desbalance viene del target, no del set de features.
- BAR sigue siendo más limpio para empezar modelado.
- OPC sigue siendo interesante, pero necesitará métricas robustas y manejo de desbalance.

Conclusión directa:

- BAR queda apto para modelado inicial.
- OPC no se descarta, pero queda marcado como target avanzado con desbalance severo.
- En Stage 06 no debemos usar accuracy como métrica principal.
- Hay que usar balanced accuracy, macro F1, matriz de confusión y class weights.

8.8 Guardado de datasets predictivos

In [107...

```
# 8.8 Guardado de datasets predictivos Stage 05
# =====

from pathlib import Path
import json
import numpy as np
import pandas as pd

# Carpeta principal Stage 05
STAGE_05_PATH = PROJECT_ROOT / "data" / "05_mnq_features"

# Subcarpetas
STAGE_05_DATASETS_PATH = STAGE_05_PATH / "predictive_datasets"
STAGE_05_SUMMARY_PATH = STAGE_05_PATH / "summaries"
STAGE_05_REGISTRY_PATH = STAGE_05_PATH / "registries"
```

```
# Crear carpetas
STAGE_05_DATASETS_PATH.mkdir(parents=True, exist_ok=True)
STAGE_05_SUMMARY_PATH.mkdir(parents=True, exist_ok=True)
STAGE_05_REGISTRY_PATH.mkdir(parents=True, exist_ok=True)

print("Carpetas Stage 05 creadas/verificadas")
print("=" * 80)
print(f"Datasets: {STAGE_05_DATASETS_PATH}")
print(f"Summaries: {STAGE_05_SUMMARY_PATH}")
print(f"Registry: {STAGE_05_REGISTRY_PATH}")
```

Carpetas Stage 05 creadas/verificadas

```
=====
Datasets: c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\05_mnq_features\predictive_datasets
Summaries: c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\05_mnq_features\summaries
Registry: c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\05_mnq_features\registries
```

In [108...

```
# 8.8.1 Guardar Los 30 datasets predictivos
# =====

saved_dataset_paths = {}

for dataset_name, df_dataset in stage08_datasets.items():
    file_path = STAGE_05_DATASETS_PATH / f"{dataset_name}.parquet"

    df_dataset.to_parquet(file_path, index=True)

    saved_dataset_paths[dataset_name] = str(file_path)

print("Datasets predictivos guardados")
print("=" * 80)
print(f"Cantidad guardada: {len(saved_dataset_paths)}")

print("\nPrimeros archivos guardados:")
for name, path in list(saved_dataset_paths.items())[:10]:
    print(f"{name}")
    print(f"  -> {path}")
```

Datasets predictivos guardados

=====
Cantidad guardada: 30

Primeros archivos guardados:

bar_p50_h30_tp15_sl10__BAR_full__all_regimes

-> c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\05_mnq_features
\predictive_datasets\bar_p50_h30_tp15_sl10__BAR_full__all_regimes.parquet

bar_p50_h30_tp15_sl10__BAR_full__regime_3

-> c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\05_mnq_features
\predictive_datasets\bar_p50_h30_tp15_sl10__BAR_full__regime_3.parquet

bar_p50_h30_tp15_sl10__BAR_reduced__all_regimes

-> c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\05_mnq_features
\predictive_datasets\bar_p50_h30_tp15_sl10__BAR_reduced__all_regimes.parquet

bar_p50_h30_tp15_sl10__BAR_reduced__regime_3

-> c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\05_mnq_features
\predictive_datasets\bar_p50_h30_tp15_sl10__BAR_reduced__regime_3.parquet

bar_p50_h60_tp15_sl10__BAR_full__all_regimes

-> c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\05_mnq_features
\predictive_datasets\bar_p50_h60_tp15_sl10__BAR_full__all_regimes.parquet

bar_p50_h60_tp15_sl10__BAR_full__regime_3

-> c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\05_mnq_features
\predictive_datasets\bar_p50_h60_tp15_sl10__BAR_full__regime_3.parquet

bar_p50_h60_tp15_sl10__BAR_reduced__all_regimes

-> c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\05_mnq_features
\predictive_datasets\bar_p50_h60_tp15_sl10__BAR_reduced__all_regimes.parquet

bar_p50_h60_tp15_sl10__BAR_reduced__regime_3

-> c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\05_mnq_features
\predictive_datasets\bar_p50_h60_tp15_sl10__BAR_reduced__regime_3.parquet

bar_p60_h60_tp15_sl10__BAR_full__all_regimes

-> c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\05_mnq_features
\predictive_datasets\bar_p60_h60_tp15_sl10__BAR_full__all_regimes.parquet

bar_p60_h60_tp15_sl10__BAR_full__regime_3

-> c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\05_mnq_features
\predictive_datasets\bar_p60_h60_tp15_sl10__BAR_full__regime_3.parquet

In [109...

```
# 8.8.2 Guardar summaries del Stage 05
# =====

# Resumen de datasets construidos
df_stage08_dataset_summary.to_parquet(
    STAGE_05_SUMMARY_PATH / "stage05_predictive_datasets_summary.parquet",
    index=False,
)

df_stage08_dataset_summary.to_csv(
    STAGE_05_SUMMARY_PATH / "stage05_predictive_datasets_summary.csv",
    index=False,
)

# Distribución de clases
df_stage08_class_distribution.to_parquet(
    STAGE_05_SUMMARY_PATH / "stage05_class_distribution.parquet",
    index=False,
)

df_stage08_class_distribution.to_csv(
    STAGE_05_SUMMARY_PATH / "stage05_class_distribution.csv",
    index=False,
)
```

```

# Balance de clases
df_stage08_class_balance_summary.to_parquet(
    STAGE_05_SUMMARY_PATH / "stage05_class_balance_summary.parquet",
    index=False,
)

df_stage08_class_balance_summary.to_csv(
    STAGE_05_SUMMARY_PATH / "stage05_class_balance_summary.csv",
    index=False,
)

print("Summaries guardados")
print("=" * 80)
print(STAGE_05_SUMMARY_PATH)

```

Summaries guardados

```

=====
c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\05_mnq_features\summa
ries

```

In [110...

```

# 8.8.3 Guardar registros del Stage 05
# =====

def make_json_serializable(obj):
    """
    Convierte objetos numpy/pandas a tipos serializables en JSON.
    """
    if isinstance(obj, dict):
        return {key: make_json_serializable(value) for key, value in obj.items()}
    elif isinstance(obj, list):
        return [make_json_serializable(value) for value in obj]
    elif isinstance(obj, tuple):
        return [make_json_serializable(value) for value in obj]
    elif isinstance(obj, (np.integer,)):
        return int(obj)
    elif isinstance(obj, (np.floating,)):
        return float(obj)
    elif isinstance(obj, (np.ndarray,)):
        return obj.tolist()
    elif pd.isna(obj) if isinstance(obj, (float, np.floating)) else False:
        return None
    else:
        return obj

stage05_final_registry = {
    "selected_targets_stage05": selected_targets_stage05,
    "stage05_feature_sets_registry": stage05_feature_sets_registry,
    "stage05_signal_decision_registry": stage05_signal_decision_registry,
    "stage05_signal_analysis_registry": stage05_signal_analysis_registry,
    "saved_dataset_paths": saved_dataset_paths,
    "n_datasets_saved": len(saved_dataset_paths),
}

stage05_final_registry = make_json_serializable(stage05_final_registry)

registry_path = STAGE_05_REGISTRY_PATH / "stage05_final_registry.json"

with registry_path.open("w", encoding="utf-8") as f:

```



```

    json.dump(
        stage05_final_registry,
        f,
        indent=2,
        ensure_ascii=False,
    )

print("Registro final Stage 05 guardado")
print("=" * 80)
print(registry_path)

```

Registro final Stage 05 guardado

```

=====
c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\05_mnq_features\registries\stage05_final_registry.json

```

In [111...

```

# 8.8.4 Verificación final de archivos guardados
# =====

saved_parquet_files = sorted(STAGE_05_DATASETS_PATH.glob("*.parquet"))
saved_summary_files = sorted(STAGE_05_SUMMARY_PATH.glob("*"))
saved_registry_files = sorted(STAGE_05_REGISTRY_PATH.glob("*"))

print("Verificación final de guardado")
print("=" * 80)

print(f"Datasets parquet guardados: {len(saved_parquet_files)}")
print(f"Archivos summary guardados: {len(saved_summary_files)}")
print(f"Archivos registry guardados: {len(saved_registry_files)}")

print("\nRuta principal Stage 05:")
print(STAGE_05_PATH)

```

Verificación final de guardado

```

=====
Datasets parquet guardados: 30
Archivos summary guardados: 6
Archivos registry guardados: 1

```

Ruta principal Stage 05:

```
c:\Users\heguu\OneDrive\Escritorio\neural_profit_local\data\05_mnq_features
```

Con esto queda cerrado el punto 8 a nivel operativo: los datasets predictivos BAR y OPC quedan guardados y listos para el Stage 06.

En el Stage 06 ya podremos entrenar modelos comparando:

- BAR_full vs BAR_reduced
- OPC_full vs OPC_reduced_level vs OPC_reduced_no_level
- all_regimes vs regime_3